

Digitale Harmonie aus historischer Dissonanz

Extraktion, Ordnung und Analyse
unstrukturierter Archivdaten
des Männerchors Murg

Sven Burkhardt

 0009-0001-4954-4426

 17-056-912

 15.08.2025



University
of Basel



Digital
Humanities
Lab

University of Basel
Digital Humanities Lab
Switzerland

Diese Arbeit befasst sich mit dem Archiv des *Männerchor Murg* in den Jahren des Zweiten Weltkrieges. Hierfür wird eine automatisierte Pipeline auf Basis von LLMs und Pattern-matching vorgestellt, mit deren Hilfe Named Entities extrahiert und weiterverarbeitet werden. Der Hauptfokus der Weiterverarbeitung liegt auf der Ausgestaltung des Entity Matching, um erkannte Entitäten mit einer Ground Truth-Tabelle abzugleichen. Ziel ist es, dieses Archiv digital zugänglich, die beteiligten Personen sowie deren Netzwerke und deren geographische Ausdehnung sichtbar zu machen.

Inhaltsverzeichnis

1	Einleitung	2
2	Forschungsstand und Forschungslücke	5
3	Korpus	8
3.1	Quellen	8
3.1.1	Quellentradierung	8
3.1.2	Quellenbeschrieb	9
3.1.3	Sichtung & Kategorisierung zu Akten	10
3.2	Digitalisierung der Quellen	11
3.3	Transkription	12
3.3.1	Tagging mit Transkribus und LLM	15
4	Methodisches Vorgehen	17
4.1	LOD – Linked Open Data	17
4.1.1	Protégé	18
4.1.2	GraphDB	19
4.1.3	LOD-Ontologie	19
4.2	Wikidata	20
4.3	GeoNames	21
4.4	Nodegoat	22
4.5	Transkriptionen (Methodenvergleich)	24
4.5.1	Tesseract	24
4.5.2	LLM	25
4.5.3	Transkribus	26
4.6	Large Language Models	28
4.6.1	Msty	28
4.6.2	Alphabet – Gemini	29
4.6.3	Anthropic – Claude Code	29
4.6.4	OpenAI – ChatGPT	31
5	Pipeline	34
5.0.1	Pipeline im Detail	34
5.1	Vorverarbeitung	36

5.2	Hauptmodul – Transkribus_to_base	38
5.3	Module im Detail	41
5.3.1	document_schemas.py	41
5.3.2	person_matcher.py	44
5.3.3	Assigned_Roles_Module.py	56
5.3.4	place_matcher.py	58
5.3.5	organization_matcher.py	61
5.3.6	letter_metadata_matcher.py	62
5.3.7	type_matcher.py	69
5.3.8	unmatched_logger.py	72
5.3.9	validation_module.py	73
5.3.10	llm_enricher.py	74
6	Projekt-Webseite	75
7	Fazit und Ausblick	78
8	Redlichkeit	80
	Bibliographie	82
	Artikel	82
	Monografien	82
	Onlinequellen	83
	Software	85
	Vorträge und Manuskripte	85
A	Anhang	87
A.1	PDF_to_JPEG.py	87
A.2	Tagging in Transkribus	88
A.2.1	Strukturelle Tags	88
A.2.2	Inhaltliche Tags	89
A.3	Prompt der LLM Vorverarbeitung	91
A.4	Übersicht fehlerhaft verarbeiteter Listen in XML	93

1 Einleitung

Die klassische Geschichtsschreibung ist seit jeher geprägt von Erzählungen über grosse Persönlichkeiten, politische Umbrüche und militärische Ereignisse. Diese durch Herodot geprägte Perspektive hat sich lange Zeit als Leitparadigma historischer Forschung etabliert. Sie lenkte den Blick häufig auf Akteure und Geschehnisse, die als herausragend oder aussergewöhnlich gelten. Das tut diese Arbeit nicht. Mittlerweile hat sich ein Perspektivwechsel vollzogen. Zunehmend rückt der Alltag in den Fokus, ebenso wie die Frage, wie soziale Strukturen entstehen, wie politische und kulturelle Positionen ausgehandelt werden und welche Beziehungsnetzwerke das gesellschaftliche Leben prägen.¹

Die vorliegende Arbeit schliesst an diesen Ansatz an. Sie richtet den Blick auf ein lokales Akteursnetzwerk während des Zweiten Weltkriegs: den Männerchor Murg und sein Umfeld. Ziel ist es, anhand der archivalischen Überlieferung ein dichtes Geflecht sozialer, organisatorischer und räumlicher Beziehungen sichtbar zu machen – und damit ein Stück Alltags- und Vereinsgeschichte im Kontext einer politisch wie gesellschaftlich hochgradig verdichteten Epoche zu rekonstruieren.

Um diese mikrohistorische Perspektive zu realisieren, werden digitale Methoden in die Geschichtswissenschaften eingeführt. Diese Arbeit stützt sich auf die Auswertung benannter Entitäten, wie Personen, Orte und Organisationen. Umgesetzt wird dies durch Large Language Models (LLMs), „generative-pretrained-transformer“ Modellen und einer algorithmischen Weiterverarbeitung.

Dieses Projekt führt derart extrahierte digitale Daten mit den Ergebnissen klassischer Archivrecherchen zusammen. Es entsteht eine neuartige digitale Repräsentation und Analyse eines bislang unerforschten Korpus. Damit verbindet die zugrunde liegende Forschung bewährte Methoden der Geschichtswissenschaft mit innovativen Ansätzen der Digital Humanities und demonstriert exemplarisch das Potenzial dieser interdisziplinären Arbeitsweise.

Der Zweite Weltkrieg gilt als gut erforschte historische Epoche, insbesondere mit Blick auf die einleitend erwähnten grossen Erzählungen. Diese Arbeit richtet den Blick hingegen auf eine kleine Gruppe von Männern. Sie haben sich in einem Männerchor aus dem beschaulichen Dorf Murg organisiert, weit entfernt von den politischen Zentren der Macht. Die Analyse ihres Archivs erlaubt es, weit mehr als nur die Geschichte eines Vereins zu

¹eine Entwicklung die 1968 von Edward Thompson angestossen wurde. Edward P. Thompson und Edward Palmer Thompson [1968](#).

rekonstruieren: Sie eröffnet Einblicke in den dörflichen Alltag unter Kriegsbedingungen, in die Veränderungen des Vereinslebens, in lokale soziale Netzwerke und in die Funktion von Kulturvereinen als Träger kollektiver Identität. Gleichzeitig wird sichtbar, wie diese lokalen Akteure in die nationalsozialistische Kulturpolitik eingebunden waren, wie sie sich an staatliche Vorgaben anpassten und welche Auswirkungen Kriegswirtschaft und Mobilisierung auf den Kulturbetrieb hatten. Schliesslich lassen sich anhand der Dokumente auch individuelle Lebenswege nachzeichnen – einschliesslich militärischer Einsätze, privater Korrespondenzen und der Rolle einzelner Mitglieder im regionalen wie überregionalen Kultur- und Vereinsnetzwerk.

Die Arbeit liefert damit nicht nur einen Beitrag zur Geschichte des Männerchores Murg, sondern öffnet einen Zugang zu bislang unbeachteten Fragen der Mikrogeschichte im Kontext des Zweiten Weltkriegs. Gleichzeitig zeigt sie exemplarisch, wie digitale Methoden es erlauben, historische Quellenbestände neu zu erschliessen, zu vernetzen und aus unterschiedlichen Forschungsperspektiven auszuwerten.

Die Aufgabenstellung dieser Arbeit lässt sich in zwei wesentliche Bereiche gliedern. Der erste Bereich umfasst die Ermittlung einer verlässlichen Grundlage (Ground Truth). Durch intensive Archivrecherchen wird etwa rekonstruiert, wer die beteiligten Personen in den Dokumenten waren, welche Lebensdaten sie hatten, woher sie stammten und um welche Orte es sich dabei handelt. Besonders bei historischen Gebäuden wie Gaststätten, Lazaretten oder militärischen Stellungen ist ein erheblicher Aufwand nötig, um deren genaue Lage und Koordinaten zu bestimmen. Ergänzend erfolgen Recherchen zu Organisationen, wobei insbesondere die Identifikation militärischer Einheiten einen aussergewöhnlich hohen Rechercheaufwand erfordert.² Diese aufwendige Erschliessung ist nicht nur für die Vollständigkeit des Korpus relevant, sondern bildet auch die Grundlage für eine präzise historische Verortung der Ereignisse, Netzwerke und Akteursbeziehungen. Sie ermöglicht es, die Quellen in einen räumlichen und institutionellen Kontext einzubetten, wodurch sich mikrohistorische Zusammenhänge erst in ihrer vollen Tiefe erschliessen lassen.

Der zweite Bereich richtet den Fokus auf den technischen Aspekt der Arbeit. Hierfür wurde eine speziell entwickelte, mehrstufige Verarbeitungs-Pipeline erstellt. Sie stützt sich auf die Transkription der historischen Dokumente, die mittels [Transkribus](#)³ erstellt werden.

²Dies wird im Kapitel [Forschungsstand und Forschungslücke](#) ausgeführt.

³transkribus.org ist eine Plattform für die Erkennung, Transkription und Auswertung historischer Dokumente

Die dort produzierten Daten werden als PAGE-XML-Dateien⁴ exportiert und durch die in dieser Arbeit vorgestellte [Pipeline](#) weiterverarbeitet. Sie erzeugt daraus strukturierte Forschungsdaten. Es geht darum, wie diese gewonnenen Daten mithilfe digitaler Methoden automatisiert extrahiert und validiert werden können. So stellt sich die Frage:

Wie lassen sich aus historischen Dokumenten aus der Zeit des Zweiten Weltkriegs automatisiert und validiert Daten gewinnen, die Auskunft darüber geben, wer wann mit wem über welche Themen korrespondiert hat? Und wie kann visualisiert werden, in welchen Beziehungen diese Personen zueinander stehen – etwa durch gemeinsame Organisationszugehörigkeit oder Verwandtschaft?

Zur besseren Orientierung soll ein kurzer Überblick über die Arbeit gegeben werden. Einleitend liefert ein geografischer und historischer Kontext die Grundlage zur Einordnung des [Korpus](#). In der Korpusbeschreibung finden sich Informationen zu den historischen Quellen, deren Tradierung und Digitalisierung, sowie zur Auswahl des für diese Arbeit verwendeten Quellenkorpus.

Die Verarbeitung der Quellen stützt sich auf diverse digitale Prozessschritte. Hierfür werden digitale Tools verwendet, die im Kapitel [Methodisches Vorgehen](#) einerseits erläutert, aber auch eingeordnet und kritisiert werden. Um bessere Transparenz zu gewährleisten, werden auch Werkzeuge beschrieben, die getestet, aber nicht in die endgültige Pipeline übernommen wurden. Die Einsatzfelder reichen von der Datenproduktion, über die Datenspeicherung bis hin zur Datenvisualisierung.

Der Hauptteil der Arbeit ist die Beschreibung der Verarbeitungs-[Pipeline](#). Diese gliedert sich in ein Hauptmodul⁵ und diverse Untermodule. Letztere sind dafür zuständig, Entitäten wie Personen, Orte, Organisationen und Daten zu extrahieren und mittels der Ground Truth zu validieren, während das Hauptmodul die Verarbeitung dieser Schritte steuert.

Die produzierten Ergebnisse werden schlussendlich auf einer eigens produzierten [Projekt-Webseite](#) visualisiert, die den Lesenden zur interaktiven Erkundung der Ergebnisse dient. Hier können beispielsweise über einen IIIF-Viewer alle digitalisierten Dokumente betrachtet und die darin erkannten Entitäten miteinander verglichen werden.

⁴Pletschacher und Antonacopoulos 2010.

⁵[Hauptmodul – Transkribus_to_base](#)

Geografischer und historischer Kontext

Die vorliegende Arbeit stützt sich auf Unterlagen aus dem Archiv des „Männerchor Murg“ dessen Nachfolge im Jahr 2021 durch die „New Gospelsingers Murg“ angetreten wurde. Murg ist eine deutsche Gemeinde am Hochrhein, rund 30 km Luftlinie von Basel entfernt. Der Ort liegt am gleichnamigen Fluss Murg, der in den Rhein mündet. Beide Gewässer bildeten über Jahrhunderte hinweg den wirtschaftlichen Motor der Region: Die Wasserkraft der Murg begünstigte früh die Ansiedlung von Mühlen, Hammerwerken und Schmieden entlang des Flusslaufs. Der Rhein bietet dazu mit seiner Drahtseil-Fähre eine bedeutende Verkehrs- und Handelsverbindung, die bis zum Ersten Weltkrieg betrieben wurde.

Mit dem Ausbau der Landstrasse, der heutigen Bundesstrasse 34, sowie dem Anschluss an die Bahnstrecke Basel–Konstanz entwickelte sich Murg im 19. Jahrhundert von einer landwirtschaftlich geprägten Siedlung zu einer Gewerbe-, Handels- und Industriegemeinde. Die Wasserkraft wurde dabei zu einem entscheidenden Standortfaktor: Die Ansiedlung der Schweizer Textilfirma *Hüssy & Künzli AG* im Jahr 1853⁶ trug wesentlich zum wirtschaftlichen Wachstum der Gemeinde bei. Zahlreiche Arbeitskräfte, vor allem aus der benachbarten Schweiz, machten Murg zu einem wichtigen Standort der regionalen Textilindustrie. Es waren jene Schweizer Arbeiter, die den *Männerchor Murg* im Jahr 1861 als *Schweizer Männerchor Murg* gründeten. Es veranschaulicht den engen Zusammenhang zwischen wirtschaftlicher Migration, Industrialisierung und lokalem Vereinswesen. Diese historische Verflechtung bildet die zentrale Grundlage für die vorliegende Untersuchung.

2 Forschungsstand und Forschungslücke

Die vorliegende Arbeit knüpft an zwei Vorarbeiten an, die in den Jahren 2019 und 2022 am Departement Geschichte der Universität Basel durchgeführt wurden. In drei Digital-History-Seminaren⁷ mit Fokus auf Transkribus werden erste Teilbestände der „Männerchor Akten 1925–1944“ erschlossen und in einem Korpus von 137 Einzeldokumenten zusammengeführt.⁸ Ein kleinerer Korpus von rund 50 Dokumenten wird mit Metadaten versehen. Erfasst werden unter anderem die genaue Position im Ordner auf Seitenebene, Kurztitel und Entstehungsdatum. Diese Metadaten bilden die Grundlage für eine erstmalige systematische Erschliessung.

⁶vgl. [Geschichte Gemeinde Murg 2025](#).

⁷vgl. Hodel [2019](#); Serif [2019](#); Serif [2022](#).

⁸vgl. Burkhardt [2022](#).

Während in einem frühen Projektschritt vorrangig häufig genannte Personennamen („Carl Burger“, „Fritz Jung“) dokumentiert werden, richtet sich der Fokus im zweiten Schritt auf die Feldpost. Ziel ist es, über die Auswertung der Feldpostnummern Rückschlüsse auf beteiligte Militäreinheiten, deren Stationierungen und Verlagerungen während des Zweiten Weltkriegs zu ziehen.

Parallel zur inhaltlichen Erschliessungen entsteht 2022 eine erste digitale Storymap mit *ArcGIS*, die zentrale Ergebnisse des Projekts öffentlich zugänglich macht. Grundlage bildet die Sichtung, konservatorische Aufbereitung und Digitalisierung von zunächst rund 30 der etwa 800 Seiten Vereinsakten. Der Teilkorpus wird entheftet, gescannt und mit Metadaten wie Absender, Datum, Feldpostnummer und Einheit versehen. Da jedes Dokument einen anderen Verfasser aufweist, erfolgt die Transkription manuell. Eine automatische Handschriftenerkennung ist aufgrund der heterogenen Schriftbilder nicht praktikabel. Am Beispiel einzelner Sänger wie *Emil Durst* lässt sich durch die Rechercheergebnisse mithilfe der Feldpostnummern und ergänzender Kartenmaterialien der Aufenthaltsort bis auf wenige Meter⁹ oder auf Gebäudeebene genau rekonstruieren. Diese Erkenntnisse werden mit historischen Karten, Luftbildern und Ortsrecherchen verknüpft und in einer interaktiven ArcGIS-Karte visualisiert, die Stationierungen, Märsche und Frontverschiebungen der Chormitglieder anschaulich darstellt.

Die in diesen Vorprojekten erarbeiteten Listen, Geodaten, Transkriptionen und Visualisierungen fließen in die vorliegende Arbeit ein und bilden eine wesentliche Grundlage für die erweiterte, automatisierte Pipeline, die im Folgenden vorgestellt wird. Dazu gehören auch die Verbandsabzeichen, taktische Zeichen¹⁰ der jeweiligen Einheiten, die auch in die Ground Truth der vorliegenden Arbeit inkorporiert werden.

Abgesehen von diesen Vorarbeiten ist der Quellenkorpus wissenschaftlich unerschlossen. Mit dieser Arbeit liegt erstmals eine umfassendere wissenschaftliche Auswertung vor. Intensive Recherchen sind daher notwendig, um die vorliegende Arbeit zu untermauern. Hierfür kommen einschlägige Fachliteratur zu den jeweiligen Fachgebieten zum Einsatz. Es sind besonders die Bücher von Alex Buchner¹¹, Christian Hartmann¹², Werner

⁹im Fall von Militäreinheiten

¹⁰vgl. Haupt 1982, S.64-66.

¹¹vgl. Buchner 1989.

¹²vgl. Hartmann 2010.

Haupt¹³, Christoph Rass¹⁴, Georg Tessin¹⁵ und Christian Zentner¹⁶ zu nennen.

Darüber hinaus werden eigene Recherchen in den Beständen des *Bundesarchivs – Militärarchiv Freiburg*¹⁷ durchgeführt. Ergänzende Recherchen stammen aus den Suchlisten des *Deutschen Roten Kreuzes (DRK)*¹⁸. Hinzu kommen philatelistische Übersichts-Websites¹⁹, die bei der Entzifferung von Briefmarken und Stempeln helfen. Absolut essentiell für den Erfolg dieser Recherchen sind Citizen-Science-Foren²⁰. Sie ergänzen und validieren eigene Forschung.

Mit der notwendigen manuellen Recherche in oben dargelegten Datenbankstrukturen wird zugleich sichtbar, wie sehr es an Brücken fehlt, um unterschiedliche Klassifikationen, fachspezifische Ordnungslogiken und semantische Webtechnologien nachhaltig miteinander zu verbinden. Ein verhältnismässig einfaches Webscraping nach Informationen zu diesem Korpus ist nahezu unmöglich. Ausgeführt werden diese Probleme unter anderem bei Smiraglia und Scharnhorst (2021)²¹, die anhand konkreter Fallstudien verdeutlichen, wie fragmentiert semantische Strukturen bislang entwickelt werden und welche Hürden bei der praktischen Verknüpfung heterogener Wissensorganisationen bestehen. Dabei benennen sie insbesondere die Herausforderungen bei der Übersetzung historisch gewachsener Klassifikationen in standardisierte semantische Formate, die Notwendigkeit dauerhafter technischer Wartung und die Abhängigkeit von nachhaltigen Infrastruktur-Partnern²².

Für eine Einordnung zu historischen Netzwerkanalysen sei auf den Sammelband *Knoten und Kanten III*²³ verwiesen. Dieser verdeutlicht, dass die historische Netzwerkanalyse zwar von einem interdisziplinär etablierten Methodenkanon profitiert, jedoch nach wie vor substanziellen Herausforderungen gegenübersteht. Dazu zählen die Fragmentierung historischer Quellen, der hohe manuelle Erfassungsaufwand und methodische Desiderate im Umgang mit zeitlichen und räumlichen Dimensionen zu den erschwerenden Faktoren einer systematischen Erfassung relationaler Strukturen. Dennoch eröffnen netzwerkanalytische

¹³vgl. Haupt 1982.

¹⁴vgl. Rass und Rohrkamp 2009.

¹⁵vgl. Tessin 1977.

¹⁶vgl. Zentner 1983.

¹⁷Hollmann 2025.

¹⁸*DRK Suchdienst / Suche per Feldpostnummer* 2025.

¹⁹*Feldpost Number Database / GermanStamps.net* 2025.

²⁰vor Allem werden verwendet: *Forum der Wehrmacht* (*Forum Geschichte der Wehrmacht* 2025) und das *Lexikon der Wehrmacht* (Altenburger 2023).

²¹vgl. Richard und Andrea 2022.

²²vgl. ebd., Kap. 2 und 5.

²³Gamper und Reschke 2015.

Verfahren – besonders im Zusammenspiel mit relationaler Soziologie und Figurationsansätzen – neue Perspektiven auf Macht, Abhängigkeiten und Akteurskonstellationen in historischen Gesellschaften.

3 Korpus

Aus dem Bestand des Ordners „*Männerchor Akten 1925–1944*“ werden für diese Arbeit ausschliesslich Akten verwendet, die während des Zweiten Weltkriegs verfasst wurden. Der Analysezeitraum erstreckt sich dementsprechend zwischen dem 01. September 1939 und dem 8. Mai 1945, dem Tag der bedingungslosen Kapitulation Deutschlands.

Die zeitliche Eingrenzung ist notwendig, um die Funktionalität der im folgenden beschriebenen Pipeline in einem klar definierten historischen Kontext demonstrieren zu können. Gleichzeitig führt diese Auswahl zu einer bewussten Reduzierung der potenziell erfassten Akteurinnen und Akteure, Orte und Organisationen. Diese Fokussierung ist insbesondere im Hinblick auf die Erstellung einer verlässlichen Ground Truth bedeutsam, die durch ergänzende Archivrecherchen mit historischen Metadaten angereichert wird.

Die Kombination aus einer präzise definierten Quellengrundlage und der digitalen Anreicherung dient dazu, das Potenzial der computergestützten Auswertung historischer Dokumente exemplarisch aufzuzeigen. Zugleich unterstreicht sie, dass die Qualität der Ergebnisse wesentlich von der sorgfältigen Eingrenzung des Korpus und der manuellen Validierung und Anreicherung abhängt.

3.1 Quellen

3.1.1 Quellentradierung

In dem Lagerraum der *New Gospel Singers Murg*, dem Nachfolgeverein des *Männerchors Murg*, werden im Jahr 2018 mehrere je ca. 800 Seiten umfassende Ordner mit historischen Unterlagen gefunden. Für diese Arbeit wird ein Ordner mit der Aufschrift „*Männerchor Akten 1925–1944*“ gewählt, da er neben dem Ordner „*Männerchor Akten 1946–1950*“ den grössten Zeitraum abdeckt. Darüber hinaus bietet er das Potential, aufschlussreiche Einblicke in das Vereinsleben in der Zeit vor und während des Nationalsozialismus, insbesondere des Zweiten Weltkrieges, zu geben. Der Ordner umfasst insgesamt 780 Seiten. Deren Inhalt kann als „Protokoll“, „Brief“, „Postkarte“, „Rechnung“, „Regierungsdokument“,

„Noten“, „Zeitungsartikel“, „Liste“, „Notizzettel“ oder „Offerte“ kategorisiert werden.

Die Unterlagen könnten bereits direkt nach ihrer Entstehung in die Ordner eingelegt worden sein. Einzelne Akten sind mit einem „Heftstreifen“, auch „Aktendulli“ genannt, zusammengeheftet. In der Plastikversion, wie er in diesen Akten vorliegt, wurde er bereits 1938 patentiert²⁴. Wer die Akten so archiviert hat lässt sich nicht mehr sagen. Der sogenannte „Archival-Bias“ des Archivars, also die Grundeinstellung, weshalb etwas aufbewahrt oder vernichtet wurde, lässt sich damit nicht mehr feststellen.

3.1.2 Quellenbeschreibung

Für diese Arbeit wurde ein Korpus selektiert, dessen Auswahl im Abschnitt [Korpus](#) näher beschrieben wird. Nachfolgende Zahlen beschränken sich auf alle Dokumente, die während des Zweiten Weltkriegs geschrieben wurden. Sämtliche Unterlagen aus dem Ordner „*Männerchor Akten 1925–1944*“ werden erfasst, benannt und tabellarisch mit groben Metadaten versehen. Diese Auflistung in der Datei *Akten_Gesamtübersicht.csv* erlaubt die Zuordnung folgender Kategorien: Briefe, Postkarten, Protokolle, Regierungsdokumente, Zeitungsartikel, Rechnungen und Offerten. Die Verteilung ist ungleichmässig: Briefe bilden mit 282 von 381 Seiten die grösste Kategorie; Rechnungen und Offerten sind jeweils nur einseitig vertreten.

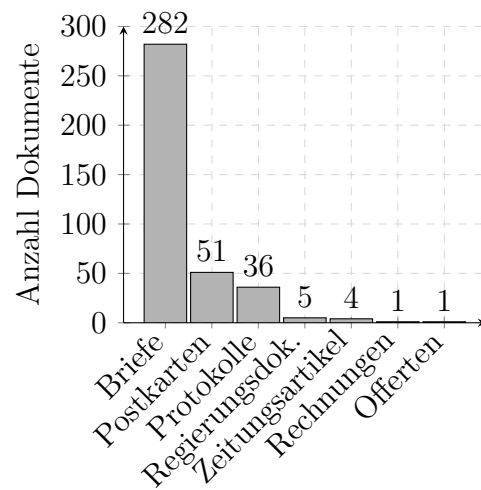


Abbildung 1: Dokumententypen im untersuchten Bestand (150 Akten – 381 Seiten).

²⁴vgl. [Heftstreifen 2023](#).

Auf Grundlage der oben erwähnten, händisch erstellten Gesamtliste der Akten können durch die systematische Benennung der Dokumente auch Rückschlüsse auf den bislang nicht untersuchten Teil des Bestandes gezogen werden. Mithilfe des Sprachmodells von OpenAI wurde auf Basis der manuell vergebenen Titel in der Gesamtübersicht²⁵ eine grobe Näherung zur Zusammensetzung des restlichen Korpus erarbeitet, wie sie in der rechtsstehenden Darstellung visualisiert ist. Diese Übersicht erhebt keinen Anspruch auf genaue Zahlen, sondern dient der Veranschaulichung, dass

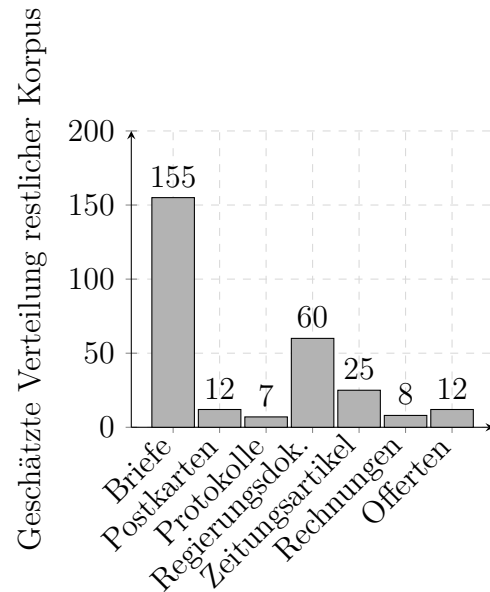


Abbildung 2: Geschätzte Verteilung der Dokumententypen im restlichen Bestand.

die Verteilung der Quellengattungen im Zeitraum vor dem Zweiten Weltkrieg möglicherweise deutlich anders gestaltet ist. Eine vertiefte Untersuchung dieser bislang unbearbeiteten Bestände ist daher notwendig und markiert eine zentrale Forschungslücke, die im Rahmen dieser Arbeit erstmals systematisch benannt wird.

3.1.3 Sichtung & Kategorisierung zu Akten

Für diese Arbeit werden alle Seiten in dem Ordner „*Männerchor Akten 1925–1944*“ zweimal gesichtet und gelesen. Beim ersten Durchgang wird explorativ nach zusammenhängenden Unterlagen gesucht, die im Anschluss zu einer Akte zusammengefasst werden können.

Ausschlaggebend für eine Zusammenfassung in einer Akte sind folgende Faktoren:

- Historische Gliederung durch Bindung (Büroklammern, Heftstreifen, etc.)
- gleiche Autorenschaft in direkt aufeinanderfolgenden Seiten
- gleiches Datum in direkt aufeinanderfolgenden Seiten
- gleiches Thema in direkt aufeinanderfolgenden Seiten

²⁵vgl. nachfolgenden Abschnitt [3.1.3](#)

Auf dieser Grundlage wird eine Aktenübersicht²⁶ im CSV-Format erstellt. Diese Tabelle setzt sich zusammen aus Aktennummern, die wiederum die Reihenfolge innerhalb des ursprünglichen Ordners beschreiben. In diesem ersten Schritt gilt die Lage als Identifikator für die Unterlagen. Jeder Nummer wird darüber hinaus ein beschreibender Titel und das Erstellungsdatum zugewiesen.

Vorgreifend soll auch die zweite Quellsichtung beschrieben sein, in der die Daten in der CSV um Metadaten auf Seitenebene und aus Transkribus ergänzt werden. Die Kategorisierung findet also in einem parallelen Prozess mit der Digitalisierung statt. Hierfür werden die digitalen Akten auf Seitenebene genau aufgebaut. Dem zugrunde liegt eine interne Seiten-ID, die den Aktennamen und die Position innerhalb der Akte kombiniert (Bsp: Akt_078_S001.jpg). Ab dem Zeitpunkt des Uploads bei Transkribus wird diese jedoch durch Transkribus-Dokument-ID abgelöst. Beide IDs werden zur besseren Nachvollziehbarkeit in der CSV notiert.

Auch inhaltlich wird nochmals schärfer kategorisiert. Mit Tags in Apple-Dateien²⁷ und der CSV wird nun folgendes erfasst:

- Handschrift
- Maschinenschrift
- Bild
- Signatur

Die in [Quellenbeschreibung](#) dargestellten Kategorisierungen werden nun als Ground Truth-Daten in die CSV aufgenommen, um sie später in der Pipeline auszuwerten. Das bedeutet, sie gelten damit in den weiteren Schritten als verbindliche Referenz zur Validierung und Auswertung.

3.2 Digitalisierung der Quellen

Überlieferte analoge Dokumente müssen zunächst fachgerecht für das Projekt und den Digitalisierungsprozess aufbereitet werden. Hierzu werden die Akten aus ihren ursprünglichen Ablagesystemen entnommen und sorgfältig von Gummibändern, Büro- und Heftklammern befreit. Diese konservatorischen Massnahmen sind notwendig, um die langfris-

²⁶genannt Akten_Gesamtübersicht.csv; in den Projektdaten

²⁷vgl. [Apple Support: Dateien-App](#)

tige Materialerhaltung zu gewährleisten, da insbesondere Korrosionsspuren ehemaliger Metallklammern die Papierfasern nachhaltig schädigen können. Zudem finden sich häufig Anzeichen von Säurefrass, sofern nicht säurefreies Archivmaterial verwendet wurde.

Für die eigentliche Digitalisierung kommt die native „Dateien“-Applikation von Apple zum Einsatz. Diese bietet neben einer vergleichsweise hochauflösenden Erfassung die Möglichkeit zur direkten Speicherung in einem Cloud-basierten Speichersystem sowie eine automatische Texterkennung (OCR). Ziel dieser Vorgehensweise ist es, die digitalisierten Inhalte möglichst schnell durchsuchbar zu machen und standortunabhängig für das Projekt zugänglich zu machen.

Die Aufnahme der Dokumente erfolgt mithilfe eines iPads, das auf einem Stativ exakt im rechten Winkel (90°) über dem zu digitalisierenden Schriftgut positioniert wird. Diese einfache, jedoch effiziente Konfiguration gewährleistet eine gleichbleibenden Bildausschnitt bei gleichzeitig hoher Verarbeitungsgeschwindigkeit. Die digital erfassten Dateien werden konsistent benannt und folgen einer vorab definierten Gesamtübersicht der Bestände. Mehrseitige Konvolute werden dabei als zusammengehörige Akteneinheiten geführt, während Einzeldokumente entsprechend separat erfasst werden. Die Archivierung erfolgt sowohl analog als auch digital auf Seitenebene, um eine möglichst feingranulare Erschließung zu ermöglichen.

Die initiale Speicherung erfolgt dabei standardmässig im PDF-Format. Für die anschließende Verarbeitung mit den unten dargestellten Transkriptionswerkzeugen müssen die Dokumente jedoch in das JPEG-Format konvertiert werden. Die Umwandlung erfolgt automatisiert mithilfe eines eigens erstellten Python-Skripts, wie in Anhang [A.1](#) beschrieben.²⁸ Es extrahiert die Seiten, speichert sie im geeigneten Format ab und ergänzt die Dateinamen systematisch um eine dreistellige, führend nullengefüllte Seitennummer.

3.3 Transkription

Nach der Digitalisierung und Konvertierung der Dokumente beginnt die eigentliche Transkription. Wie im Kapitel [Transkriptionen \(Methodenvergleich\)](#) dargestellt, entscheidet sich dieses Projekt bewusst für Transkribus als zentrale Plattform. Ausschlaggebend ist insbesondere die Möglichkeit, ein eigenes HTR-Modell auf Basis einer projektspezifischen Ground Truth zu trainieren, sowie die integrierten Funktionen zur Annotation von Named

²⁸Burkhardt [2025b](#).

Entities direkt im Transkriptionsprozess. Für die effiziente Transkription soll im Folgenden der Workflow beschrieben werden, der einen Mixed Method Ansatz verfolgt.

Wie das Beispiel [Abbildung 3](#) rechts verdeutlichen soll, sind viele der Akten schwer entzifferbar. Auch Transkribus kommt wegen der kleinen Sets unterschiedlicher Autoren und Autorinnen für spezifische Handschriften an seine Grenzen. Mit Expertise sind diese nur mit hohem zeitlichen Aufwand transkribierbar. Die Baselines²⁹ ist in diesem Beispiel verhältnismässig homogen. Schwierigkeiten bereiten jedoch Postkarten oder Zeitungsartikel, die mit einer komplexeren Schriftsetzung einen höheren Aufwand in der Transkription benötigen. Ausgangspunkt für dieses Projekt ist das generische Modell *The German Giant I*, das mit

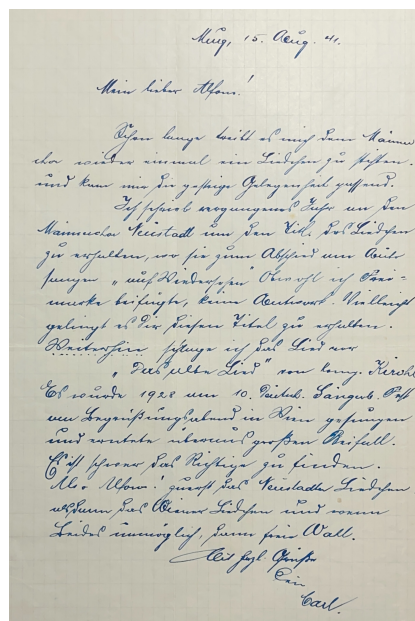


Abbildung 3: Beispiel für handschriftlichen Text in Akte_076

einer CER³⁰ von 8,30% zunächst auf 70 Akten angewendet wird. Sie umfassen 158 Seiten mit insgesamt 22.155 Wörtern. Die Ergebnisse sind dabei jedoch sehr unpräzise, wie [Abbildung 4](#) veranschaulicht. In insgesamt vier Transkriptionsdurchläufen über diese Selektion wird daher manuell eine Ground Truth für ein eigenes Modell erstellt und gleichzeitig regelbasiert getaggt. Die Tags erfassen strukturiert Personen, Orte, Daten und Organisationen. Ein eigenes Kapitel hierzu findet sich im Anhang [Tagging mit Transkribus und LLM](#). Für die OCR-Erkennung wird eingangs auch ChatGPT verwendet, das in der direkten Texterkennung aus Bilddateien als nicht ausreichend zuverlässig bewertet werden muss.³¹

²⁹Baselines = Schriftausrichtung

³⁰CER (Character Error Rate): Kennzahl für die Anzahl falsch erkannter Zeichen.

³¹Ausführlich wird dies in [OpenAI – ChatGPT](#) erläutert.

Bei der oben ausgeführten Erstellung des Groundtruth-Korpus wird bei der manuellen Korrektur OpenAIs ChatGPT-4o-Modell für die Rechtschreibprüfung verwendet. Die vermeintliche Schwäche bei der Transkription, passende Begriffe zu halluzinieren, stellt sich hier als besonders hilfreich heraus. Gerade bei der Rekonstruktion fehlender Wörter oder Satzteile zeigt sich die Stärke des wahrscheinlichkeitsbasierten LLMs. In Kombination mit der philologischen Expertise bei der Entzifferung einzelner Buchstaben entsteht so ein kollaborativer Transkriptionsprozess, bei dem Maschine und Mensch sich wechselseitig ergänzen. Die automatische Transkription wird in [Abbildung 4](#) dargestellt, die überarbeitete LLM-Version folgt in [Abbildung 5](#). Die so entstehende Ground Truth wird für das Training des Modells ([ModelID: 287793](#))³² verwendet. Dieses trainierte Modell erreicht eine CER von 6,58% und kommt anschliessend für die automatische Transkription der übrigen Dokumente zum Einsatz. Auch hier ist eine manuelle Überprüfung der durch das eigens trainierte Modell erstellten Transkription unabdingbar, wobei sich die knapp 1.7% geringere CER sich jedoch beim Korrekturaufwand bereits bemerkbar macht. Gleichzeitig wird diese Korrektur für das Taggen benutzt, das folgend beschrieben werden soll.

Murg. 15. Aug 41
 Mein lieber Alfons!
 Sehen lunge Leitt es mich dem Männerchor wieder einmal ein Liedchen zu stehen. und kam mir die gestege Gelegenheit gussend. Männechor Venstad um den Title das Liedchen zu erhalten, wo sie zum Abschied am Aute sängen "auf Wiederschen Owohl ich Frei! märke beifügte, keine Aentwarb. Vielleicht gelingt es Dir diesen Itel zu erhalten. Weiterhin sänge ich fal Lied nur "Bas alte Lied" von being. Rerohl
 Es wurde 1928 am 10. Dachub. Sängerb. Frst von Begrüssungsabend in Dien gesungen. und ertete überaus grossen Reifall.
 Es ich schwer das Richtige zu finden.
 Aler Alfon, werst das Vemsladler Liedchen. alsdann das Biener Lidchen und wenn Leides unmöglich, dann freu Nall.
 Mit herz. Grüsse
 Dein
 Carl

Abbildung 4: Transkription durch *The German Giant I*-Modell der [Abbildung 3](#)

Murg, 15. Aug. 41
 Mein lieber Alfons!
 Schon lange treibt es mich, dem Männerchor wieder einmal ein Liedchen zu stiften, und kam mir die günstige Gelegenheit gelegen.
 Ich schrieb vergangenes Jahr an den Männerchor Venstad, um den Titel des Liedchens zu erhalten, das sie zum Abschied am Auto sangen: „Auf Wiedersehen, o wohl ich frei!“
 Ich fügte eine Frankierung bei, erhielt jedoch keine Antwort. Vielleicht gelingt es Dir, diesen Titel zu erhalten.
 Weiterhin sang ich das Lied nur „Das alte Lied von Wien“. Obwohl es am 10. Dezember 1928 beim Sängerbund-Fest von Begrüssungsabend in Wien gesungen wurde und überaus grossen Beifall ertete, ist es schwer, das Richtige zu finden.
 Aber Alfons, zuerst das Venstadler Liedchen, dann das Wiener Liedchen und wenn beides unmöglich, dann Fröhlichsein.
 Mit herzlichen Grüssen
 Dein
 Carl

Abbildung 5: Durch ChatGPT unterstützte Transkription von [Abbildung 4](#).

³² (vgl. [Transkribus 2024](#))

3.3.1 Tagging mit Transkribus und LLM

Während der manuellen Korrektur der Transkriptionen erfolgt parallel die Annotation zentraler Entitäten. Transkribus bietet hierfür ein flexibles Tagging-System, mit dem sowohl strukturelle als auch semantische Informationen direkt im Dokument markiert werden können. Im Zentrum stehen dabei Tags für Personen, Orte, Organisationen und Datumsangaben. Diese Kategorien sind für die spätere Analyse besonders relevant, etwa für die Modellierung historischer Netzwerke oder die Kontextualisierung von Ereignissen.

Ein Mixed-Method-Verfahren kommt dort zum Tragen, wo die Transkription an ihre Grenzen stösst: Fehlende Buchstaben, fehlerhafte Worttrennungen oder unleserliche Handschriften lassen sich durch die Kombination aus Modellwissen und menschlichem Quellenverständnis rekonstruieren. ChatGPT liefert hier auf Basis des Kontexts plausible Vorschläge, die von einer historisch geschulten Person geprüft und übernommen oder verworfen werden. Dieser kollaborative Vorgang erhöht die semantische Genauigkeit der rekonstruierten Passagen enorm.

Ein Beispiel zeigt die schrittweise Entwicklung einer Transkription: Ausgehend von einem gescannten Originalbrief ([Abbildung 3](#)) wird zunächst eine maschinelle Transkription erstellt ([Abbildung 4](#)), die anschliessend durch ein LLM geglättet und lesbarer gemacht wird ([Abbildung 5](#)).

In einem letzten Schritt erfolgt die manuelle Annotation mit Transkribus-Tags (6): Hier werden etwa Murg und Wien als Orte, Alfons, Carl und Kirch1 als Personen sowie das Deutsch. Sängerb. Fest als Ereignis markiert. Auch der Liedtitel „Das alte Lied von Wien“ wird in seinen Bestandteilen zwischen Person, Ort und kulturellem Kontext aufgeschlüsselt. Für unklare oder unleserliche Textstellen, wie etwa das Fragment "auf Wiederschen"[sic], kommt das Tag `unclear` zum Einsatz – häufig auf Grundlage einer Vorschlagsformulierung durch ChatGPT. Zusätzlich zur semantischen Markierung ermöglicht Transkribus auch die Kennzeichnung struktureller Eigenschaften.

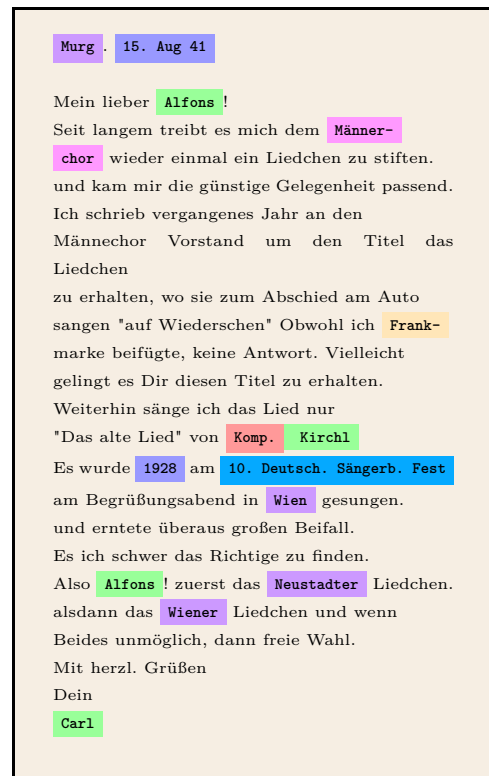


Abbildung 6: Tags der Transkription von Abbildung 5.

So wird beispielsweise die Abkürzung V.D.A. – für *Verein für das Deutschtum im Ausland* – mit dem Tag `abbrev` versehen, auch wenn diese Tags in der XML-Exportstruktur teilweise nicht vollständig erhalten bleiben (vgl. Kapitel 4.5).

Für die spezifischen Anforderungen dieses Korpus wird das Tagging-Schema gezielt erweitert, etwa um den benutzerdefinierten Tag `signature`, der handschriftliche Unterschriften maschinenlesbar ausweist. Das zweite Beispiel – ein poetischer Brief an Otto (Abbildung 6, unten) – zeigt die Anwendung dieses Verfahrens in lyrischer Sprache. Auch hier werden alle erwähnten Personen (u. a. Otto, Lina Fingerdick, Otto Bollinger, Alfons Zimmermann), Orte (Murg, Laufenburg (Baden), Rhina) sowie Organisationen (Männerchor) mit den entsprechenden Tags versehen. Die adressierte Funktion *Vereinsführer des Männerchor* wird dabei als Rolle und Organisation und zusätzlich semantisch als Personenbezug erfasst .

Fehlerhafte oder fehleranfällige Passagen – insbesondere historisch bedingte Schreibungen oder Transkriptionsunschärfen – werden mit dem Tag `sic` versehen. In diesen Fällen folgt die standardisierte Lesart unmittelbar auf das markierte Original, wodurch ein differenzierter Umgang mit dem Quelltext sichergestellt ist.

Alle Tags werden während des Transkriptionsprozesses konsistent dokumentiert und in einem projektspezifischen Regelwerk festgehalten.³³ Dieses dient nicht nur der internen Nachvollziehbarkeit, sondern auch als Grundlage für die spätere Verarbeitung durch Sprachmodelle, die auf die gleichen semantischen Kategorien angewiesen sind. Das strukturierte Tagging bildet somit die Brücke zwischen manueller Quellenarbeit und automatisierter Weiterverarbeitung.

4 Methodisches Vorgehen

Digitale Methoden spielen für die Durchführung dieser Arbeit eine zentrale Rolle. Von der Digitalisierung der Quellen über die Transkription bis hin zur Auswertung durchlaufen die Daten zahlreiche Prozessschritte, die mithilfe von Large Language Models, Deep-Learning-Modellen und anderen digitalen Werkzeugen verarbeitet und visualisiert werden. Die Auswahl der Tools orientiert sich dabei an Kriterien wie Verfügbarkeit (Open Source vs. proprietär), Kompatibilität, Community-Support, erforderlichem Arbeitsaufwand und selbstverständlich dem konkreten Mehrwert für die Forschungsfragen.

In diesem Kapitel werden sowohl Werkzeuge vorgestellt, die tatsächlich eingesetzt wurden, als auch solche, die sich im Verlauf des Projekts als ungeeignet erwiesen. Transparenz ist hierbei ein wesentlicher Aspekt: Ein grosser Teil der Methodik entwickelte sich erst im Forschungsprozess selbst. Da sich Large Language Models rasant weiterentwickeln, ist nicht immer von Beginn an klar, ob ein Tool für den eigenen Anwendungsfall geeignet ist. Um diese Unsicherheiten zu dokumentieren, werden hier auch gescheiterte Versuche dargestellt. Dazu gehört der nachfolgende Linked-Open-Data-Ansatz.

4.1 LOD – Linked Open Data

Linked Open Data (LOD) bezeichnet einen dezentral organisierten Ansatz zur Veröffentlichung und Verknüpfung strukturierter Daten im Web. Ziel ist es, Datensätze verschiedener Institutionen und Akteure maschinenlesbar zugänglich zu machen und über standardisierte Formate wie RDF und SPARQL miteinander zu verbinden³⁴. Wesentliches Merkmal der LOD-Cloud ist dabei die Nutzung semantischer Beziehungen, insbesondere Äquivalenzen einzelner Daten. Hierfür wird häufig das Prädikat `owl:sameAs` genutzt,

³³vgl. [Tagging in Transkribus](#)

³⁴vgl. Garoufallou und Ovalle-Perandones 2020, S. VI und 13f.

um z.B. mit `:Choir owl:sameAs wd:Q131186` eine eigene Instanz als identisch mit der Wikidata-Entität für einen Chor zu deklarieren. Klassen oder Instanzen können so aus unterschiedlichen Datenquellen eindeutig identifiziert und zusammengeführt werden.

Die vom World Wide Web Consortium (W3C) entwickelte OWL Web Ontology Language ist damit ein zentrales Werkzeug für die Realisierung von LOD.³⁵ Mit ihr lassen sich Ontologien definieren, die Domänen über Klassen, Individuen und deren Relationen formal beschreiben. Sie ermöglichen, logische Schlussfolgerungen zu ziehen, um verteilte Datenbestände zu verknüpfen und maschinenlesbar auszuwerten. Besonders relevant ist dabei `owl:sameAs`, das als Identitätsrelation fungiert: Es deklariert Instanzen, die in unterschiedlichen Quellen unter verschiedenen URIs³⁶ geführt werden, als dasselbe reale Objekt³⁷ und ermöglicht so eine präzise Zusammenführung von Informationen — ein Grundpfeiler für die Interoperabilität im Semantic Web. Die OWL-Spezifikation baut auf RDF³⁸ auf und erweitert es um zusätzliche Konzepte. Die RDF-Daten werden häufig im Turtle-Format (TTL) serialisiert, einer textbasierten Notation für RDF, die eine kompakte, leicht lesbare Schreibweise bietet. Dieses Format eignet sich besonders für den Austausch und die manuelle Bearbeitung von RDF-Tripeln. Die Sprache liegt in drei Varianten vor³⁹, die sich im Grad ihrer Ausdrucksstärke unterscheiden.⁴⁰ Insbesondere OWL DL bietet einen praktikablen Mittelweg zwischen hoher Ausdruckskraft und vollständigem, entscheidbarem Schliessen (Reasoning) und ist daher für viele LOD-Anwendungsfälle geeignet.

Das Potenzial wird diese Form der Datenverknüpfung bislang nicht von allen Websites konsequent umgesetzt.⁴¹ Für die technische Umsetzung für diese Arbeit werden zwei zentrale Werkzeuge erprobt. Das sind Protégé zur Modellierung der Ontologie und GraphDB für deren Verwaltung und Abfrage.

4.1.1 Protégé

Bis sich der Fokus der Arbeit im weiteren Verlauf verlagert, kommt *Protégé* zur praktischen Modellierung der Ontologie zum Einsatz. Protégé ist eine weit verbreitete Open-

³⁵ vgl. *OWL Guide 2004*.

³⁶ Abk. **URI** Uniform Resource Identifier

³⁷ vgl. *ebd.*, 2.3. Data Aggregation and Privacy.

³⁸ Abk. **RDF** Resource Description Framework

³⁹ OWL Lite, OWL DL und OWL Full

⁴⁰ vgl. *ebd.*, 1.1. The Species of OWL.

⁴¹ vgl. Garoufallou und Ovalle-Perandones *2020*, S. 14.

Source-Software zur Erstellung, Visualisierung und Verwaltung von Ontologien. Die grafische Oberfläche unterstützt eine intuitive Klassendefinition, Relationserstellung und Instanzverwaltung. Mit Hilfe von Plugins können darüber hinaus logische Konsistenzprüfungen durchgeführt und Ontologien direkt im OWL-Format exportiert werden, um sie in LOD-Workflows einzubinden. Die initiale Version der Ontologie für dieses Projekt entsteht im Codeeditor *Visual Studio Code*, wird aber schnell vollständig in Protégé überarbeitet. Damit bildet das Programm die Grundlage für erste Experimente mit Abfragen in SPARQL.

4.1.2 GraphDB

Für die Speicherung und Abfrage der Ontologie wird *GraphDB* verwendet. GraphDB ist eine spezialisierte RDF-Triplestore-Datenbank, die es ermöglicht, grosse Mengen an semantisch verknüpften Daten effizient zu verwalten. Mit der integrierten SPARQL-Schnittstelle können Benutzer gezielt nach Instanzen, Klassen und Relationen suchen und komplexe Muster in den Datenbeständen erkennen. Im Rahmen dieser Arbeit dient GraphDB als Backend, um die in Protégé entwickelte Ontologie zu testen und mit realen Entitäten aus den untersuchten Quellen abzugleichen.

4.1.3 LOD-Ontologie

Ein wichtiger Aspekt dieser Arbeit ist die Unstrukturiertheit relevanter Informationen. Aus diesem Grund wird auf der Basis der oben beschriebenen Semantik begonnen, eine eigene Ontologie zu entwickeln, die die identifizierten Entitäten systematisch erfasst. Beim Schreiben dieser initialen Ontologie aus rund 2000 Zeilen Code erweist sich schnell ein neues Problem. Die Datengrundlage aus den geschilderten Vorprojekten (vgl. [Forschungsstand und Forschungslücke](#)) ist zu klein, um daraus eine aussagekräftige Netzwerkanalyse zu machen. Hierfür erweisen sich die Unterschiede der Daten zusätzlich als zu gross und damit im zeitlichen Rahmen als nicht realisierbar. Der Fokus

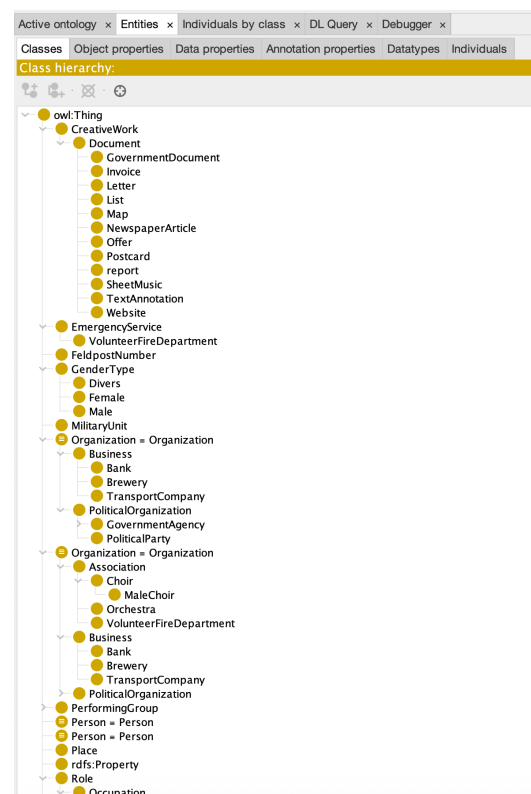


Abbildung 7: Ausschnitt der TTL-Ontologie.

der Arbeit verschiebt sich dementsprechend von der Ontologieentwicklung auf die Extraktion von Entitäten. Hinzu kommen externe Quellen und deren Zugänglichkeit. Die Hauptquellen für Informationen über militärische Einheiten und deren Feldpostnummern sind das „Forum der Wehrmacht“⁴² und der „Suchdienst des DRK“⁴³. In beiden Fällen liegen die Daten jedoch nicht als LOD vor. Das Forum stellt einfache Strings dar, das Deutschen Roten Kreuz OCR-PDF⁴⁴ historischer Suchlisten aus der Nachkriegszeit. Ein manuelles Recherchieren dieser Daten scheint zu diesem Zeitpunkt den Rahmen der Arbeit zu sprengen. Daher wird im Januar 2025 entschieden, den Aufbau einer LOD Datenbank in dieser Form abubrechen und einen Fokus auf die Extraktion von Named Entites und das Entity Matching zu legen. Die bis zu diesem Schritt geleistete Vorarbeit beim Sortieren und Klassifizieren von Entitäten, wird in späteren Prozessschritten wieder aufgegriffen⁴⁵. Auch die Recherche der IDs in Wikidata und Geonames, wie im Folgenden beschrieben, wird weiterverwendet.

4.2 Wikidata

Wikidata⁴⁶ ist eines der zentralen Repositorien für Linked Open Data und bietet eine hohe Interoperabilität durch standardisierte URIs, SPARQL-Endpunkte und offene APIs zu den Entitäten. Jede Entität erhält dabei eine eindeutige, persistente URI (z.B. `wd:Q131186` für einen Chor), die in LOD-Szenarien als stabiler Referenzpunkt dient. Neben anderen betonen Martinez & Pereyra Metnik (2024) beispielsweise:

*„Wikidata stands out for its great potential in interoperability and its ability to connect data from various domains.“*⁴⁷

Wikidata entspricht, ebenso wie das nachfolgend beschriebene GeoNames, den FAIR-Prinzipien: Die Daten sind **F**indable und **A**ccessible, **I**nteroperable und **R**eusable.⁴⁸

Im Rahmen dieser Arbeit dient Wikidata als zentrale externe Referenz, um lokal erho-bene Entitäten mit international etablierten Datenobjekten zu verknüpfen und so ihre Interoperabilität sicherzustellen. Die Plattform ermöglicht eine eindeutige Identifizierung sowie die maschinenlesbare Anreicherung zusätzlicher Informationen.

⁴² vgl. Altenburger 2023.

⁴³ vgl. *DRK Suchdienst / Suche per Feldpostnummer* 2025.

⁴⁴ OCR = Optical Character Recognition

⁴⁵ siehe Abschnitt *Nodegoat*

⁴⁶ vgl. *Wikidata* 2025.

⁴⁷ Martinez und Metnik o. D.

⁴⁸ vgl. Wilkinson u. a. 2016, S. 2.

Die praktische Umsetzung offenbart jedoch eine strukturelle Einschränkung: Etwa 70% der für diese Arbeit eigens angelegten Einträge auf Wikidata werden trotz systematischer Verknüpfung mit anderen dort verwalteten Entitäten⁴⁹ im Rahmen der Community-Moderation wieder entfernt. Das zeigt einerseits die hohen internen Qualitätsanforderungen auf, andererseits werden diese jedoch nicht klar kommuniziert. Mit rigidem Löschen neuer Einträge wird die Verlässlichkeit und der Nutzen der geleisteten Arbeit erheblich begrenzt. Aufwand und Unsicherheit über die Persistenz der Einträge machen den ursprünglich vorgesehenen LOD-Ansatz in dieser Form nicht praktikabel. Daher findet es innerhalb der Pipeline optionale Anwendung. Wenn einzelne Entitäten eine Wikidata-ID haben, wird diese in die [Nodegoat](#)-Datenbank aufgenommen und für das Entity Matching verwendet.

4.3 GeoNames

Ebenso wie Wikidata bietet *GeoNames*⁵⁰ eine Open-Source-Plattform für interoperable Daten. GeoNames fokussiert sich hierbei auf geografische Informationen und stellt eine umfassende Datenbank mit über 25 Millionen Ortsnamen und rund 12 Millionen eindeutigen geografischen Objekten bereit. Die Plattform integriert Daten zu Ortsnamen in verschiedenen Sprachen, Höhenlagen, Bevölkerungszahlen und weiteren Attributen aus unterschiedlichen nationalen und internationalen Quellen. Sämtliche Geokoordinaten basieren auf dem WGS84-System⁵¹ und können über frei zugängliche Webservices oder eine API abgerufen werden. Darüber hinaus erlaubt GeoNames registrierten Nutzenden, bestehende Datensätze über eine Wiki-Oberfläche zu bearbeiten oder zu ergänzen, wodurch eine kollaborative Qualitätssicherung gewährleistet wird.

GeoNames wird in dieser Arbeit intensiv zur Referenzierung von Ortsnamen verwendet und bildet die Basis für die Ground Truth, wie sie in den Kapiteln [Nodegoat](#) und [place_matcher.py](#) beschrieben ist. So werden mögliche Ortsnamen als Kandidaten beispielsweise automatisch in diesem Modul per GeoNames-API abgefragt. Im Gegensatz zu Wikidata wird hier von Beginn an darauf verzichtet, eigene Ortsdatensätze zu ergänzen. Dies liegt einerseits an den klar kommunizierten Community Guidelines und andererseits

⁴⁹etwa Armeen, Militäreinheiten, Orten und Personen

⁵⁰vgl. [GeoNames 2025](#).

⁵¹WGS84: *geodätische Grundlage des Global Positioning System (GPS)* ;vgl. [WGS84 / Landesamt für Geoinformation und Landesvermessung Niedersachsen 2025](#).

daran, dass der Datensatz, mit Ausnahme einiger sehr lokaler Flurnamen⁵², als weitgehend vollständig gelten kann. Da GeoNames auf aktuelle Daten ausgerichtet ist, fehlen historische Gebäude wie Gaststätten oder Spitäler konsequenterweise in der Datenbank. Diese Lücke ist zwar erwartbar, bleibt jedoch im Rahmen historischer Auswertungen relevant.

4.4 Nodegoat

Nachdem sich die Implementierung von Linked Open Data (LOD) für das vorliegende Projekt aufgrund des hohen zeitlichen Aufwands als nicht realisierbar erwiesen hat, wird mit **Nodegoat**⁵³ eine praktikable und zugleich forschungsnahe Alternative eingeführt. Im Folgenden soll das Tool näher beschrieben und seine Anwendung für das Projekt erläutert werden.

Nodegoat ist eine webbasierte Plattform, die sich besonders in den digitalen Forschungsprojekten in den Geisteswissenschaften etabliert hat. Es unterstützt Forschende in der Modellierung, Verwaltung, Analyse und Visualisierung komplexer Datenbestände. Ein zentrales Merkmal von Nodegoat ist die grafische Benutzeroberfläche, die eine vergleichsweise niedrige Einstiegshürde bietet. Auch Forschenden ohne tiefgehende Programmierkenntnisse wird so die Möglichkeit eröffnet, eigene Datenmodelle zu definieren, zu pflegen und weiterzuentwickeln. Kritisiert werden muss an Nodegoat jedoch die fehlende Dokumentation. Viele Informationen finden sich nur über Drittparteien⁵⁴ oder, wie unten geschildert, auf konkrete Nachfrage bei den Entwicklern. Der so geleistete Support ist jedoch ausgesprochen zeitnah und umfassend. Beispielsweise wird der IFrame des Public Interface wegen dieses Projekts so angepasst, dass man die Anzeigetools nun optional deaktivieren kann. Eine Verbesserung von Nodegoat, von der auch weitere User in Zukunft profitieren können.⁵⁵ Die Plattform folgt einem modularen Prinzip: Über das UI⁵⁶ können beliebig viele Datenmodelle erstellt werden, die sich flexibel an die spezifischen Forschungsfragen anpassen lassen. Diese hohe Individualisierbarkeit der Datenstrukturen erlaubt es, innerhalb kürzester Zeit projektspezifische Datenbanken zu konzipieren und fortlaufend zu erweitern oder an sich ändernde Bedürfnisse anzupassen.

Die Grundstruktur eines Modells unterscheidet in Nodegoat zwischen sogenannten

⁵²Der „Totenbühl“ in Murg ist beispielsweise ein solcher Flurname

⁵³kessels_Nodegoat_2013.

⁵⁴gubler_Nodegoat_nodate.

⁵⁵implementiert wird diese Funktion im September 2025.

⁵⁶Abk.:UI User-Interface

Object Descriptions und **Sub-Objects**. Erstere legen die grundlegenden Merkmale eines Objekts fest, etwa Zeichenketten, Zahlen oder Verweise auf andere Einträge. So kann eine **Person** in diesem Projekt neben einem Feld für Vor- und Nachname auch eine Referenz auf ein **Gender**-Objekt enthalten, das eine eindeutige Geschlechtszuordnung ermöglicht.

Für das hier behandelte Forschungsvorhaben übernimmt Nodegoat die zentrale Verwaltung der Ground Truth-Daten, die später als CSV-Export in die Pipeline integriert werden. Für die Ground Truth werden folgende Entitäten modelliert:

Entitäten	
Personen	Orte
Organisationen	Ereignisse
Dokumente	Rollen
Geschlechter	

Tabelle 1: Übersicht der erfassten Entitäten

Die Auflistung ist dabei so geordnet, dass die Entitäten mit der grössten Anzahl oder Vielfalt an zugehörigen Sub-Objects zuerst genannt werden. **Sub-Objects** erlauben eine noch vielfältigere Abbildung abhängiger Informationen. Das können in dieser Arbeit zum Beispiel Quellenbelege sein, sowie temporale oder lokale Attribute, die einem Hauptobjekt zugeordnet werden. Lokale Attribute werden in Verknüpfung mit [GeoNames](#) und für Länder, Flurnamen oder Gewässer im Geojson-Format erstellt.⁵⁷ So lassen sich beispielsweise für **Persons** in den Sub-Objekten **Geburt** oder **Tod** das Datum und der Ort modellieren, sowie die Todesursache notieren.

Die in den Kapiteln [Transkriptionen und Methoden](#), [Forschungsstand und Forschungslücke](#) sowie [Unmatched Logger](#) beschriebenen Verarbeitungsschritte bilden die Grundlage für die in Nodegoat hinterlegten Entitäten. Die strukturierte Erfassung dient dabei nicht nur der internen Qualitätssicherung, sondern ermöglicht auch eine nachhaltige Referenzierbarkeit. Erreicht wird das durch eindeutig vergebene Identifikatoren, die beispielsweise beim Abgleich von Personendaten genutzt werden. So können Textstellen präzise mit den zugehörigen Datenbankeinträgen verknüpft werden.

Ein Beispiel hierfür ist die Modellierung der Rolle. Sie orientiert sich an den in den Unter-

⁵⁷Weiterführend: Thomson, Wilde und Roach [2017](#).

lagen vorkommenden Strings. Auf den Einsatz externer Schemata oder Thesauri wurde bewusst verzichtet, da diese zwar die Vergleichbarkeit erhöht, jedoch aufgrund des erforderlichen höheren Normalisierungsaufwands weniger präzise Ergebnisse geliefert hätten.⁵⁸

Darüber hinaus wird Nodegoat über seine API⁵⁹ mit einer eigens entwickelten Webanwendung verknüpft. Diese Webanwendung fungiert als publikumsorientierte Such- und Präsentationsplattform für die erarbeiteten Quellen. Die in den strukturierten JSON-Daten enthaltenen Nodegoat-IDs verknüpfen hier ebenfalls jede identifizierte Named Entity eindeutig mit dem zugehörigen Objekt in der Nodegoat-Datenbank.⁶⁰

In grösseren Projekten wird für die AIP-Nutzung ein Client für die jeweiligen Nutzer erstellt, mit dem wiederum ein Passkey generiert wird. Alternativ kann auch ein allgemeiner API-User als Funktionsadresse eingerichtet werden, was in diesem Ein-Personen-Projekt jedoch nicht notwendig ist. Über die API werden Objekt-IDs abgefragt, die dann später im Webtool zur Zuweisung der jeweiligen Entitäten verwendet werden. Da der Zugang zur API pro Stunde limitiert ist, werden die Abfragen auf der Projektseite eingeschränkt sein.

4.5 Transkriptionen (Methodenvergleich)

4.5.1 Tesseract

Da bereits zu Beginn des Projekts klar ist, dass ein Grossteil der Datenverarbeitung mit Python-Code erfolgen soll, wird gezielt nach Werkzeugen gesucht, die eine automatische Transkription von gescannten Dokumenten ermöglichen. Als besonders etabliert erweist sich die OCR-Engine Tesseract, ein Open-Source-Projekt zur Texterkennung in Bilddateien. Tesseract wird seit den 1980er-Jahren entwickelt, zunächst von Hewlett-Packard, später von Google weitergeführt, und ist über GitHub öffentlich zugänglich.⁶¹

Die Software basiert seit Version 4 auf einem LSTM-basierten⁶² neuronalen Netzwerk,

⁵⁸Dieser Aufwand zeigt sich beispielsweise in abweichenden historischen Schreibweisen, fehlenden Entsprechungen für vereinsinterne Funktionsbezeichnungen, semantischen Verschiebungen über die Zeit und der Mehrdeutigkeit mancher Rollenbegriffe.

⁵⁹Programmierschnittstelle

⁶⁰an dieser Stelle sei ein Rückverweis auf die Kapitel [GeoNames](#) und [Wikidata](#) gegeben.

⁶¹vgl. Weil, Pugin und Dovev 2025.

⁶²Abk.: **LSTM** steht für *Long Short-Term Memory*; Architektur der frühen Generation rekurrenter neuronaler Netzwerke *RNNs*. LSTMs wurden entwickelt, um Sequenzdaten zu verarbeiten und dabei sowohl kurzfristige als auch langfristige Abhängigkeiten in der Datenfolge zu erfassen – ein typisches Beispiel sind Texte, Sprache, oder Handschrift; vgl. Beck u. a. 2020, p.1-2.

das besonders bei der Erkennung von zusammenhängenden gedruckten Textzeilen eine hohe Genauigkeit bietet. Tesseract unterstützt neben modernen Schrifttypen auch historische Schriftsätze wie Fraktur, was es besonders geeignet für den Einsatz in digitalisierten Archiven macht.⁶³

Vorbereitend für den Einsatz von Tesseract müssen alle gescannten PDF in das JPEG Format umgewandelt werden, wofür ein kurzes Python-Script verwendet wird⁶⁴. Im praktischen Einsatz scheiterte die Integration von Tesseract jedoch an der Heterogenität des Korpus: uneinheitliche Layouts, wechselnde Schrifttypen, maschinen- und handschriftliche Texte sowie komplexe Textverläufe. Beispielhaft sollen hier überlagerte oder mehrspaltig angeordnete Passagen auf Postkarten und Zeitungsartikeln genannt werden, die zu massiven Erkennungsfehlern führen. Auch mit angepassten Segmentierungsparametern kann keine zufriedenstellende Texterkennung erzielt werden.

Tesseract wird daher nicht weiterverwendet.

4.5.2 LLM

Analog zum Einsatz von Python⁶⁵ stellt die Integration von Large Language Models (LLMs) von Beginn an einen zentralen Bestandteil der Projektkonzeption dar. Aus diesem Grund wird in einer frühen Phase auch der Einsatz von LLMs, spezifisch ChatGPT⁶⁶, bei der Transkription der Unterlagen erprobt.

Der Einsatz von LLMs wie ChatGPT für die Transkription historischer Quellen erweist sich als ambivalent. Während die Modelle nach gezielter Anleitung eine erstaunlich präzise Rekonstruktion von Layoutstrukturen und maschinell erfassten Textdaten leisten, bestehen erhebliche Einschränkungen. Im Detail sind das erhebliche Probleme bei der semantischen Genauigkeit. Das LLM beginnt schnell mit sinnverändernden Haluzinationen, die unklare Textpassagen aus dem gelernten Kontext stimmig auffüllt. Eine genaue Transkription, mit forcierter Notation von unklaren Stellen⁶⁷ gelingt in der Regel nicht. Die Verarbeitung handschriftlicher Dokumente scheitert weitgehend und führt zu stark spekulativen oder fehlerhaften Inhalten.

⁶³vgl. Weil, Pugin und Dovev 2025.

⁶⁴vgl. Burkhardt 2025b.

⁶⁵siehe Abschnitt [Tesseract](#)

⁶⁶eine detaillierte Erläuterung findet sich in [OpenAI – ChatGPT](#)

⁶⁷Beispielsweise durch das Einfügen von „[...]“

Hinzu kommen zu Projektbeginn technische Begrenzungen: Da ein API-Zugang zu OpenAI noch nicht verfügbar ist, erfolgt der Zugriff über die Weboberfläche. Diese stösst bei umfangreichen Eingaben rasch an Kapazitätsgrenzen; Sitzungen brechen häufig ab oder lassen sich nicht zuverlässig fortsetzen. Das kann zu Inkonsistenzen in der Promptstrukturierung und dem Kontext des LLMs führen, was wiederum direkten Einfluss auf die Verarbeitung der Unterlagen hat.

Zum Zeitpunkt der explorativen Nutzung stellt der integrierte Content-Filter der Modelle das zentrale Hindernis dar. Inhalte mit Bezug zum Nationalsozialismus führen zu einem sofortigen Abbruch der Verarbeitung. Beispielhaft sollen etwa Grussformeln wie „*Heil Hitler*“ genannt sein. Auffällig ist jedoch, dass sich diese Filtermechanismen durch alternative Schreibweisen in den Quellen umgehen lassen. Die Schreibweise „*Heil – Hitler*“ umgeht den Filter komplett und wird ohne Einschränkung transkribiert.

Ohne den Zugang zur API und dem damit notwendigen Umweg über den Webclient zeigt sich zudem, dass LLMs ohne persistente Promptstrukturierung dazu neigen, wichtige Hintergrundinformationen zu vergessen. Eine Kombination aus Ground-Truth-gestützter Anleitung und manuellem Review ist daher notwendig, um eine verlässliche Transkription zu gewährleisten. Sie wird in dem Abschnitt [Large Language Models](#) näher ausgeführt.

Aus den genannten Gründen kommt auch eine Transkription mittels generativem LLM nicht zum Einsatz.

4.5.3 Transkribus

Transkribus ist eine webbasierte Plattform zur automatisierten Handschriftenerkennung (HTR) und Texterkennung (OCR), die sich seit ihrer Entwicklung im EU-Projekt READ (Recognition and Enrichment of Archival Documents)⁶⁸ als Standardwerkzeug in den digitalen Geschichtswissenschaften etabliert hat⁶⁹. Betrieben wird Transkribus durch die READ-COOP SCE, einer europäischen Genossenschaft.

Die Plattform bietet zwei zentrale Zugriffsmöglichkeiten: einerseits die schlanke Webanwendung *Transkribus Lite*, andererseits den *Expert Client*, eine umfangreiche Desktopsoftware zur Bearbeitung und Verwaltung grosser Dokumentenkorpora. Beide Varianten ermöglichen die Transkription von gescannten Dokumenten, die Annotation von struktu-

⁶⁸vgl. [Recognition and Enrichment of Archival Documents / READ / Projekt / Fact Sheet / H2020 2025](#).

⁶⁹vgl. Mühlberger 2019.

rellen und semantischen Einheiten sowie den Export in verschiedenen Dateiformaten.

Die Nutzung des Expert Clients erlaubt darüber hinaus eine detaillierte Kontrolle über Transkriptionsprozesse und das zugrunde liegende Datenmanagement. Über integrierte Schnittstellen lassen sich grosse Datenmengen effizient verwalten. Auch externe FTP-Clients können zur Anbindung an das interne Dateisystem verwendet werden, um beispielsweise umfangreiche Digitalisate in strukturierter Form einzubinden.

Ein zentrales Merkmal von Transkribus ist die Möglichkeit *Tags* zu vergeben. Diese umfassen sowohl strukturelle Merkmale (wie Abkürzungen, Unklarheiten oder Layout-Elemente), als auch semantische Einheiten (wie Personen, Orte, Organisationen und Daten). Tags können individuell erweitert oder angepasst werden und werden im XML-Export maschinenlesbar dargestellt.

Die Exportfunktion von Transkribus erlaubt den Download der Transkriptionen im standardisierten PageXML-Format. Dieses Format ist auf die langfristige Nachnutzung struktureller Informationen ausgelegt und bildet die Grundlage für weiterführende Auswertungsschritte, etwa in Digital Humanities-Projekten.

In der praktischen Handhabung zeigt sich jedoch eine teils deutliche Diskrepanz zwischen den im Interface sichtbaren Informationen und der tatsächlichen XML-Ausgabe. So werden beispielsweise benutzerdefinierte Abkürzungsaufösungen oder Listenstrukturen nicht zuverlässig im XML ausgegeben. Informationen, die manuell innerhalb der Transkriptionsumgebung gepflegt wurden, gehen im strukturierten Export unter Umständen verloren. Insbesondere bei Listenobjekten, etwa für Personenverzeichnisse oder Inventare, bleibt die XML-Struktur häufig leer. Dies lässt sich zwar durch den Export per Transkribus-API umgehen, die löst jedoch neue Probleme aus. Nach ersten Erkenntnissen scheint das so produzierte PageXML nicht den gängigen Konventionen zu entsprechen, wodurch im Preprocessing durch das LLM massive Störungen für Listen produziert werden.⁷⁰ Sie werden daher ohne Preprocessing durch das LLM in unbearbeiteten der Version des Transkribus Exports verwendet.

Diese Einschränkungen werden auch in aktuellen Studien festgestellt. So verweisen Capurro et al.⁷¹ im Rahmen ihrer Analyse mehrsprachiger Handschriftenkorpora auf signifikante Herausforderungen bei der automatisierten Layoutanalyse sowie bei der Verarbeitung

⁷⁰siehe [OpenAI – ChatGPT](#), Eine Liste aller fehlerhaften Dateien ist im Anhang [A.4](#)

⁷¹Capurro, Provatorova und Kanoulas [2023](#).

komplexer Dokumentstrukturen. Sowohl beim Tagging als auch bei der Postcorrection sei weiterhin eine umfangreiche manuelle Nachbearbeitung notwendig, um konsistente und weiterverwendbare Datenformate zu erzeugen.

Trotz dieser Limitierungen liegt der methodische Mehrwert von Transkribus insbesondere in der Möglichkeit, ein eigenes HTR-Modell auf Basis einer spezifischen Ground Truth zu trainieren. Dies erlaubt es, auf charakteristische Eigenschaften eines konkreten Korpus einzugehen und so die Character Error Rate (CER) gegenüber generischen Modellen deutlich zu reduzieren. Darüber hinaus kann durch strukturierte Annotation eine Grundlage für die spätere Modellbewertung oder den Vergleich mit LLM-basierten Verfahren geschaffen werden.

Insgesamt stellt Transkribus eine leistungsfähige Plattform zur initialen Bearbeitung und Annotation historischer Quellen dar. Die automatisierte Erkennung unterstützt den Einstieg in umfangreiche Korpora, ersetzt jedoch nicht die editorische Kontrolle und Nachbearbeitung. Gerade für forschungsorientierte Projekte mit Fokus auf strukturierte, semantisch angereicherte Daten bleibt eine kritische Auseinandersetzung mit den technischen Grenzen unerlässlich.

4.6 Large Language Models

Ein zentrales Werkzeug bei der Verarbeitung der historischen Quellen ist die weiter unten näher beschriebene Python-Pipeline, die auf der Verarbeitung von XML-Dateien basiert. Vorgreifend sei erwähnt, dass diese XML-Verarbeitung ein Large Language Model (LLM) zum Custom-Tagging nutzt. Nebst dem Tagging stellt das Programmieren dieser Pipeline eine der Kernherausforderungen dieses Forschungsprojekts dar. Für das Tagging und die Entwicklung der Pipeline werden verschiedene Large Language Models intensiv getestet und eingesetzt.

4.6.1 Msty

Um ein dafür geeignetes LLM zu evaluieren, werden zu Beginn des Projektes beispielhafte Prompts erstellt und deren Ergebnisse systematisch verglichen. Um diesen Vergleich zu erleichtern, wird die Desktop-Anwendung Msty⁷² eingesetzt. Zu den zentralen Funktionen gehören parallele Chatinterfaces („Parallel Multiverse Chats“), eine flexible Verwaltung

⁷²vgl. *Msty - Using AI Models made Simple and Easy* 2025.

lokaler Wissensbestände („Knowledge Stacks“)⁷³, sowie eine vollständige Offline-Nutzung ohne externe Datenübertragung. Msty dient dazu, verschiedene Modelle zu testen, durch die Parallel Multiverse Chats Antworten zu vergleichen und Konversationen strukturiert zu verzweigen und auszuwerten.

Hervorzuheben ist, dass dies kein klassisches Benchmarking auf Basis vergleichbarer Resultate ist. Es wird zu diesem frühen Projektzeitpunkt weder systematisch überprüft, welche Qualität der jeweilige Codeteil hat, noch wird gemessen, wie viel Prozent der Named Entities jeweils richtig erkannt werden. Der direkte Vergleich der getesteten LLMs liefert jedoch schnell ein Bild, welches Modell sich für die gleiche Aufgabe besser eignet. Erprobt werden zu Beginn des Projektes im November 2024 die folgenden Anbieter und Modelle:

- Alphabet – Gemini
- Anthropic – Claude
- OpenAI – ChatGPT

Sie sollen nachfolgend eingeordnet und deren Verwendung erläutert werden.

4.6.2 Alphabet – Gemini

Google Gemini⁷⁴ wird im Dezember 2023⁷⁵ von Google zunächst einer eingeschränkten Userzahl verfügbar gemacht und gilt als direkte Antwort auf OpenAIs ChatGPT. Es nimmt in dieser Arbeit keinen grossen Raum ein, da es sich im direkten Vergleich mit ChatGPT zum Zeitpunkt des Tests im Winter 2024 und Frühjahr 2025 als weniger präzise bei der Annotation von XML-Files und weniger zuverlässig beim Coden in Python herausstellte. Zur Falsifizierung dieser Annahme gelegentlich durchgeführte Überprüfungen im April und Juni 2025 brachten das selbe Ergebnis. Auch aus ökonomischen Gründen wurde auf ein zusätzlich zu OpenAI abgeschlossenes Abonnement verzichtet.

4.6.3 Anthropic – Claude Code

Claude Code wird am 22. Mai 2025 offiziell veröffentlicht⁷⁶ und ist bereits ab dem 24. Februar 2025 in einer öffentlichen Betaphase verfügbar. In diesem Projekt erfolgt die In-

⁷³vgl. *Msty - Using AI Models made Simple and Easy* 2025.

⁷⁴ehemals Google Bard

⁷⁵*Gemini – unser größtes und leistungsfähigstes KI-Modell 2023.*

⁷⁶*Claude Code 2025.*

tegration des Tools bereits sehr früh. Am 3. April 2025 wird es erstmals eingesetzt, um komplexe Aufgaben bei der Entwicklung der Verarbeitungs- und Analysepipeline direkt in VS-Code⁷⁷ zu übernehmen.

In den ersten Tagen erzielt Claude Code beeindruckende Resultate. Es unterstützt erfolgreich bei der Generierung von Programmcode, insbesondere bei strukturierenden Aufgaben bei der Initialisierung von Python-Funktionen. Einzelne Funktionsbausteine lassen sich durch das Tool effizient erstellen, was zunächst zu einer spürbaren Beschleunigung des Entwicklungsfortschritts führt.

Im weiteren Verlauf zeigen sich jedoch klare Begrenzungen. Bereits im April ist der Kontextumfang des Projekts so gross, dass Claude Code Schwierigkeiten hat, über mehrere Module hinweg konsistent zu arbeiten. Eine zentrale Schwäche besteht darin, dass projektweite Variablen nicht zuverlässig erkannt und übergeben werden. So kann etwa die Variable `mentioned_persons` aus dem Modul `person_matcher.py` nicht als wichtig erkannt und korrekt in `letter_matcher.py` eingebunden werden.

Ein weiteres Problem ist die tiefgreifende Veränderung bestehender Funktionsstrukturen. Claude Code verändert mitunter voll funktionsfähige Module, ohne auf deren interne Abhängigkeiten Rücksicht zu nehmen. Diese Eingriffe führen häufig dazu, dass vormals stabile Komponenten nicht mehr korrekt ausgeführt werden. Variablennamen werden ohne Anzeige modifiziert, Dictionaries durch Listen ersetzt. Der Aufwand für das anschliessende Debugging übersteigt in vielen Fällen den Nutzen der automatisierten Generierung. Zwar können einzelne Änderungsschritte von Claude Code angezeigt werden, Veränderungen können aber so subtil sein, dass sie oft unbemerkt bleiben.

Hinzu kommen erhebliche Nutzungskosten. Aufgrund der Projektgrösse entstehen bereits im April für einzelne Prompts Kosten von bis zu vier US-Dollar. Durch präzises Prompting⁷⁸ lässt sich dieser Betrag zwar begrenzen, doch unterschreitet der Preis pro Anfrage selten 0,40\$. Die Arbeit mit Claude Code ist damit nicht nur zeitintensiv durch den notwendigen Korrekturaufwand, sondern auch kostenintensiv im operativen Betrieb.

Nach einer intensiven Probephase von etwa fünf Wochen wird der Einsatz von Claude Code im Projekt vor dem Public Release beendet. Die Entscheidung basiert auf einer kritischen Abwägung von Nutzen, Aufwand und Nachhaltigkeit im Hinblick auf die lang-

⁷⁷**Abk.:** *Visual Studio Code*, die verwendete Programmierumgebung

⁷⁸Beispielsweise durch Angabe von Dateipfaden und Zeilennummern

fristige Wartbarkeit des Codes.

4.6.4 OpenAI – ChatGPT

Im Verlauf der Arbeit wird ChatGPT intensiv in mehreren Funktionen genutzt. Dazu zählen Generierung und Überarbeitung von Quellcode, Fehleranalyse in Python- und LaTeX-Skripten, strukturierte Formulierung von Dokumentationsinhalten⁷⁹ sowie die semantische Annotation von Personen, Orten und Ereignissen im Transkriptionskorpus. Zum Einsatz kommen insbesondere die Modelle *GPT-4o*, *GPT-4.5*, *o3-mini* sowie deren Varianten *mini high* und *GPT-4.1 mini*⁸⁰. Bereits vor der eigentlichen Konzeptionsphase erfolgen erste Versuche mit dem Vorgängermodell *GPT-3.5*, um grundlegende Anwendungsbereiche für historische Datenverarbeitung auszuloten. Im Folgenden sollen die drei Einsatzfelder **Custom-GPT**, **Custom-Projekt** und **API** beschrieben und eingeordnet werden.

Custom-GPT

Ein speziell für das Projekt konfiguriertes *Custom-GPT*⁸¹ wird in dieser Frühphase eingerichtet und dient als initialer Funktionstest für den Einsatz nicht-domänenspezifischer multimodaler Sprachmodelle. Dieses Modell⁸² basiert auf projektspezifischen Anweisungen, Orts- und Personenlisten sowie Ground Truth-Daten aus dem Korpus des Männerchors. Es stellt damit eine Vorform der heute verfügbaren „Projekte“-Funktion im Webinterface von ChatGPT dar. Konfiguriert wird es auf die Verarbeitung und Analyse historischer Dokumente. Zum Einsatz kommen in der Vorphase des Projektes erste Transkriptionen, Metadaten aus der *Akten_Gesamtübersicht.csv* und ein erster kleiner Korpus sowie ein Manuskript von einem Chormitglied⁸³. Getestet wird, ob das Modell erkannte Personennamen mit einer bereitgestellten Namensliste vergleichen kann und die Verarbeitung diverser Formate (CSV, Excel, PDF, Bilddateien, Klartext) unterstützt.

Der Abgleich mit den hinterlegten Namenslisten erweist sich als unzureichend. Die Ergebnisse stellen sich als methodisch unzuverlässig bzw. nicht reproduzierbar heraus. Auch eine Weiterverarbeitung durch eine vermeintlich direkte Abfrage von Wikidata zur semanti-

⁷⁹in der Code-Dokumentation, Modulzusammenfassung oder beim Überarbeiten dieses Textes

⁸⁰vgl. [Model Release Notes 2025](#).

⁸¹**Abk.:** GPT = Generative Pretrained Transformer

⁸²vgl. [GPT Modell forschung-mannerchor-murg 2025](#).

⁸³Durst 1948, Dieses autobiografische Manuskript wurde anfangs noch in den Korpus zu integrieren versucht, später jedoch entfernt. Eingesetzt wird es in der Verifikation der Archivforschung (bspw. im Militärarchiv Freiburg).

schen Anreicherung historischer Entitäten bleibt ohne belastbare Resultate. Wikidata-IDs werden willkürlich generiert, so verweist die angebliche ID für München auf die Golden Gate Bridge. Gleichwohl zeigen sich punktuelle Stärken bei der automatisierten Erkennung von Strings, etwa bei der Identifikation potenzieller Personen- oder Ortsnamen. Dies legt den Grundstein für den Preprocessing-Schritt⁸⁴ und die später automatisierte NER-Pipeline⁸⁵. Im Dezember 2024 führt OpenAI „Custom-Projekte“ ein. Ab diesem Zeitpunkt erfolgt der Wechsel zur Projektfunktion, die im Folgenden beschrieben wird.

Custom-Projekt

Projekte ermöglichen eine persistente, thematisch fokussierte Interaktion mit dem Modell über längere Zeiträume hinweg. Dabei wird kontextuell relevantes Hintergrundwissen – etwa über Datenstrukturen, verwendete Tools, Entitätenlisten oder Ground Truth-Dateien – dauerhaft im System gespeichert. Diese Informationen stehen in allen Konversationen innerhalb des Projekts zur Verfügung, ohne erneut übergeben werden zu müssen. Im Unterschied zu den oben genannten *Custom GPTs*, bei denen ein Modell über eine Benutzeroberfläche gezielt mit Systemmeldungen und Beispieldaten konfiguriert wird, basiert das Projektkonzept nicht auf einer einmaligen statischen Initialisierung, sondern auf fortlaufender Kontexterweiterung durch Nutzerinteraktionen. Projekte sind nicht öffentlich verfügbar, nicht durch Dritte nutzbar, und bieten keine explizite Konfigurationsmaske. Für dieses längerfristige Forschungsvorhaben mit sich entwickelnden Anforderungen eignet sich die Projekt-Funktion gut. Sich entwickelnde Änderungen im Code können nicht nur durch regelmässigen Uploads neuer Dateien abgebildet werden, sondern werden fortlaufend durch die Interaktion mit dem Modell erweitert, ergänzt und angepasst. So erhält das Projekt auch einen Überblick über andere Module, an denen gerade gearbeitet wird, und schliesst sie in den Antworten ein. Insgesamt werden in dem Custom-Projekt „Masterarbeit“ 17 Dateien hochgeladen, darunter fallen die Ground Truth-Dateien und die einzelnen Python-Module. Der Hauptfokus des Custom-Projektes liegt auf der Unterstützung beim Coden der im Kapitel [Module im Detail](#) erläuterten Codeteile. Mit dem Update auf ChatGPT 5⁸⁶ Anfang August scheint diese Funktion jedoch weniger präzise Ausgaben zu machen, als mit dem vorangegangenen GPT-4.5. Die Grad der Halluzination wirkt deutlich höher, ohne dass dies wegen des geringen zur Verfügung stehenden Zeitrahmens durch Benchmarking belegt werden könnte.

⁸⁴vgl. [Vorverarbeitung](#)

⁸⁵vgl. [Hauptmodul – Transkribus_to_base](#)

⁸⁶OpenAI [2025](#).

API

Für die automatisierte Verarbeitung und Annotation historischer Dokumente wird in diesem Projekt die Programmierschnittstelle (API) von OpenAI verwendet. Die API ermöglicht eine präzise Steuerung zentraler Parameter wie den Modelltyp, die Kontextlänge und die Temperatur⁸⁷. Die Nutzung der API erleichtert die Integration in automatisierte, skalierbare Workflows und erlaubt eine Wiederverwendung von Prompts über grosse Dokumentenmengen hinweg. Gleichwohl ist die tatsächliche Reproduzierbarkeit der Ergebnisse nur eingeschränkt gegeben. Selbst bei identischen Eingaben und konstanten Parametern kann es zu leichten Variationen in den Ausgaben kommen, insbesondere bei höheren Temperatureinstellungen oder bei Modellen mit probabilistischer Antwortgenerierung. Reproduzierbarkeit ist daher in diesem Kontext primär als funktionale Replizierbarkeit von Abläufen, weniger als identische Ergebniswiedergabe zu verstehen.⁸⁸ So ist eine der Grundannahmen dieser Arbeit, dass ChatGPT den Prompt exakt so ausführt und den Inhalt der historischen Texte komplett unverändert lässt. Dies soll in einem späteren Projektschritt durch eine automatisierte Pipeline überprüft werden ⁸⁹.

Es zeigt sich jedoch bereits jetzt, dass die Texte bei einer Transkribus-Rückgabe verändert werden. Alle Dokumente, die in Transkribus statt als **Textregion** als **Listenobjekt** ausgezeichnet wurden, produzieren Fehler.⁹⁰ Angenommen werden muss, dass der Listen-Export üblichen XML-Konventionen nicht entspricht, weshalb das LLM Fehler beim Schliessen und Öffnen der Tags produziert. Ausserdem sind diese Listen-PageXML deutlich länger. Dies führt nach ersten Erkenntnissen zum Überschreiten des Token-Limits der API, die Rückgabe bricht mitten im Dokument ab. Das löst wiederum erstgenannten Fehler aus.

Ein zentrales Anwendungsfeld der API im Projekt ist die Named Entity Recognition, die über prompt-basierte Anfragen realisiert wird. Dabei kann auf ein aufwendiges Modelltraining und damit einhergehende Groundthruth-Generierung verzichtet werden, da LLMs wie GPT-4 auch ohne Fine-Tuning eine kontextsensitive Entitätserkennung ermöglichen. Bereits 2020 verweisen Brown et al. darauf, dass sich die Lösung nicht explizit trainierter

⁸⁷Die Temperatur steuert den Grad an Zufälligkeit in der Textgenerierung. Bei niedrigen Werten (z.B. 0,2) bevorzugt das Modell sehr wahrscheinliche Ausgaben; bei höheren Werten (bis 1,0) steigt der Anteil weniger wahrscheinlicher, "kreativer" Antworten.

⁸⁸Ähnliche Ergebnisse finden sich auch vgl. J. J. Wang und V. X. Wang 2025, S.2 + S.4 + S.6.

⁸⁹Beispielsweise durch stichprobenartigen Abgleich einzelner Page-XMLs vor und nach der Verarbeitung, einem Stripping aller Tags und einem direkten Stringvergleich mit Levenshtein und einer ausgegebenen Check-Summe. Aufgrund zeitlicher Limitationen ist dies jedoch aktuell nicht der Fall

⁹⁰nicht nur bei ChatGPT, sondern auch im Export selbst, vgl. [Transkribus](#)

Tasks mit zunehmender Grösse des Models verbessert:

*Each increase has brought improvements in text synthesis and/or downstream NLP tasks, and there is evidence suggesting that log loss, which correlates well with many downstream tasks, follows a smooth trend of improvement with scale [KMH+20]. Since in-context learning involves absorbing many skills and tasks within the parameters of the model, it is plausible that in-context learning abilities might show similarly strong gains with scale.*⁹¹

Dementsprechend wird OpenAIs LLM in diversen konzeptionellen und operativen Phasen dieses Projektes verwendet. Konkrete Beispiele hierfür finden sich unter anderem in dem Kapitel [Vorverarbeitung](#).

5 Pipeline

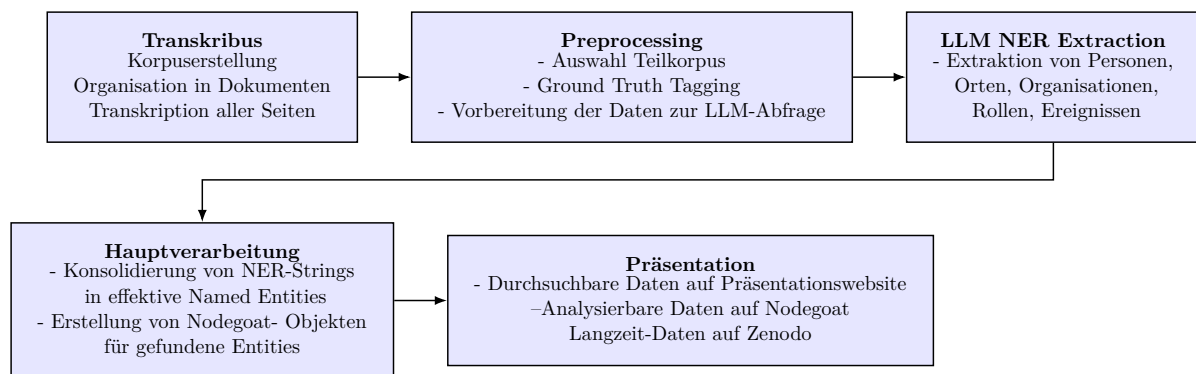


Abbildung 8: Übersicht aller Verarbeitungsschritte

Der Ablauf vom analogen Dokument bis zur Visualisierung aller Ergebnisse lässt sich in fünf Schritte gliedern. Sie reichen von der Korpusbildung und Transkription über die Korpusbildung hin zu einer neuartigen Named Entity Recognition durch ein LLM. Diese Named Entities werden im Anschluss in einem modular gestalteten Programm konsolidiert. Alle Ergebnisse werden auf einer Website durchsuchbar dargestellt.

5.0.1 Pipeline im Detail

Die untenstehende Grafik soll im Folgenden als visuelle Orientierung dienen. Sie bildet die logische Gliederung der Pipeline ab, wie sie im weiteren Verlauf des Kapitels analysiert wird. Als Schematische Zeichnung wird sie als Referenz in den nachfolgenden Modulen verwendet. So sollen die verschiedenen Verarbeitungsschritte und Modulabhängigkeiten eingeordnet werden. Als zentrale Übersicht gliedert sich diese Darstellung in drei farblich differenzierte Ebenen.

⁹¹vgl. Brown u. a. 2020, S.4.

Grün markiert sind externe Ressourcen, die die Grundlage der Verarbeitung bilden. Dazu zählen zum einen XML-Dateien, die aus dem Transkriptionsprogramm Transkribus stammen, und zum anderen strukturierte Ground Truth-Daten im CSV-Format, die zuvor aus Nodegoat exportiert wurden.

Orange steht für die zentralen Verarbeitungsschritte der Pipeline. Dazu gehört insbesondere das Preprocessing durch das LLM, sowie das Hauptmodul `transkribus_to_base.py`. Dieses Modul koordiniert den gesamten Verarbeitungsablauf.

Blau gekennzeichnet sind die einzelnen Funktionsmodule, in die sich `transkribus_to_base.py` unterteilt. Diese übernehmen spezialisierte Aufgaben, etwa die Erkennung und Anreicherung von Personen, Orten, Organisationen oder Ereignissen.

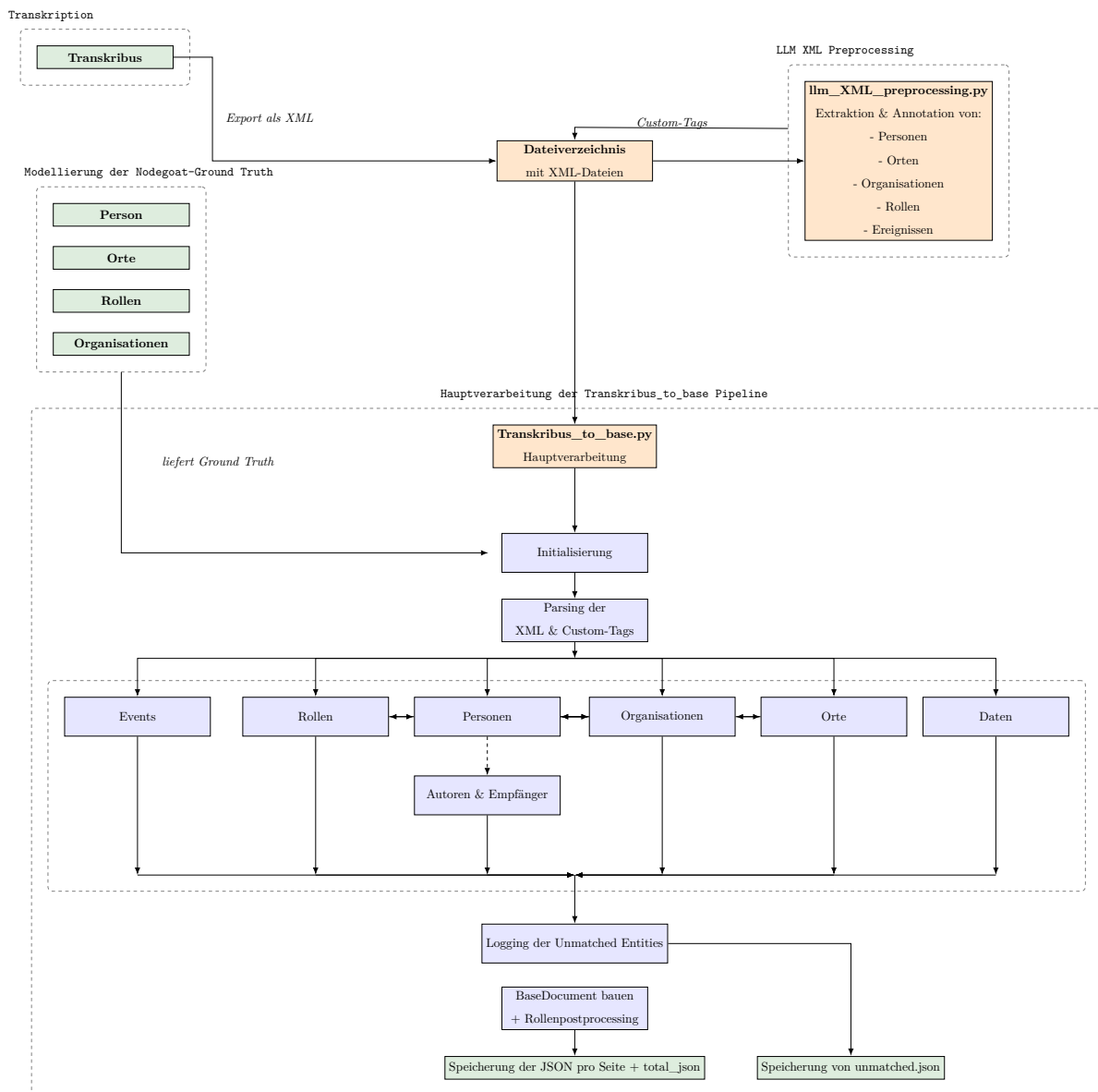


Abbildung 9: Übersicht der gesamten XML-to-JSON-Pipeline

Eine detaillierte Beschreibung der jeweiligen Prozesse erfolgt in den Abschnitten [Transkriptionen \(Methodenvergleich\)](#) und [Nodegoat](#) sowie [Vorverarbeitung](#), [Hauptmodul – Transkribus_to_base](#) und [Module im Detail](#).

5.1 Vorverarbeitung

Neben Transkribus, das bei der Transkription einen elementaren Schritt in der Vorverarbeitung aller Dokumente darstellt, müssen die Seiten für bessere Ergebnisse weiter vorbereitet werden. Die Kernherausforderung der vorliegenden Arbeit ist die Named Entity Recognition (NER). Wie bereits im Abschnitt [Transkribus](#) ausgeführt, werden viele Inhalte bereits während des Transkriptionsprozesses manuell auf Basis von im Anhang erläuterten Regeln getaggt⁹². Dieser händische Prozess erlaubt hohe Genauigkeit. Deshalb überschreitet er aber den begrenzten zeitlichen Rahmen aufgrund der Menge an Unterlagen. Daher wird für das Projekt auch eine zweite Verarbeitungsform gewählt, eine **NER durch ein LLM**.

Im Zentrum des Vorverarbeitungsskripts steht eine strukturierte Anbindung an die OpenAI-API, um die PAGE-XML-Dateien aus dem Transkribus-Export automatisiert mit Annotationen anzureichern. Das Skript `llm_xml_preprocessing.py` ist modular aufgebaut und folgt einer klar definierten Abfolge von Verarbeitungsschritten.

Die Vorverarbeitung beginnt mit der Funktion `get_api_client()`, die eine Verbindung zur OpenAI-Programmierschnittstelle aufbaut. Dabei wird der API-Schlüssel über eine Umgebungsvariable geladen und für spätere Anfragen bereitgestellt. Die zentrale Annotation erfolgt in der Funktion `annotate_with_llm()`, die den vollständigen Unicode-Text der XML-Datei verarbeitet und dem Modell GPT-4o übergibt. Grundlage ist ein präzise formulierter Prompt⁹³, der die Struktur des zu erwartenden Outputs definiert. Der Prompt spezifiziert, dass ausschliesslich `<TextLine>`-Elemente bearbeitet und mit einem `custom`-Attribut versehen werden dürfen. Innerhalb dieses Attributs werden ausschliesslich tatsächlich erkannte Entitäten in einheitlichem Format codiert, darunter Personen (`person`), Rollen (`role`), Orte (`place`), Organisationen (`organization`), Daten (`date`) sowie autoren- und empfängerbezogene Markierungen (`author`, `recipient`, `creation_place`). Für jedes Tag gibt es genaue Anweisungen an das Modell. Hinzu kommen Einstellung zur Temperatur, die in dem Prompt auf 0 gesetzt wird.⁹⁴ Auch ein Beispielresultat der Annotation wird jedes Mal mitgeliefert, um möglichst wenig Varianz in den Antworten zu erhalten. Gleichzeitig liefert das Beispielresultat auch Informationen

⁹²vgl. Anhang [Tagging in Transkribus](#)

⁹³vgl. Anhang [Prompt der LLM Vorverarbeitung](#)

⁹⁴Die Temperatur steuert den Grad an Zufälligkeit in der Textgenerierung. Bei niedrigen Werten (z.B. 0,2) bevorzugt das Modell sehr wahrscheinliche Ausgaben; bei höheren Werten (bis 1,0) steigt der Anteil weniger wahrscheinlicher, "kreativer" Antworten.

über Abkürzungen, die nicht von dem Modell als Organisationen erkannt werden. Das „WhW - das Winterhilfswerk“ ist eine Abkürzung, die offenbar nicht in den Trainingsdaten des Modells vorkommt. Obwohl es sich hierbei um ein generelles Modell handelt, erlaubt dieses Vorgehen stabile und brauchbare Daten.

Die Antwort des Sprachmodells ist eine vollständige XML-Datei, die sämtliche bestehenden Strukturinformationen beibehalten soll. Der Rückgabewert wird zunächst in der Funktion `clean_llm_output()` auf mögliche Formatierungsfehler überprüft, gesammelt, und manuell bereinigt.⁹⁵ Diese Funktion extrahiert den tatsächlichen XML-Inhalt aus dem Rückgabe-String, etwa wenn dieser durch Markdown-Wrapper wie ````xml-content```` eingefasst wurde.

Die konkrete Verarbeitung einzelner Dateien erfolgt über die Funktion `process_file()`, die eine originale Transkribus-XML-Datei einliest, an das Modell übergibt, das Ergebnis prüft und anschliessend unter verändertem Dateinamen (Suffix `_preprocessed`) abspeichert. Vor dem Schreiben erfolgt eine strukturelle Validierung mittels XML-Parser, um syntaktische Fehler oder unvollständige Rückgaben zu erkennen. Fehlermeldungen werden protokolliert, fehlerhafte Dateien übersprungen.

Die eigentliche Ausführung der Batch-Verarbeitung wird durch die `main()`-Funktion gesteuert. Diese durchläuft das in `TRANSKRIBUS_DIR` konfigurierte Arbeitsverzeichnis, wobei alle Unterordner der Form `<7-stelliger Ordner>/Akte_<Nummer>/page/` rekursiv analysiert werden. XML-Dateien, die bereits mit `_preprocessed` enden, werden übersprungen. Zusätzlich wird ein Verzeichnis ignoriert, wenn bereits mehr als fünfzig Prozent der Dateien annotiert wurden. Die verbleibenden Dateien werden schrittweise mit dem Modell verarbeitet. Hintergrund ist eine kosteneffiziente Verarbeitung, die verhindern soll, dass Dateien mehrfach durch die API bearbeitet werden. Die Funktion `annotate_with_llm()` berechnet darüber hinaus die Anzahl der vom Modell verarbeiteten Eingabe- und Ausgabetokens. Auf Basis dieser Werte wird für jede Datei eine Schätzung der anfallenden API-Kosten vorgenommen. Diese Informationen werden für jede Anfrage protokolliert, um eine transparente Kostenkontrolle sicherzustellen.

Am Ende dieses Prozesses entsteht pro annotierter Seite eine neue, syntaktisch validierte XML-Datei, die alle Annotationen als `custom`-Attribute enthält. Diese dienen in den nachfolgenden Modulen der Pipeline (insbesondere `transkribus_to_base.py`) als Grundlage für die strukturierte Extraktion und Validierung von Entitäten.

⁹⁵vgl. 2

5.2 Hauptmodul – Transkribus_to_base

Das Skript `Transkribus_to_base.py` bildet das zentrale Haupt- oder Orchestrierungsmodul des Projekts. Es ist der Ausgangspunkt für alle nachgelagerten Verarbeitungsschritte⁹⁶ und ermöglicht erst deren korrekte Ausführung, indem es systematisch auf die spezialisierten Extraktions- und Validierungsmodule zugreift. Das Modul überführt die aus Transkribus exportierten XML-Dateien in eine validierte JSON-Repräsentation, die dem projektspezifischen Basischema (`document_schemas.py`) entspricht. Die JSON-Daten bilden die Grundlage für weitere Analysen und der Darstellung auf der Projektwebsite, wobei alle erkannten Entitäten eindeutig und konsistent modelliert werden.

Initialisierung und Konfigurationsprüfung

Bevor die eigentliche Verarbeitung eines XML-Dokuments beginnt, lädt das Modul Informationen zu `Personen`, `Orten`, `Organisationen` und `Rollen`, die aus *Nodegoat*-Exporten stammen. Diese Dateien werden jeweils am Anfang des Skripts eingelesen und für die gesamte Dauer des Verarbeitungsprozesses im Speicher gehalten.

Darüber hinaus werden im Rahmen der Initialisierung verschiedene Konfigurationsprüfungen durchgeführt. So wird sichergestellt, dass alle benötigten Schlüssel für externe APIs (z.B. Sprachmodell- oder Wikidata-Zugriff) vorhanden sind und erforderliche Systempfade sowie Debug-Verzeichnisse korrekt gesetzt sind. Erst wenn diese Vorbedingungen erfüllt sind, wird die eigentliche Dokumentenverarbeitung gestartet.

Das Modul ist dabei nicht nur auf die Verarbeitung einzelner XML-Dateien beschränkt, sondern analysiert automatisiert ganze Aktenverzeichnisse. Über die Funktion `process_transkribus_directory()` werden alle enthaltenen Unterordner rekursiv durchsucht, wobei die enthaltenen XML-Dateien jeweils identifiziert, eingelesen und verarbeitet werden. Die Pipeline sucht gezielt nach Daten, die die Vorverarbeitung durch `llm_xml_preprocessing.py` durchlaufen haben. Die Steuerung erfolgt dabei über die in der Vorverarbeitung entstandenen Ordnerstruktur. Diese wird nach der siebenstelligen Akten-ID und dem Unterordner `pages` durchsucht. Anschliessend werden die dort hinterlegten XML-Dateien geöffnet und verarbeitet. Die Funktion `process_subdirectories()` ermöglicht dabei die getrennte Verarbeitung einzelner Seiten einer Akte, während `main()` als zentraler Einstiegspunkt fungiert und den gesamten Batch-Verarbeitungsprozess steuert.

XML-Verarbeitung und Dokument-Identifikation

Die Verarbeitung beginnt daher mit dem Einlesen der XML-Datei. In einem ersten Schritt werden aus dem XML-Header Informationen wie `docId`, `pageId`, die Collection-ID (`tsid`) sowie

⁹⁶vgl. [Module im Detail](#)

Referenzen zu Bild- und Quelldateien extrahiert. Diese Metadaten bilden die Basis für die eindeutige Identifikation jeder Seite. Zusätzlich wird aus dem Dateinamen mit einem Lookup in der `Akten_Gesamtübersich.csv` das Dokumentformat (z.B. Brief, Postkarte, Protokoll) abgeleitet. Dieses wird später im Feld `document_type` gespeichert und dient der Klassifikation und zielgerichteter Weiterverarbeitung.⁹⁷

Extraktion des Transkripts

Parallel dazu wird der vollständige Transkriptionstext aus den `<TextEquiv>` bzw. `<Unicode>`-Blöcken extrahiert. Der daraus resultierende Fliesstext bildet die zentrale Grundlage für alle nachgelagerten Analyseverfahren.

Die in den einzelnen `TextLine`-Elementen gespeicherten `custom`-Attribut werden im nächsten Schritt durch die zentrale Funktion `extract_custom_attributes()` systematisch ausgelesen. Jede Zeile wird dabei einzeln auf enthaltene Custom-Tags geprüft. Wird ein solches erkannt, erfolgt eine strukturierte Extraktion der in den Attributen angegebenen `offset`- und `length`-basierten Textsegmente.

Verwendung interner Hilfsfunktionen

Zur robusten Verarbeitung der teilweise komplex verschachtelten XML-Annotationen nutzt das Modul eine Reihe projektinterner Hilfsfunktionen. Diese übernehmen spezifische Parsing- und Normalisierungsaufgaben und gewährleisten eine zuverlässige Interpretation der extrahierten Textsegmente.

Beispielsweise extrahiert die Funktion `extract_wikidata_id()` maschinenlesbare Wikidata-Referenzen aus den `custom`-Attributen, während `parse_custom_attributes()` die Attributstruktur einzelner `TextLine`-Elemente in ein standardisiertes Dictionary überführt. Weitere Funktionen wie `normalize_role_name()`, `correct_swapped_name()` oder `get_place_name()` sorgen für eine einheitliche Formatierung von Rollenbezeichnungen, Personennamen und Ortsangaben.

Diese Hilfsfunktionen bilden die technische Grundlage für die korrekte Zerlegung, Bereinigung und Weiterverarbeitung der XML-Anteile.

Standardstruktur und Ground Truth

Alle extrahierten Entitäten werden in einer standardisierten Datenstruktur gesammelt, die nach fünf Kategorien gegliedert ist: `persons`, `roles`, `organizations`, `places` und `dates`. Diese bilden das zentrale Ausgangsmaterial für alle nachfolgenden Verarbeitungs- und Validierungsschritte sowie für die finale Konvertierung in ein `BaseDocument`-konformes JSON-Format. Die

⁹⁷beispielsweise im [Assigned_Roles_Module.py](#)

Extraktion selbst wird durch spezialisierte Subfunktionen durchgeführt, die jeweils auf eine bestimmte Entitätskategorie ausgerichtet sind.

Dabei greifen die Extraktionsfunktionen intern jeweils auf die initial geladenen Ground Truth-Dateien zurück. Zusätzlich zu Custom-Tags werden Organisationen direkt im Fliesstext mittels `match_organization_from_text()` identifiziert und in die Datenstruktur übernommen. Grund hierfür ist das fehlende Training des LLMs für nationalsozialistische Organisationen und deren Akronyme.

Fallback mit Fliesstextextraktion

Zusätzlich zu den durch das Sprachmodell erzeugten Custom-Tags werden weitere Entitäten heuristisch aus dem Fliesstext erkannt. Besonders betrifft dies Organisationen, die mittels `match_organization_from_text()` identifiziert und in die Datenstruktur übernommen werden. Grund hierfür ist das fehlende Training des LLMs für nationalsozialistische Organisationen und deren Akronyme. Auch Rollenbezeichnungen, die in unmittelbarer Nähe zu Personennamen vorkommen, werden im Kontext so regelbasiert durch `assign_roles_to_known_persons()` extrahiert. Auch hier wird das Ergebnis validiert und, sofern die Rolle einer standardisierten Ontologie entspricht, in das Feld `role_schema` überführt.

Die Kombination der verschiedenen Erkennungsmethoden kann zu Duplikaten führen. Um konsistente und eindeutige Entitäten zu erzeugen, erfolgt ein deduplizierender Abgleich über die Funktion `deduplicate_and_group_persons()`. Diese vergleicht alle Personen aus den Kategorien `authors`, `recipients` und `mentioned_persons` untereinander. Dabei werden vorhandene Scores (wie `match_score`, `recipient_score` und `confidence`) zusammengeführt und priorisiert.

Das so angereicherte Dokument wird in ein Objekt der Klasse `BaseDocument` überführt. Dieses enthält strukturierte Felder für Metadaten, Volltext, Autoren, Empfänger, erwähnte Personen, Orte, Organisationen, Datumsangaben und Ereignisse. Jede Entität wird gemäss den Typdefinitionen in `document_schemas.py` validiert. Ein abschliessender Validierungsschritt erfolgt über `validate_extended()`, das auf Fehler in der Struktur, Inkonsistenzen oder fehlende Pflichtfelder prüft.

Abschliessend wird das Dokument im JSON-Format gespeichert. Neben der Einzelseite wird auch eine aggregierte Datei `total_json.json` fortgeschrieben, in der alle Seiten einer Akte gesammelt werden. Zusätzlich wird für jede Seite eine Prüfung auf nicht zuordenbare Entitäten durchgeführt. Diese werden in der Datei `unmatched.json` gespeichert, um eine spätere manuelle Nachbearbeitung zu ermöglichen. Das Ergebnis dieser Konvertierung bildet die Grundlage für die weitere Verarbeitung in `Nodegoat` sowie für die explorative Analyse der Akteursnetzwerke.

Debugging und Logging nicht auflösbarer Entitäten

Während der Verarbeitung werden potenziell fehlerhafte oder unvollständige Entitäten automatisch markiert und protokolliert. Dazu zählen insbesondere Personen ohne Namen, mit unklarer Rollenbezeichnung oder nicht eindeutiger Zuordnung. Solche Fälle erhalten in den jeweiligen Modulen einen `match_score` unterhalb definierter Schwellenwerte sowie ein entsprechendes `review_reason`, um sie für eine manuelle Nachbearbeitung zu kennzeichnen.

Die Funktion `log_unmatched_entities()` sammelt alle nicht gematchten oder unvollständigen Entitäten und speichert sie in einer separaten Datei `unmatched.json`. Dadurch wird eine systematische Qualitätssicherung und Nachkorrektur der Extraktionsergebnisse ermöglicht. Zusätzlich erfolgen gezielte `[DEBUG]`-Ausgaben in der Konsole, die auf kritische Fälle hinweisen und die manuelle Überprüfung unterstützen.

Dieser Logging-Mechanismus bildet die Brücke zwischen automatisierter Extraktion und kuratierender Nachbearbeitung und ist damit ein zentrales Qualitätsinstrument der Pipeline. Auf dieser Basis kann, wie im Kapitel 5.3.8 beschrieben, die Ground Truth erweitert werden.

5.3 Module im Detail

Diese im Abschnitt beschriebenen Prozesse steuern jeweils die Verarbeitung der jeweiligen Entitäten in den spezialisierten Modulen der Pipeline. Der nachfolgende Abschnitt soll einen detaillierten Überblick über die zugrundeliegenden Probleme bei der NER, und den in dieser Arbeit vorgeschlagenen Lösungsansätzen für diesen spezifischen Quellenkorpus liefern. Dabei werden die jeweiligen Verarbeitungsschritte entlang des Codes präsentiert, der für diese Arbeit ausgearbeitet wurde.

5.3.1 `document_schemas.py`

Neben der technischen Konsistenz bei der Zusammenarbeit der Module braucht es auch eine inhaltliche Qualitätssicherung aller extrahierten Inhalte aus den Transkribus-Dokumenten des Projekts. Hierfür definiert das Modul `document_schemas.py` die zentrale Schema- und Datenstruktur zur Modellierung. Es gewährleistet die einheitliche Repräsentation, Serialisierung und Validierung der im Projekt verarbeiteten Entitäten und ihrer Relationen. Die definierten Klassen bilden die Grundlage für die JSON-Ausgabe der angereicherten Dokumente und dienen zugleich der Nachvollziehbarkeit und strukturierten Weiterverarbeitung in externen Anwendungen wie Nodegoat. Diese Schemata sollen nachfolgen erläutert werden, da sie das Fundament für die Verarbeitung von Entitäten in den jeweiligen Modulen bilden.

BaseDocument

Alle genannten Entitäten werden im zentralen Objekttyp `BaseDocument` zusammengeführt. Es handelt sich um eine abstrakte Parent-Class, von der alle spezifischeren Child-Class abgeleitet werden. . Sie bildet die Grundlage für die strukturierte Speicherung jedes einzelnen Dokuments und enthält unter anderem:

- einen Attributblock mit `document_type`, `object_type`, `creation_date`, `creation_place`, `recipient_place`
- Listen der `authors`, `recipients` und `mentioned_persons`
- Listen von `mentioned_organizations`, `mentioned_places`, `mentioned_events` und `mentioned_dates`
- die Originaltranskription (`content_transcription`) sowie inhaltliche Tagging-Kategorien (`content_tags_in_german`)
- optionale Zusatzinformationen im Feld `custom_data`, z.B. Zwischenoutputs oder Debug-Daten

Zur automatisierten Konvertierung vom oder zum JSON-Format stehen die Methoden `to_dict()`, `from_dict()`, `to_json()` und `from_json()` zur Verfügung. Darüber hinaus erlaubt die Methode `validate()` eine strukturierte Prüfung der Einträge auf Konsistenz und Vollständigkeit, etwa im Hinblick auf unvollständige Daten oder ungültige Formate.

Person

Die Klasse `Person` bildet Einzelpersonen ab, wie sie in den Briefen, Postkarten oder Protokollen erscheinen. Neben klassischen Namensfeldern (`forename`, `familyname`, `title`) unterstützt die Klasse auch alternative Namen (`alternate_name`) und Rolleninhalte. Die Rollenverarbeitung erfolgt in einem zweistufigen Verfahren: Zum einen wird die Freitextrolle (`role`) als Originaleintrag gespeichert, zum anderen erfolgt eine normierte Zuordnung über `role_schema`, basierend auf dem internen Mapping in `Assigned_Roles_Module.py`.

Titel wie "Herr" oder "Frau" dienen in `person_matcher.py` zugleich als Grundlage zur Ableitung des Geschlechts der Person. Das Feld `gender` erlaubt in `document_schemas.py` eine geschlechtsspezifische Zuordnung, basierend auf Titelbezeichnungen oder Ground Truth-Matching.

Ergänzt wird die Person um Kontexte wie `associated_place` und `associated_organisation`, sowie um Bewertungsparameter wie `match_score`, `recipient_score`, `confidence` und `mentioned_count`.⁹⁸ Letzterer gibt an, wie oft

⁹⁸Detailliert wird auf diese Logiken in [person_matcher.py](#) eingegangen, sie sollen hier nur umrissen

die Person im Transkript erwähnt wurde, unter Berücksichtigung einer kontextbasierten Zähllogik. `match_score` beschreibt einen Wert der Wahrscheinlichkeit, dass es sich bei der im Dokument beschriebenen Person auch um die Person handelt, die schlussendlich gematcht wurde. Vor- und Nachnamen erhöhen beispielsweise den Score, während das Fehlen solcher einzelner Informationen den Wert senken.

Das Feld `confidence` beschreibt die Modell-Sicherheit der Zuordnung, z.B. beim LLM-basierten Matching. `recipient_score` bewertet die Wahrscheinlichkeit, dass es sich bei der Person um den tatsächlichen Empfänger des Dokuments handelt.

Eine optionales `needs_review`-Flag sowie ein `review_reason` ermöglichen die gezielte Markierung unsicherer oder maschinell nur teilweise auflösbarer Fälle.

Organization

Die Klasse `Organization` dient der Abbildung aller im Text genannten Vereine, Gruppen oder Institutionen, darunter z.B. Gesangsvereine, und NS-Organisationen. Neben dem Namen, Typ und eventuellen Alternativnamen (`alternate_names`) wird insbesondere die eindeutige `Nodegoat_ID` gespeichert. Für den Spezialfall militärischer Einheiten enthält die Klasse ein separates Feld `feldpostnummer`. Da sie in der Pipeline jedoch nicht separat getaggt und verarbeitet sind, werden diese aktuell ausschliesslich über die Ground Truth abgerufen. Über `match_score` und `confidence` werden auch in der Klasse `Organization` Qualität und Unsicherheiten der Zuordnung nachvollziehbar gemacht. Organisationen können zusätzlich über das Feld `associated_place` mit einem geographischen Ort verknüpft werden, z.B. „Männerchor Murg“ mit „Murg“.

Place

Die Klasse `Place` strukturiert geographische Orte, wie sie z.B. als Absender-, Empfänger- oder Veranstaltungsorte im Dokument auftreten. Unterstützt werden neben der Hauptbezeichnung (`name`) auch alternative Formen (`alternate_place_name`), sowie standardisierte Identifikatoren aus Geonames, Wikidata und Nodegoat. Diese Orte sind über eigene Felder sowohl im Metadatenblock (`creation_place` , `recipient_place`) als auch in der Liste `mentioned_places` verortet, falls erstere nicht genau zugeordnet werden können. Wie bei Personen, kann auch bei Orten über das Flag `needs_review` eine manuelle Überprüfung unsicherer Matches angestossen werden.

Event

Ereignisse werden über die Klasse `Event` abgebildet. Diese enthält einen Namen, eine optionale Beschreibung, Datumsangaben und Referenzen auf beteiligte Personen, Orte und Organisatio-

werden.

nen. Über das Feld `inferred` wird angegeben, ob das Ereignis direkt im Text genannt oder aus dem Kontext erschlossen wurde. Die genaue Event-Extraktion findet sich in `event_matcher.py`.

Documenttype

Für die wichtigsten Dokumenttypen existieren spezialisierte Unterklassen wie `Brief`, `Postkarte` oder `Protokoll`, die zusätzliche Felder (z.B. `greeting`, `postmark`, `meeting_type`) aufnehmen. Diese können zentral über `create_document()` automatisch erzeugt werden, wenn ein Dokumenttyp erkannt wurde. Diese spezialisierten Erkennungstypen stehen aktuell lediglich für eine spätere Verwendung bereit, und kommen in der aktuellen Anwendung nur in der Benennung des jeweiligen Typs zum Einsatz.

Alle Ergebnisse werden anschliessend in strukturierter Form in eine JSON-Datei geschrieben, wobei jeder Eintrag dem oben definierten Schema entspricht.

5.3.2 `person_matcher.py`

Die Erkennung und das Matching von Personen stellt bei Weitem die grösste Herausforderung in diesem Projekt dar. Wie vorausgehend bereits angedeutet, gibt es eine Vielzahl von Möglichkeiten, eine Person in einem Text zu benennen oder nur anzudeuten, was ein zuverlässiges Matching erschwert. Oft ist der Kontext relevant, um Personen eindeutig zu identifizieren. Hinzu treten spezifische Herausforderungen der Textverarbeitung. Ein auf Pattern Matching basierender Stringabgleich ist beispielsweise kasus- und genussensitiv, sodass die Algorithmen ein Wort im Nominativ Maskulinum ebenso zuverlässig erkennen müssen wie im Akkusativ Femininum. Hierfür ist eine systematische Normalisierung der Strings erforderlich. Ebenso ist entscheidend, welche Zeichenketten überhaupt in die Personenprüfung einbezogen werden und welche Kriterien bestimmen, ob eine Zeichenkette als Personenreferenz gilt. Initialen, Signaturen, Nennung einzelner Vornamen oder Nachnamen und die Verwendung von Spitznamen oder Rollen sind lediglich einige Möglichkeiten, eine Person zu benennen. Daraus ergibt sich für den `Person_Matcher` allein auf heuristischer Ebene eine grosse Komplexität, die im Folgenden nur andeutungsweise dargelegt werden kann.

Für deren Prüfung arbeitet der `Person_matcher` mit Ground Truth aus Nodegoat, und den im Anschluss erläuterten Modulen `Assigned_Roles_Module` und dem `Organization_Matcher` zusammen. Für einen Überblick soll die Abbildung 10 dienen, die den technischen Aufbau des Moduls schematisch darstellt.

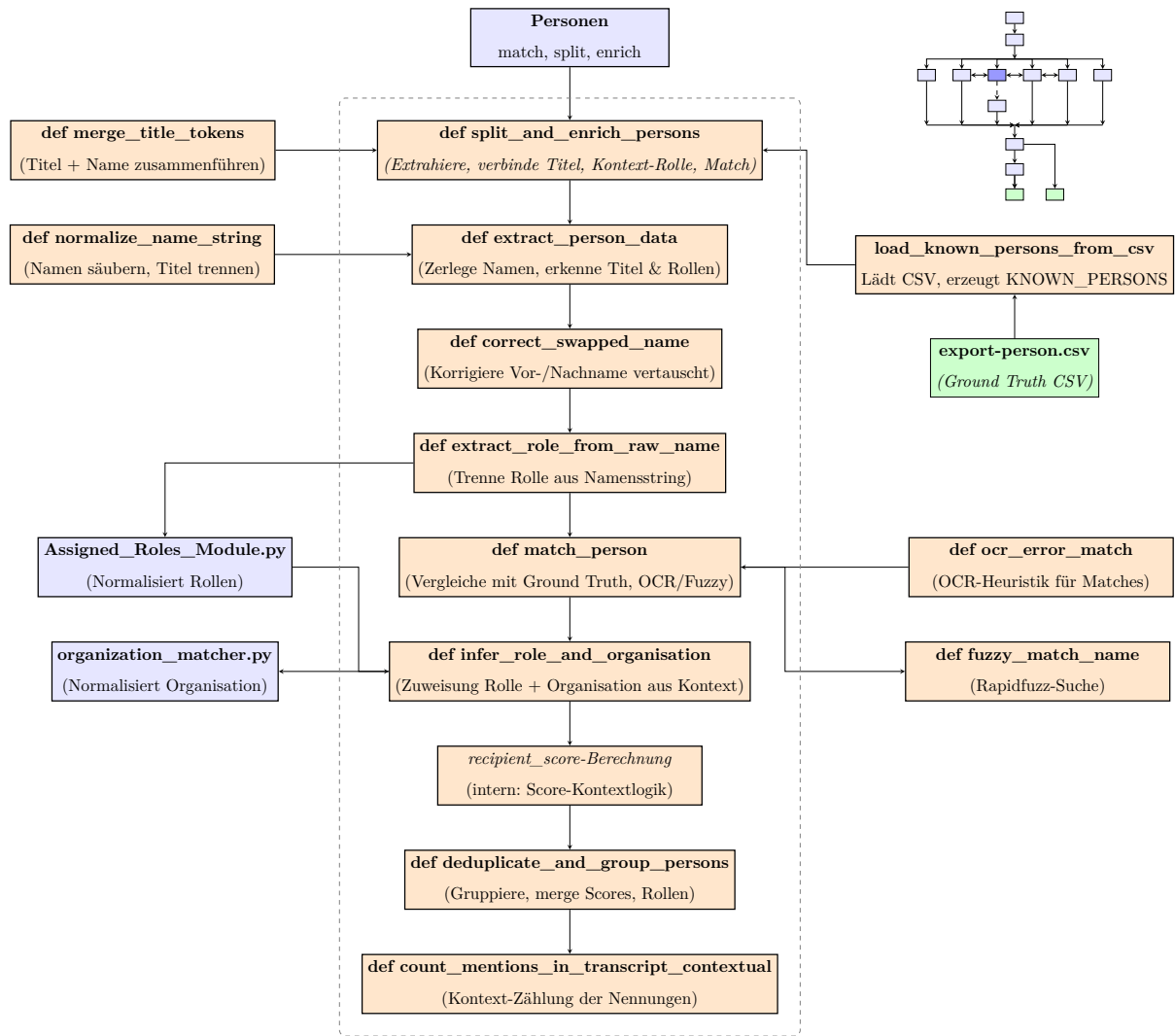


Abbildung 10:
 Oben links: Prozessdiagramm für `Personen_matcher.py` ,
 Oben rechts: Pipelineübersicht

Weil der so repräsentierte Code sehr verschachtelt ist, soll im nachfolgenden Kapitel der Fokus auf der inhaltlichen Verarbeitung liegen. So lässt sich das Modul in diese Sektionen unterteilen:

1. Blacklist & Konfiguration
2. Initialisierung
3. Namen- und Titelerkennung
4. Levenshtein-Fallback
5. Fuzzy-Matching
6. Entitätenzuordnung
7. Extraktion von Personendaten
8. Segmentierung & Anreicherung
9. Deduplikation
10. Detail-Info zum besten Match

Die Darstellung orientiert sich bewusst an der Codierungshierarchie, um die interne Logik und Abfolge der Verarbeitungsprozesse nachvollziehbar abzubilden. Zur visuellen Strukturierung sind zentrale Module kursiv und abhängige Unterfunktionen unterstrichen hervorgehoben.

Im Laufe dieser Arbeit sind vier Parameter entwickelt worden, die für die Zuordnung von Strings

zu Personen mit einem anschliessenden Match zu Personen aus der Ground Truth ermöglichen sollen. Sie lauten **Vorname**, **Nachname**, **Rolle** und **Geschlecht**. Ihre Verarbeitung wird nun beschrieben.

Blacklist & Konfiguration

In der Initialisierung des Moduls `person_matcher.py` werden neben der Ground Truth-Tabelle `export-person.csv` eine Vielzahl domänenspezifischer Konfigurationslisten geladen, um die Erkennung und Validierung von Personennamen zu unterstützen. `NICKNAME_MAP` liefert beispielsweise ein Dictionary mit üblichen deutschen Spitznamen.⁹⁹ Diverse Blacklists statuieren Worte, die etwa nicht menschlich sind (bsp. "freundliche Grüsse"). Zu den Blacklists zählt auch `Unmatchable_Single_Names`, die auf nicht eindeutig verknüpfbare Namen hinweist.¹⁰⁰ Die Liste `Pronoun` und nach Geschlecht zusammengefasste `Title_Tokens`¹⁰¹ markieren Tokens zur gezielten Weiterverarbeitung in nachgelagerten Matching- und Anreicherungsschritten. Alle Ausschlusslisten werden in `NON_PERSON_TOKENS` zusammengeführt und dienen der Negativselektion im gesamten Matchingprozess. So ausgezeichnete Strings fallen durch die Prüfungsschritte.

Thresholds, CSV-Laden und Ground Truth-Erstellung

Für das Matching werden zwei essentielle Thresholds definiert, die erreicht werden müssen. `forename` muss eine 80% Ähnlichkeit, `familyname` eine 85% Ähnlichkeit erreichen. Diese Zahlen ergeben sich durch unterschiedliche Tests. Sie bieten einen Kompromiss aus Toleranz gegenüber beispielsweise OCR-Fehlern, und liefern dennoch präzise Trefferquoten.¹⁰² Alle Personendaten werden als Liste von Dictionaries¹⁰³ aus der Ground Truth geladen. Sie werden in die Variablen `KNOWN_PERSONS`, `Ground Truth_SURNAMES` und `Ground Truth_FORENAMES` übergeben, um später gegen die genannten Thresholds zu testen.

Namensnormalisierung und Titelerkennung

Für eine konsistente Weiterverarbeitung werden alle erkannten Namensstrings standardisiert, und auf mögliche Rollenstrings überprüft. Notwendig wird das, weil Rollen durch das LLM oft, und richtigerweise, als Person markiert werden. Daher durchlaufen die Namensstrings die zwei Normalisierungen `normalize_name_string` und `normalize_name`. Erstere bereinigt den Text grob und trennt Titel vom Rest. Da in den Quellen oft Initialen vorkommen¹⁰⁴, wird zwischen

⁹⁹Durch ChatGPT generiertes Mapping geläufiger deutscher Vor- und Kosenamen auf ihre kanonische Form, einige Fälle sind händisch ergänzt

¹⁰⁰"Otto" oder "Döbele" sind so häufig vorkommende Namen, dass sie alleine nicht für ein [Entitätenzuordnung](#) ausreichen: Otto häuft sich mindestens 11 Mal.

¹⁰¹ `TITLE_TOKENS = MALE_TITLE_TOKENS | FEMALE_TITLE_TOKENS | NEUTRAL_TITLE_TOKENS`

¹⁰²Da oft nur `familynames` in den Dokumenten vorkommen, und sie damit für das Matching wichtiger sind, haben diese einen höheren Threshold

¹⁰³ `List[Dict[str, str]]`

¹⁰⁴Bsp.: "A. Zimmermann"

den beiden Funktionen ebenfalls auf eine Initiale geprüft. Die Funktion `normalize_name()` extrahiert dann eventuell vorhandene Titel mithilfe eines dynamisch aus `TITLE_TOKENS` generierten regulären Ausdrucks. Anschliessend werden Klammern entfernt, Unicode-Zeichen vereinheitlicht¹⁰⁵, Diakritika¹⁰⁶ gelöscht und Sonderzeichen bereinigt. Die verbleibenden Namensbestandteile werden in Vor- und Nachname aufgeteilt. Dabei berücksichtigt die Funktion auch Nickname-Mappings, indem erkannte Kurzformen wie `Willi` automatisch in ihre kanonische Form `Willhelm` überführt werden. Enthält der Name nur ein Token, wird dieses als Vorname gewertet; andernfalls erfolgt eine Trennung in `forename` und `familyname`. Das Ergebnis ist ein Dictionary mit den Feldern `title`, `forename` und `familyname`, die in späteren Matching- und Review-Schritten verwendet werden. So wird mit `normalize_name_string` aus "Dr. Willi Müller" → `forename:Willhelm, lastname:müller, title:"Dr"`

Levenshtein-Fallback

Nach der Namensnormalisierung folgen zwei zentrale Funktionen zur Fehlerkorrektur und Heuristik-basierter Umkehrung von Namensbestandteilen.

Die Funktion `ocr_error_match()` verwendet die Levenshtein-Distanz zur Berechnung der Ähnlichkeit zwischen einem gegebenen Namen und einer Liste möglicher Kandidaten, um einzelne Tokens gegen bekannte Ground Truth-Namen zu prüfen. Dabei wird der Eingabename `name` zunächst in Kleinschreibung normalisiert (`name_lower`), ebenso wie jeder Kandidat aus der Vergleichsliste (`cand_lower`). Obwohl beide Seiten formal gleich behandelt werden, liegt der Unterschied in ihrer Herkunft: `name_lower` entstammt der unkorrigierten Texterkennung und kann daher auch Schreibfehler beinhalten, während `cand_lower` aus kuratierten Ground Truth-Listen stammt und damit als Referenz dient. Die Distanz zwischen beiden wird berechnet und der beste Treffer mit der geringsten Distanz wird als Ergebnis zurückgegeben. Ergänzt wird dieser Treffer um einen prozentualen Match-Score.

Darauf aufbauend prüft `correct_swapped_name()`, ob Vor- und Nachname im gegebenen String möglicherweise vertauscht wurden. Dazu wird jeweils ermittelt, ob der Vorname einem bekannten Nachnamen entspricht oder umgekehrt. Neben direktem Lookup in `KNOWN_FORENAMES` und `KNOWN_SURNAMES` kommt hier ebenfalls ein Levenshtein-Fallback zum Einsatz, um kleine Abweichungen zu tolerieren. Wird auf diese Weise erkannt, dass eine Umkehrung wahrscheinlicher ist, wird das Tupel entsprechend getauscht zurückgegeben.

Fuzzy-Matching

Um Namensähnlichkeiten robust zu erkennen, implementiert `fuzzy_match_name()` ein Fuzzy-Matching-Verfahren auf Basis des `fuzz.ratio()`-Scores. Für jedes Vergleichspaar

¹⁰⁵auch NFKD-Normalisierung genannt, **Abk.:** Normalization Form Compatibility Decomposition

¹⁰⁶Sonderbuchstaben, Bsp: ä, ê, š, ç

(`name / candidate`) wird anschliessend ein Ähnlichkeitswert berechnet, und der beste Treffer bestimmt. Liegt der höchste Score über dem anfangs definierten Threshold, wird der entsprechende Kandidat samt Score zurückgegeben; andernfalls wird `None` geliefert. Dieses Verfahren ergänzt die Levenshtein-basierte Fehlerbehandlung um eine robustere, Token-basierte Ähnlichkeitsmessung, die auch bei Namensvarianten und OCR-Fehlern zuverlässige Resultate liefert.

Entitätenzuordnung

Der `Person_Matcher` verwendet nun die vorverarbeiteten Namensstrings als `person`, und die wahrscheinlichen Personen der Ground Truth als `candidate` um diese zusammenzuführen. Gleich zu Beginn wird der Sonderfall "nur Rolle, keine Namen" geprüft. Wenn Der Personen-Tag nur eine Rolle, aber keinen Namen enthält, wird ein Minimal-Objekt mit niedrigem Score zurückgegeben. Zudem erhält das Objekt dann den `Review-Reason`:"Nur Rolle ohne vollständigen Namen erkannt".

Inhalts-Filter

Es folgt zur Absicherung eine Normalisierung des Strings aus dem Text auf Roh-(Vor-)Namen, und ein Check, ob die Strings in den Blacklists (`Pronoun_Tokens`, `Role_Tokens`, `Non_Person_Tokens`) vorkommen, um sie frühzeitig auszuschliessen. Diese Fälle liefern für `Match:None, 0`

Unique-Name

Wenn nur der Vor- oder Nachname vorhanden ist, wird geprüft, ob der Name eindeutig in der Ground Truth vorkommt. Falls ja, wird hierfür sofort ein Match ausgegeben.

Blacklist-Checks

Hier werden problematische Tokens vor dem eigentlichen Matching identifiziert und blockiert. Getestet werden `Non_Person_Tokens` und `Role_Tokens` in zwei if-Blöcken und deren Vorkommen in Vor- oder Nachnamen. Im zweiten Teil wird erkannt, ob ein Rollenbegriff fälschlich als Vorname identifiziert wurde.

Initialen-Check

Falls der Vorname `fn` lediglich eine Initiale darstellt, wird mit `is_initial(fn)` versucht, einen passenden Eintrag in `candidates` zu finden. Dazu wird die Initiale mit dem ersten Buchstaben des Vornamens jedes Kandidaten verglichen und zusätzlich geprüft, ob der normalisierte Nachname (`ln`) mit demjenigen des Kandidaten übereinstimmt. Wird ein solcher Eintrag gefunden, wird er mit hoher Übereinstimmung (`match_score = 95`) zurückgegeben.

Fuzzy Matching über Vornamen und Nachnamen

Liegt ein Vorname `fn` mit mindestens drei Zeichen¹⁰⁷ sowie ein Nachname `ln` vor, wird zunächst versucht, anhand eines Teilabgleichs zu matchen. Der Anfang des gegebenen Vornamens (`fn`) muss mit dem Kandidatennamen übereinstimmen, während der Nachname exakt übereinstimmt. In diesem Fall wird der Treffer mit einem Score von `89` zurückgegeben.

Zusätzlich erfolgt eine *Reverse-Prüfung*, um mögliche Vertauschungen von Vor- und Nachname zu erkennen. Dabei wird geprüft, ob die normalisierten ersten vier Zeichen des eingegebenen Nachnamens mit dem Vornamen eines Kandidaten übereinstimmen und gleichzeitig die normalisierten ersten vier Zeichen des eingegebenen Vornamens mit dessen Nachnamen. Voraussetzung ist ausserdem, dass nicht bereits eine Übereinstimmung der geprüften Variablen `fn` und `ln` mit den Feldern `forename` und `familyname` des Kandidaten vorliegt. Wird diese Bedingung erfüllt, werden die Namensfelder im Rückgabeobjekt vertauscht und der Match mit einem reduzierten Score von `88` zurückgegeben.

Levenshtein-Fallback bei Namensvertauschung

Zunächst wird geprüft, ob der normalisierte Vorname `fn` exakt mit dem eines Kandidaten übereinstimmt und der Nachname eine Levenshtein-Distanz von höchstens 1 zum Nachnamen des Kandidaten aufweist. In diesem Fall wird ein Match mit Score `90` zurückgegeben.

Anschliessend erfolgt eine Reverse-Prüfung. Dabei wird getestet, ob der Nachname `ln` mit dem Vornamen eines Kandidaten übereinstimmt und der Vorname `fn` dem Nachnamen des Kandidaten bis auf eine sehr geringe Levenshtein-Distanz von 1 entspricht. Das soll absichern, dass vertauschte Namensbestandteile auch mit geringen OCR-Fehlern korrekt gematched und nicht weiter behandelt werden.

Matching über Einzelkomponenten

Dieser Abschnitt deckt Fälle ab, in denen nur unvollständige Namensinformationen vorliegen. Ist lediglich ein Nachname vorhanden, wird ein fuzzy-Matching gegen alle Kandidaten-Nachnamen mittels [Fuzzy-Matching](#) durchgeführt; erreicht ein Kandidat den konfigurierten Threshold, wird dieser mit dem berechneten Score übernommen. Liegen weder Vor- noch Nachname vor, aber eine Rollenbezeichnung (`role_raw`), erfolgt ein Abgleich mit `role` und `alternate_name` der Kandidaten; bei Übereinstimmung wird ein Treffer mit Score `80` zurückgegeben.

Zusätzlich werden Namensbestandteile, die als generisch oder nicht personenspezifisch klassifiziert sind (`NON_PERSON_TOKENS`), speziell behandelt. Ist der Vorname ein solcher unerwünschter Token, aber ein Nachname vorhanden, wird dieser gegen alle `familyname`-Felder geprüft. Umgekehrt wird bei generischem Nachnamen der Vorname geprüft. Als letzte Rückfallebene kommt

¹⁰⁷Vornamen mit 2 Buchstaben existieren in dem Groudtruth-Satenset nicht.

`ocr_error_match()`¹⁰⁸ zum Einsatz. Ist nur ein Nachname vorhanden, wird dieser mit einer Toleranz von bis zu zwei Zeichen mit den Nachnamen aller Kandidaten verglichen; bei Übereinstimmung wird der entsprechende Treffer mit Score `85` übernommen. Alle Varianten markieren strukturunscharfe, aber potenziell gültige Entitäten.

Matching über Nachname und Gender

Dieser Abschnitt implementiert eine gender-basierte Disambiguierung. Sie greift dann, wenn der Nachname mit mehreren Personen übereinstimmt, aber das Geschlecht bekannt oder ableitbar ist. Zunächst wird versucht, dieses anhand von Titeln wie `"Frau"` oder `"Herr"` zu bestimmen. Ist anschliessend sowohl ein Nachname als auch ein valides Geschlecht verfügbar, werden alle Kandidaten mit identischem Nachnamen gesammelt und auf Übereinstimmung des Geschlechts gefiltert. Existiert genau ein solcher Treffer, wird dieser mit hoher Konfidenz (`confidence = "gender_ln_match"`) und einem Score von `95` zurückgegeben, ohne dass eine manuelle Nachprüfung erforderlich ist. Das Matching über Geschlecht und Nachname ist besonders wichtig, da Frauen in den Unterlagen bis auf sehr wenige Ausnahmen nie mit Vornamen genannt werden. Ohne den Geschlechtsabgleich würden sie mit (Ehe-)Männern gematched werden. Diese Funktion versucht also aktiv, gegen den Geschlechterbias in den historischen Unterlagen vorzugehen.¹⁰⁹

Last Fallback - Aufnahme mit Review

Falls mindestens eines der Felder `fn` (Vorname), `ln` (Nachname) oder `role_raw` (Rolle) vorhanden ist, aber kein reguläres Matching möglich war, wird ein Fallback-Objekt erzeugt und zur manuellen Nachprüfung markiert (`needs_review = True`). Die Funktion prüft, welche Felder fehlen, und protokolliert dies als `review_reason` (z.B. `missing_forename`; `missing_role`). Eine Aufnahme erfolgt nur, wenn entweder der Vor- oder Nachname in einer Ground Truth-Liste vorkommt (`appears_in_Ground_Truth()`) oder eine Rollenbezeichnung vorhanden ist. In diesem Fall wird ein Personeneintrag mit `confidence = "partial-no-id"` und `match_score = None` zurückgegeben. Andernfalls wird das lückenhafte Personobjekt vollständig verworfen.

Extraktion von Personendaten

Die Funktion `extract_person_data(row)` dient der strukturierten Aufbereitung einzelner Personeneinträge. Dafür holt sich die Funktion Informationen über die jeweilige Datenzeile aus der `export-person.csv`. Ziel ist es, aus den Namensangaben und Metadaten ein standardisiertes Personenobjekt zu erzeugen, wie in `document_schemas.py` beschrieben.

¹⁰⁸aus [Levenshtein-Fallback](#)

¹⁰⁹Geschlechterbias tritt bei dieser Arbeit in diversen Formen auf, *Source Bias*, *Archival Bias*, *Model Bias*, und *Algorithmic Bias*. Letzterer soll mit dieser Funktion abgeschwächt werden. Vgl.: Burkhardt 2025a.

Der übergebene Namensstring wird zu Beginn über eine dynamische RegEx mit `Title_tokens` nach Titeln durchsucht. Wird ein solcher Titel erkannt, wird er im Feld `title` gespeichert und aus dem Namensstring entfernt. Der verbleibende Name wird auf mögliche Rolleninhalte analysiert, die über `extract_role_from_raw_name()` aus dem `Assigned_Roles_Module.py` ermittelt werden. Hierbei wird exemplarisch der String `Schriftführer Ganter` in die Bestandteile `role=Schriftführer` und `name=Ganter` aufgetrennt.

Anschliessend wird der bereinigte Namensstring in Einzelteile zerlegt. Das erste Token wird als Vorname, das letzte als Nachname interpretiert. Um typische OCR-Fehler oder Namensinversionen zu korrigieren, wird nochmal die Funktion `correct_swapped_name()` aufgerufen, welche Vor- und Nachnamen bei Bedarf vertauscht.

Für den Rollenteil des Eintrags wird zunächst das Feld `role` aus der ursprünglichen CSV-Zeile übernommen. Falls bereits beim Parsing eine Rolle erkannt wurde, wird diese priorisiert. Der erkannte Rollentext wird dann durch `normalize_and_match_role()` in eine standardisierte Form überführt. Über `map_role_to_schema_entry()` erfolgt anschliessend die semantische Abbildung auf ein normiertes Rollenschema¹¹⁰.

Auch das Geschlecht der Person wird in diesem Schritt bestimmt. Dazu wird zunächst das übergebene Geschlechtsfeld aus der Ground Truth gemapt, bereinigt und vereinheitlicht. Sollte dieses keine gültige Angabe enthalten, wird als Fallback `infer_gender_for_person()` aufgerufen. Diese Funktion versucht, das Geschlecht über den Titel oder durch Abgleich mit der Ground Truth-Liste zu ermitteln.

Neben Namen, Titel, Rolle und Geschlecht werden auch zusätzliche Kontextinformationen aus der Ground Truth extrahiert, sofern vorhanden. Dazu zählen `alternate_name`, der Wohnort und weitere. So kann für jedes Dokument garantiert werden, dass alle verfügbaren Daten für eine gegebenenfalls manuelle Prüfung vorhanden sind. Plausibilitätsprüfungen funktionieren so unmittelbar, ohne Zwischenschritte. Wird eine Person mit einem für sie untypischen Ort gematched, etwa weit vom Heimatort entfernt, kann so später gezielt nach dieser Reise geforscht werden.

Die Funktion erzeugt abschliessend ein vollständiges Personenobjekt mit Attributen wie `confidence`, `match_score` und `needs_review`, die für spätere Matching- und Validierungsschritte relevant sind. Personen ohne `Nodegoat-ID` werden standardmässig zur manuellen Nachprüfung markiert.

Für Sonderfälle, bei denen keine Namen, sondern lediglich Rollentitel wie `„Vereinsführer“` erkannt wurden, dient die Funktion `detect_and_convert_role_only_entries()`. Sie prüft, ob das einzige übergebene Token einem bekannten Rollenschema entspricht. Ist dies der Fall,

¹¹⁰ `role` ist der detektierte String, `role_schema` die normalisierte Form

wird eine strukturierte Rollenangabe ohne Namensinformation erzeugt und ebenfalls zur Überprüfung markiert.

Schliesslich wird in Zusammenarbeit mit dem `Assigned_Roles_Module` mit `extract_metadata_names()` eine Funktion bereitgestellt, um auch aus dem Fliesstext zusätzliche Personenhinweise zu extrahieren. Hierbei kommen wieder RegEx zum Einsatz, die typische Anredeformen oder Signaturen am Briefende erkennen und zur weiteren Analyse aufbereiten. Mehr dazu im nachfolgenden Abschnitt.

Zur systematischen Dokumentation unvollständiger Einträge wird zusätzlich die Funktion `get_review_reason_for_person()` bereitgestellt. Sie prüft, ob für die betreffende Person kritische Felder wie Vorname, Nachname oder Rolle fehlen und erzeugt eine entsprechende Review-Kennzeichnung. Diese Information wird im Feld `review_reason` gespeichert.

`merge_title_tokens()` sorgt dafür, dass Titel wie „Frau“ oder „Herr“ korrekt mit dem darauffolgenden Namen verbunden und das Geschlecht basierend auf dem Titel automatisch ergänzt wird. Dies ermöglicht insbesondere in der Vorverarbeitungsphase eine präzisere Initialisierung der Felder `title` und `gender`, bevor die Einträge an die nachgelagerte Matching-Logik übergeben werden. Verwendung finden beide Funktionen zur Nachprüfung im nun folgenden `split_and_enrich` Block.

Segmentierung und Anreicherung

Auf die Extraktion einzelner Personenattribute folgt die zentrale Funktion `split_and_enrich_persons()`, die den Übergang von unstrukturierten oder teilstrukturierten Personentokens zu formalisierten Personenobjekten koordiniert. Sie übernimmt Listen roher Einträge – typischerweise extrahiert aus Fliesstexten oder XML-Tags – und durchläuft mehrere Verarbeitungsschritte: Strukturprüfung, Normalisierung, Kontextergänzung und semantisches Matching.

`split_and_enrich_persons()` übernimmt die übergeordnete Steuerung von `match_person()`, die für den eigentlichen Abgleich einzelner Personen mit der Ground Truth-Datenbank verantwortlich ist. Erstere bereitet die Einträge so auf, dass diese überhaupt matchfähig werden. Das geschieht etwa durch Zusammenführung getrennter Namensbestandteile, Titelzuordnung, Rollenerkennung aus dem Dokumentkontext oder heuristische Strukturvervollständigung.

Sobald ein Eintrag formalisiert ist, ruft die Funktion intern `match_person()` auf und erhält daraus ein Matching-Ergebnis mit `match_score`, `confidence`-Wert und ggf. einer `Nodegoat_ID`. Dieses Ergebnis wird entweder direkt übernommen oder, bei unklarer Übereinstimmung, durch zusätzliche Heuristiken wie Namensinversion oder Rollenprüfung weiterverarbeitet. Auf diese Weise verbindet `split_and_enrich_persons()` die initiale Rohdatensegmentierung mit der

konkreten Identifikation bekannter Personen aus dem Ground Truth.

1. Vorverarbeitung

Zunächst wird jeder Eintrag auf seine Struktur geprüft. Bereits vollständige Objekte mit `forename`, `familyname` und `Nodegoat_ID` werden übernommen und minimal bereinigt. Teilstrukturierte Einträge hingegen werden auf mehrzeilige Namen überprüft, indem benachbarte Tokens wie `Herr` und `Alfons Zimmermann` zusammengeführt werden. Zusätzlich wird die Funktion `merge_title_tokens()` aufgerufen, um Titel mit nachfolgenden Namen zu verbinden. In dieser Kombination kann das Geschlecht identifiziert und zu der Person ergänzt werden.

2. Tokenisierung und heuristische Vervollständigung

Für alle anderen Einträge erfolgt wieder eine Zerlegung: Der Inhalt des `name`-Feldes wird in einzelne Tokens aufgespalten, und das erste Token wird auf das Vorkommen typischer Titel geprüft. Wird ein solcher erkannt, wird er als `title` gespeichert und das zugehörige Feld `gender` entsprechend gesetzt.

Besonders betont werden muss hier ein Sonderfall. Manchmal folgt nach dem weiblichen Titel ein männlicher Vorname, wie etwa in „Frau Alfons Zimmermann“. In solchen Fällen wird der männliche Vorname als nicht zur Person gehörig gewertet und entfernt.¹¹¹ Anschliessend wird das verbleibende Token als Nachname interpretiert und im Feld `familyname` gespeichert; das Feld `forename` bleibt in der Regel leer, um fehlerhafte Zuordnungen zu vermeiden.

Werden hingegen keine Titel erkannt, der Namensstring ist aber auswertbar, wird der gesamte Inhalt als Nachname behandelt. Darüber hinaus werden Tokens, die vollständig in einer Blacklist vorkommen durch Vergleich mit `NON_PERSON_TOKENS` identifiziert und verworfen.

Alle so erzeugten oder überarbeiteten Einträge werden in einer neuen Liste `processed_raw_persons` gesammelt und am Ende des Abschnitts in die bereits erwähnte Variable `raw_persons` überführt, die dann als Grundlage für die nächste Verarbeitungsstufe dient.

3. Kontextbasierte Rollenerkennung

Mithilfe des `transcript` werden erkannte Namen im Text lokalisiert. Dazu wird der vollständige Text zeilenweise durchsucht und geprüft, ob der kombinierte Namensstring einer Person in einer der Zeilen enthalten ist. Bei einem Treffer wird die unmittelbare Textumgebung mit der Funktion `infer_role_and_organisation()` analysiert.

Diese Funktion sucht mit RegEx nach typischen Mustern für Rollentitel und überprüft, ob er zusammen mit einer zugehörigen Organisationseinheit genannt wird. Erkennt das System eine solche Struktur, etwa `Leiter des V.D.A.`, so werden die Felder `role` und

¹¹¹Dies ist notwendig, weil Frauen in den Quellen häufig relational über ihre Ehemänner adressiert werden: `Frau Alfons Zimmermann` bezeichnet tatsächlich die `Frau von Alfons Zimmermann`. Ein Beispiel für Genderbias-Verzerrung in historischen Dokumenten, vgl. (Burkhardt 2025a).

`associated_organisation` im Personenobjekt automatisch ergänzt.

Neben expliziten Mustern wie `<Rolle>` in der `<Organisation>` erkennt die Funktion auch unstrukturierte Rollennennungen anhand von Schlagwortlisten (`ROLE_PATTERNS`) oder typischer Rollendenungen wie `-führer`, `-leiter`, `-wart`. In Fällen, in denen keine Organisation erkannt wird, bleibt das zugehörige Feld leer, während die Rolle trotzdem gespeichert wird. Durch diese Kontextanalyse können auch Rollenbezüge erfasst werden, die im eigentlichen Namenseintrag nicht enthalten waren.

4. Matching gegen Ground Truth

Im folgenden Schritt wird versucht, jede zuvor bereinigte und kontextualisierte Person mit einem Eintrag aus dem Ground Truth abzugleichen. Dafür wird ein `raw_token` gebildet, bestehend entweder aus dem ursprünglich erkannten String oder der Kombination aus `forename` und `familyname`. Sofern bereits alle Kernfelder (`forename`, `familyname`, `Nodegoat_ID`) vorhanden sind, wird der Eintrag ohne erneutes Matching übernommen und in die Liste `matched` eingefügt. Die Daten können bereits aus vorangegangener manueller Annotation in Transkribus oder exakter Extraktion stammen.

Für alle übrigen Personen wird die Funktion `extract_person_data()`¹¹² aufgerufen, um aus dem Namen sowie gegebenenfalls bekannten Titeln oder Geschlechtsangaben ein standardisiertes Personenobjekt zu generieren. Dieses Objekt wird an `match_person()` übergeben, wo der eigentliche Abgleich gegen den Ground Truth erfolgt. Wird ein Treffer mit ausreichender Ähnlichkeit gefunden, wird die Person samt `match_score`, `confidence`-Label und Nodegoat-ID als gematcht markiert.

Um Redundanzen zu vermeiden, wird zusätzlich ein eindeutiger Schlüssel aus Vor- und Nachnamen oder der ID generiert.¹¹³ Kommt derselbe Schlüssel mehrfach vor, wird der erste gültige Treffer priorisiert. Kann keine eindeutige Übereinstimmung festgestellt werden, wird der Eintrag zur manuellen Nachprüfung in die Liste `unmatched` aufgenommen und mit dem `review_reason` versehen.

Datenkonsolidierung

Die Funktion `deduplicate_persons()` konsolidiert alle im Dokument identifizierten Personeneinträge. Ziel ist es, Dopplungen zu erkennen, redundante Einträge zu verwerfen und für jede erkannte Person einen einzigen, mit allen zur Verfügung stehenden Informationen angereicherten Datensatz zu erzeugen. Die Gruppierung basiert dabei entweder auf der eindeutigen `Nodegoat_ID`, auf einem vollständig erkannten Namen oder, als letztes Mittel, auf einem generierten Dummy-Schlüssel bei Rolleneinträgen ohne Namensangabe.

¹¹²siehe Abschnitt 10

¹¹³je nach vorhandenen Daten

1) Gruppierung:

Im ersten Schritt wird jede Person einem Gruppenschlüssel (`key`) nach in 10 beschriebenen Mustern zugeordnet. Diese Gruppierung wird in einem Dictionary `person_groups` gesammelt.

2) Auswahl der besten Repräsentation pro Gruppe

Für jede Gruppe wird ein repräsentativer Haupteintrag gewählt. Die Auswahl erfolgt anhand folgender Hierarchie. Einträge mit gültiger `Nodegoat_ID` und hohem `match_score` werden bevorzugt. Andernfalls wird der am besten bewertete Eintrag über Fuzzy-Matching bestimmt. Falls kein eindeutiger Treffer ermittelt werden kann, wird der vollständigste Originaleintrag übernommen und als `needs_review = True` markiert.

3) Anreicherung der Felder:

Innerhalb jeder Gruppe wird der gewählte Haupteintrag mit zusätzlichen Informationen aus den übrigen Gruppeneinträgen angereichert. `mentioned_count` wird als Summe aller Vorkommen im Text berechnet. Der höchste Wert für `recipient_score` wird übernommen, um die Wahrscheinlichkeit einer Empfängerschaft zu erfassen.¹¹⁴ Alle erkannten `role`-Felder werden vereinheitlicht und als Liste zusammengeführt. Leere oder unvollständige Felder wie `title`, `alternate_name`, `associated_place` oder `associated_organisation` werden, falls möglich, aus anderen Gruppenmitgliedern ergänzt. Organisationseinträge werden bei Bedarf durch die Hilfsfunktion `flatten_organisation()` aus verschachtelten Strukturen in eine flache Repräsentation überführt. Zusätzlich wird für jede `Nodegoat_ID` geprüft, ob aus der Ursprungsliste ein höherer `recipient_score` vorliegt. Dieser wird bei Bedarf rückwirkend übernommen.

4) Fallback bei unklaren Matches:

Falls keine `Nodegoat_ID` vorhanden ist, wird ein Fuzzy-Match gegen die bekannte Ground Truth durchgeführt (`match_person()`). Bei $\text{Score} \geq 90$ wird der Eintrag angereichert, andernfalls verbleibt er als ungematchter Rohdatensatz. Stimmen entweder Vor- oder Nachname mit einem vorhandenen Eintrag überein, erfolgt zusätzlich eine heuristische Fusion unter Berücksichtigung der Namensähnlichkeit.

5) Logging und Debugging:

Für jede nicht gematchte oder manuell überprüfungsbedürftige Person wird ein entsprechender Debug-Eintrag ausgegeben. Besonders fehleranfällige Fälle (z.B. Personen mit Rolle aber ohne Namen) werden mit niedrigem `match_score` und passender `review_reason` markiert. Die Hilfsfunktion `get_best_match_info()` dient in diesem Kontext als standardisiertes Protokoll für manuelle Prüfprozesse. Dieser Schritt initiiert die manuelle Nacharbeitung von ungematchten Entitäten, wie sie im [unmatched_logger.py](#) beschrieben ist.

¹¹⁴Je höher der Score, desto wahrscheinlicher ist die Person der tatsächliche Empfänger des Dokuments.

6) Kontextuelle Zählung der Nennungen:

Ergänzend zur initialen Zählung durch die Custom-Tags erfolgt mit `count_mentions_in_transcript_contextual()` eine kontextbasierte Überprüfung im Transkript. Die Suche erfolgt zeilenweise nach vollständigem Namen, Vor- oder Nachname. Zusätzlich wird geprüft, ob innerhalb von ± 1 Zeile die zugehörige Rolle erscheint. Um Doppelzählungen zu vermeiden, wird pro Kontextblock nur eine Zählung vorgenommen. So wird sichergestellt, dass auch aus dem Fliesstext korrekt abgeleitete Personenbezüge in die `mentioned_count`-Werte einfließen. Die Zählung wird durchgeführt, um daraus beispielsweise die Wichtigkeit von Personen ableiten zu können. Werden Personen oft genannt, indiziert dies eine hohe Wichtigkeit innerhalb des Dokumentes oder Netzwerkes. Eine statistische Auswertung wird so möglich, erfolgt bislang jedoch noch nicht. Sie kann mit den gewonnen Daten zu einem späteren Zeitpunkt umgesetzt werden.

5.3.3 Assigned_Roles_Module.py

Wie im `person_matcher.py` deutlich wird, ist auch das Erkennen von Rollen essentiell für das Erkennen von Personen. Sei dies um Informationen zu einzelnen Personen anzureichern, oder um sie anhand einer reinen Rollennennung im Text zu identifizieren. Für den vorliegenden Korpus wurden Rollen in fünf Kategorien strukturiert, die in Nodegoat modelliert sind: **Familie**, **Politisches Amt**, **Beruf**, **Vereins-Rolle**, und **Militärangehörige**. Durch diese Strukturierung ermöglichen Rollen auch die Definition von Beziehungstypen. Eine Rolle verbindet demnach immer eine Person mit einer Organisation,¹¹⁵ oder etwa durch Verwandtschaft mit einer anderen Person. Rollen beschreiben in dieser Modellierung auch viel mehr Points of Interest,

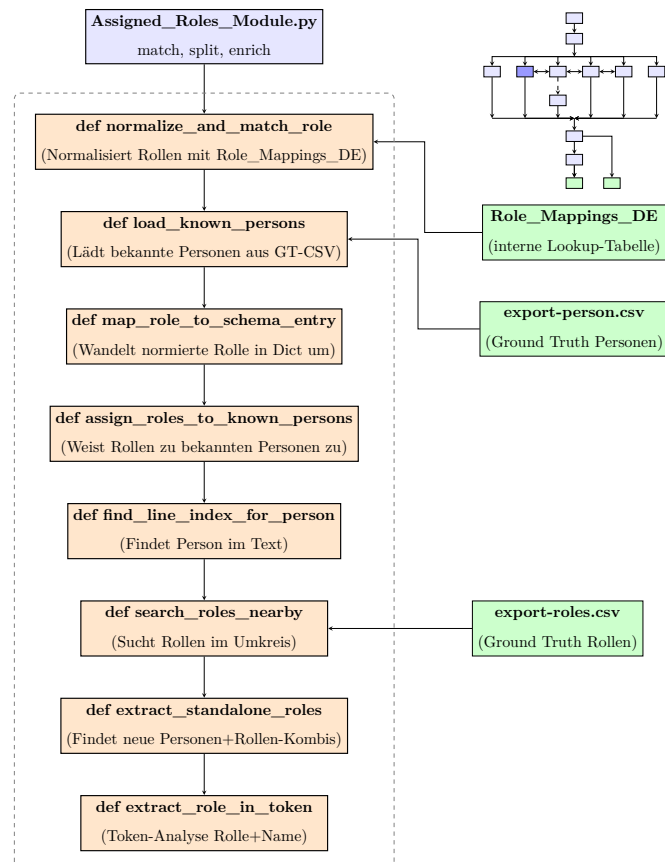


Abbildung 11:
Links: Prozessdiagramm für
`Assigned_Roles_Module.py`,
Rechts: Pipelineübersicht

¹¹⁵die Organisation muss dabei nicht bekannt sein; Sie wird zur Blank-Node

die potentielle Verbindungen aufzeigen. Das in der Kategorie `Familie` eingeordnete `Fräulein` ist beispielsweise kein Familien-Attribut im klassischen Sinne, sondern indiziert, dass eine Person ledig sein könnte. Diese leichte Unschärfe erhöht jedoch die pragmatische Verarbeitung in diesem Segment.

Weil Personen und Rollen so eng verknüpft sind, werden im `Assigned_Roles_Module.py` auch beide Ground Truth-CSV Tabellen initial geladen. Hinzu kommt eine interne Lookup-Tabelle `Role_Mapping_DE`, welche Synonyme und alternative Schreibweisen von Rollenbezeichnungen enthält. Neben dem Grundformen aus der Ground Truth werden alle Rollen auch in `normalize_and_match_role` grammatisch flexiert.¹¹⁶ Umgesetzt wird dies durch regelbasiertes Stemming mit festgelegten Endungen.¹¹⁷ Das Verfahren ist gezielt auf das kleine, bekannte Vokabular aus der `export-roles.csv` optimiert, insbesondere für maskuline Rollenform. Weibliche Rollennamen kommen im Untersuchungskorpus ausserhalb der Familienkategorie nur ein einziges Mal vor.¹¹⁸ Der Algorithmus prüft dennoch auch weibliche Endungen, um den Genderbias des Algorithmus zu minimieren. So bleibt der Code für eine potentielle Ausweitung des Korpus zu einem späteren Zeitpunkt robust.

Aus allen erkannten Varianten wird eine vollständige Liste `POSSIBLE_ROLES` erstellt, die als Vokabular für die Rollenextraktion dient. Innerhalb des Korpus treten Rollenbezeichnungen nicht einheitlich auf, sondern variieren. Sie lassen sich in drei Szenarien festlegen, die per Regex komplexe zusammengesetzte Möglichkeiten identifizieren und lösen.

- `ROLE_AFTER_NAME_RE`

Erkennt Rollen, die einer Person nachgestellt sind.

Beispiel: Alfons Zimmermann, Vereinsführer Männerchor Murg

- `ROLE_BEFORE_NAME_RE`

Erkennt Rollen, die einer Person vorangestellt sind, ggf. mit Organisationsbezug.

Beispiel: Vereinsführer des Männerchors Alfons Zimmermann

- `STANDALONE_ROLE_RE`

Erkennt Rollen, die alleinstehend oder nur mit Organisation genannt werden.

Beispiele: Vereinsführer oder Vereinsführer Männerchor Murg

Die Strings von mögliche Präpositionen zu bereinigen ist hierfür besonders wichtig. Gleichzeitig können diese in Kombination von Personen oder Organisationen als Indikator für eine Rolle verwendet werden.

Nach dieser Normalisierung wird jede erkannte Rollenbezeichnung mit

¹¹⁶d.h. dem jeweiligen Kasus angepasst.

¹¹⁷vgl. auf Github im `Assigned_Roles_Module.py`: # 3. Maskuline Flexionsendungen zurückführen (z.B. „vorsitzenden“ → „vorsitzender“) `suffixes = ["en", "ern", "em", "e", "n", "er", "es", "nt", "ner", "ners"]`

¹¹⁸die Rolle: `Damenschneiderin`

`map_role_to_schema_entry()` in einen strukturierten Schemaeintrag überführt. Dieser enthält eine eindeutige Rollen-ID aus der Ground Truth, die standardisierte Rollenbezeichnung, die zugehörige Rollenkategorie. Wenn sie aus dem Kontext extrahierbar sind, werden auch Bezüge zu einer Organisation, deren Name oder ID ergänzt.

Rollen, die nicht unmittelbar in der Form *Name-Rolle-Organisation* auftreten, werden durch eine gezielte Kontext- und Umfeldauswertung ergänzt. Dazu lokalisiert `find_line_index_for_person()` zunächst die betreffende Person im Transkript. Daraufhin untersucht `search_roles_nearby()` definierte Zeilenumfelder auf Rollenhinweise. Wie oben erläutert dienen Präpositionen als positive Indikatoren für Rollen-Relationen. Ergänzend identifiziert `extract_standalone_roles()` alleinstehende Funktionsangaben. Sofern möglich werden diese über nahe Organisationsnennungen oder globale Heuristiken aneinander gekoppelt. Komposita aus Namen und Rollen innerhalb eines Personen- oder Rollen XML-Tags werden mittels `extract_role_in_token()` in ihre Bestandteile zerlegt. Die so gewonnenen Rollen werden anschliessend mit bekannten Personen aus `export-person.csv` und Organisationen aus dem `organization_matcher` verknüpft. Dabei weist `assign_roles_to_known_persons()` Rollen bestehenden Personenobjekten aus dem `person_matcher` zu und ergänzt fehlende Felder, während Organisationsbezüge extrahiert und mit der Ground Truth abgeglichen werden.

Da Rollen mehrfach oder in unterschiedlichen Varianten auftreten können, erfolgt abschliessend eine Konsolidierung. Hierbei werden identische Rollenbezüge (gleiche Person, gleiche Organisation, gleiche Rolle) zusammengeführt, unsichere Fälle mit den Feldern `needs_review` und `review_reason` markiert und im Zusammenspiel mit dem `validation_module.py` Plausibilitätsprüfungen durchgeführt.

5.3.4 place_matcher.py

Während der `Person_Matcher` hochkomplexe Strukturen aus Vor- und Nachnamen, alternativen Namensformen und zugehörigen Rollen verarbeitet, mag die Identifikation von Ortsangaben auf den ersten Blick vergleichsweise trivial erscheinen. Eine eingehendere Analyse der zugrunde liegenden historischen Quellen zeigt jedoch, dass auch Ortsnamen eine erhebliche Komplexität aufweisen können.

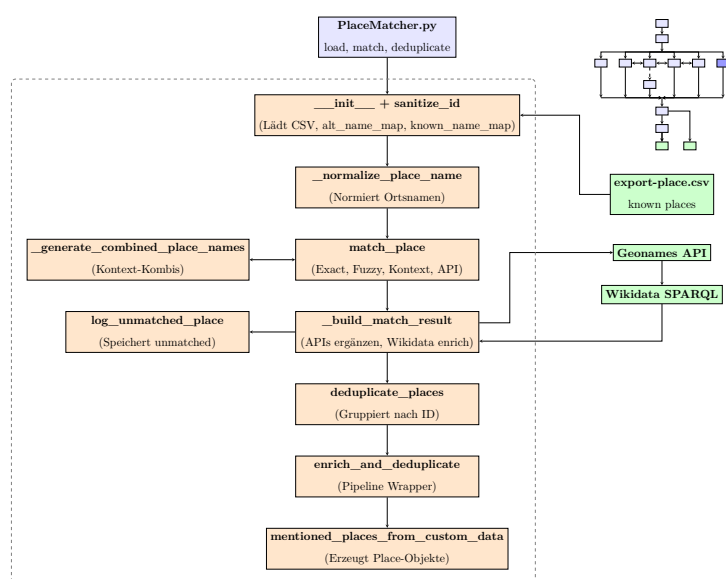


Abbildung 12:

Links: Prozessdiagramm für `place_matcher.py`,
Rechts: Pipelineübersicht

Das beginnt bei der grundlegenden Frage, was als Ort überhaupt zu definieren ist. Im Rahmen dieser Arbeit konnten acht Orts-Typen identifiziert werden, die es aus den Dokumenten extrahiert werden müssen: **Kontinente**, **Länder**, **Regionen**, **Siedlungen**, **Berge**, **Seen**, **militärische Stellungen** und **Gebäude**. Historische Strassenamen und Hausnummern können nicht mehr ohne teils immensen Rechercheaufwand nachvollzogen werden. Die Komplexität von Geografika steigert sich auch dadurch, dass sie oft Flächen und/oder Punkte meinen und einander beinhalten können. Ausserdem unterliegen sie im historischen Kontext weiteren komplizierten Veränderungen.¹¹⁹ Für die Ground Truth muss also beachtet werden, dass es sich bei dem Ort Murg nicht um den gleichnamigen Ort in der Schweiz handelt, aber auch, dass es nicht der selbe ist, wie in Geonames hinterlegt; Denn das befindet sich in Deutschland, nicht im Deutschen Reich. Geonames kann daher nur als Referenz für die Koordinaten gelten. Da für die Ground Truth der Fokus auf eben diesen Koordinaten oder GeoJSON Daten liegt, werden Adressen daher ausgenommen.

Um oben gelistete Entitäten unverwechselbar zu beschreiben, nutzt diese Arbeit eine Kombination mehrerer Identifier. Auf der einen Seite ist das die **Nodegoat-ID** aus der Ground Truth. Auf der anderen Seite wird diese durch Verknüpfungen der IDs von **GeoNames** und **Wikidata** ergänzt. Damit werden die Geo-Daten **FAIR**¹²⁰ und sind anschlussfähig an bestehende ¹²¹ Infrastrukturen.

Der **Person_matcher** löst gleich mehrere Probleme bei der Named Entity Recognition und des Entity Matchings. Das grundlegendste Problem ist dabei die Absicherung gegen inkonsistente Schreibweisen. Diese können durch OCR-Fehler entstehen, weil sie historische Schreibweisen oder auch Regionalismen¹²² sind. Für den Ort Murg existieren beispielsweise 26 unterschiedliche Schreibweisen, die in knapp über ein Dutzend Grundformvarianten¹²³ abgeleitet werden können. Die Schreibweise meint hier die Verwendung von Zeichen, die Grundform eine bereinigte Version.¹²⁴ Diese Formen müssen für die RegEx-Prüfungen bekannt sein, da diese ansonsten scheitern würden. Essentiell hierfür ist eine verlässliche Ground Truth.

Die Matching-Logik selbst kombiniert mehrere Verfahren. Zunächst erfolgt ein exakter Abgleich mit der Ground Truth, gefolgt von einem Fuzzy-Matching bei OCR-Fehlern oder oben ausgeführter alternativer Schreibweisen. Für nicht erkannte Orte greift ein API-basierter Ab-

¹¹⁹So mag eine Adresse in Lothringen unverändert geblieben sein, gehörte im 20. Jh. jedoch teils zu Deutschland, teils zu Frankreich. Geografische Analysen müssen daher nicht nur klären „Wo?“, sondern auch „Wann?“.

¹²⁰Findable, Accessible, Interoperable, Retrievable

¹²¹namerefsubsec:LOD

¹²²Regionalismen: regionale sprachliche Eigentümlichkeiten

¹²³Darunter etwa Murg/Baden, Murg (Baden), Murg Hochrhein, Murg a. HRh. u.a.

¹²⁴aus Murg (Baden) oder Murg /Baden wird immer murg baden, im Ggs. zu Murg Hochrhein, oder Murg am Rhein

gleich über GeoNames und Wikidata, dessen Ergebnisse mit Scorewerten versehen und mit Konfidenzstufen markiert werden. Scheitert auch dieser Lookup, werden die Daten durch den `unmatched_logger.py` verarbeitet und für die manuelle Weiterverarbeitung aufbereitet.

Auch der `place_matcher` fusst seine internen Verarbeitungsschritte auf das Tagging und der Vorarbeit durch das LLM. Manchmal sind diese Tags jedoch wegen des domänenunspezifischen Trainings zu sensitiv. So können Stadtteile beispielsweise als zwei separate Orte annotiert werden. Daher ist das Zusammenfügen von möglichen Segmenten zu einem bekannten Ort eine der Stärken des Moduls. Als Beispiel hierfür dient der Ortsname **Laufenburg-Rhina**. Würde die Pipeline die beiden Teilorte **Laufenburg** und **Rhina** als separate Orte erkennen und nicht als gemeinsame Menge **Laufenburg-Rhina**, wäre das Ergebnis verfälscht. Ein Brief bekäme so absurderweise zwei Orte, die gleichzeitig als Absendeort gelten würden. Der `place_matcher` überprüft daher mit einem `Content-Window` von ± 1 Wort innerhalb einer Zeile sowie über Zeilengrenzen hinweg auf möglichen Namen oder Alternativnamen. Diese Kombinationen werden einzeln mit der Ground Truth abgeglichen. Das Verfahren erlaubt auch das Matching komplexer Ortsnamen.

Neben den offensichtlichen Matching-Prozessen enthält das Modul auch eine Reihe von Funktionen, die im Hintergrund zur Sicherung der Datenqualität beitragen. So validiert `is_valid_place_name()` alle eingehenden Ortsreferenzen auf formale Plausibilität und filtert unbrauchbare oder irrelevante Einträge (z.B. Adjektivphrasen oder Kontextfragmente) konsequent aus. Die Funktion `_extract_name_only()` trennt relevante Ortsnamen zudem von begleitenden Funktionswörtern wie „in“, „bei“ oder „aus“, um die Vergleichbarkeit mit der Ground Truth zu erhöhen.

Diese unscheinbaren, aber zentralen Verarbeitungsschritte bilden die Grundlage für die im Anschluss durchgeführte Deduplikation. Hierbei werden erkannte Ortsreferenzen über `deduplicate_places()` zu konsolidierten Einträgen zusammengeführt. Kriterien hierfür sind identische IDs, aber auch Ähnlichkeiten in der normalisierten Schreibweise. Orte, die sich in keiner Form eindeutig zuordnen lassen, werden automatisiert im Logfile `unmatched_places.json` vermerkt und für eine spätere manuelle Prüfung vorbereitet.

5.3.5 organization_matcher.py

`organization_matcher.py` dient der Erkennung und Normalisierung von Organisationen in historischen Transkripten. Es kombiniert reguläre Ausdrücke mit unscharfem Vergleich (fuzzy matching), um eingegebene Strings mit bekannten Organisationen aus der Ground Truth-Datei `export-organisationen.csv` abzugleichen.

`extract_organization({})` bereinigt im nächsten Schritt die Eingabestrings zunächst, indem sie umschliessende Klammern entfernt,

innere Satzzeichen säubert und führende bzw. abschliessende Interpunktion streicht. Darüber hinaus wird eine Blacklist abgeglichen, um etwa Einträge wie „Verein“ als alleinstehende Organisation auszuschliessen - ein klares Match kann hier algorithmisch vorerst nicht gewährleistet werden.

Die bekannten Organisationen werden über die Funktion `load_organizations_from_csv()` eingelesen. Dabei entsteht eine strukturierte Liste von Dictionaries, die mit den Anforderungen aus `document_schemas.py` kompatibel ist und Hauptnamen, alternative Bezeichnungen sowie Identifier enthält. Um die Organisationen vergleichbar zu machen wird in `def normalize_org_name ( )` in drei Schritten normalisiert.

Zunächst werden mithilfe einer RegEx¹²⁵ sämtliche Zeichen entfernt, die nicht alphanumerisch sind. Erlaubt bleiben dabei Gross- und Kleinbuchstaben (einschliesslich deutscher Umlaute), Ziffern sowie Leerzeichen. So wird etwa aus dem Ausdruck „Männerchor Murg e.V.“ der bereinigte String „Männerchor Murg eV“.

Im zweiten Schritt wird der bereinigte String durch `.lower()` vollständig zu Kleinbuchstaben transformiert. Abschliessend werden mit `.strip()` etwaige führende oder nachfolgende Leerzeichen entfernt, sodass der Rückgabewert für weitere Vergleichsoperationen normiert vorliegt. An einem konkreten Beispiel verdeutlicht wird aus:

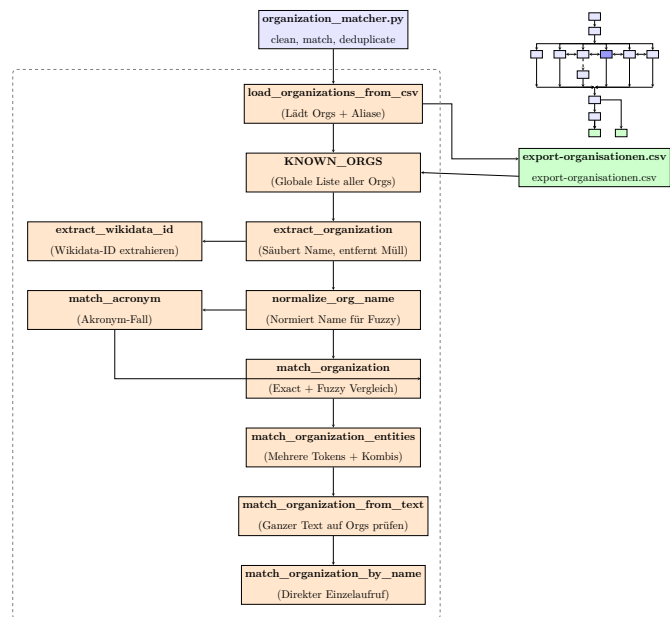


Abbildung 13: Links: Prozessdiagramm für `organization_matcher.py`, Rechts: Pipelineübersicht

¹²⁵ **Abk.:** Regular Expression


```
Input: " (Männerchor Murg e.V.) "
```

```
Output: "männerchor murg ev"
```

Um nicht nur vollständige Namen zu prüfen, sondern auch Akronyme¹²⁶, wird gezielt nach solchen in den `alternate_names` bekannter Organisationen gesucht.

Das Matching der Organisationen erfolgt im Anschluss in den drei zentralen Funktionen `match_organization()`, `match_organization_from_text()` und `match_organization_entities()`.

Die Funktion `match_organization()` bildet dabei den Kern des Vergleichsprozesses. Sie prüft zunächst, ob es sich bei der Eingabe um ein kurzes Akronym handelt und nutzt hierfür die Funktion `match_acronym()`. Falls kein solcher Sonderfall vorliegt, wird der Eingabewert mithilfe von Fuzzy Matching mit `fuzz.ratio` gegen alle bekannten Organisationsnamen sowie deren alternative Bezeichnungen geprüft. Das Ergebnis ist das bestbewertete Match mit einem Score oberhalb eines standardmässig auf vordefinierten Schwellenwerts.

Die Funktion `match_organization_from_text()` erweitert diesen Vergleich auf vollständige Transkriptionstexte. Dabei wird zunächst überprüft, ob ein bekannter Organisationsname direkt im Text vorkommt. Falls dies nicht der Fall ist, wird ein fuzzy-Teilvergleich durchgeführt (`fuzz.partial_ratio`), um auch unvollständige oder abweichende Schreibweisen zu erkennen. `match_organization_entities()` verarbeitet eine Liste roher Organisationseinträge aus der Extraktion und bereinigt sie mit `extract_organization()`. Zudem werden aufeinanderfolgende Namen wie „Männerchor“ und „Murg“ zu einem kombinierten Eintrag zusammengeführt. Anschliessend wird für jeden dieser bereinigten Namen ein fuzzy-Matching gegen die Ground Truth durchgeführt, wobei Treffer mit Score und `confidence = "fuzzy"` zurückgegeben werden.

5.3.6 letter_metadata_matcher.py

Der `letter_metadata_matcher` widmet sich der Extraktion inhaltlicher Metadaten aus Briefen und Postkarten. Der Fokus liegt auf Informationen über die Autorenschaft, den Empfänger und Empfängerinnen sowie deren Rollen. Darüber hinaus sollen auch Daten und Orte nachvollzogen werden, mit einem Fokus auf den Absende- und Empfangsort. Da diese Briefe und Postkarten sehr ähnlich strukturiert sind, liegt der Fokus dieses Moduls auf der Extraktion ihrer Daten.

¹²⁶Abkürzungen

Pattern Matching in Briefen und Postkarten

Die zentrale Annahme ist, dass diese Metadaten meist im Textkörper selbst enthalten sind und anhand wiederkehrender sprachlicher Muster bzw. Indikatoren erkannt werden können. Ein Autor oder eine Autorin beendet einen Brief beispielsweise in der Regel mit einer Grussformel und einer Unterschrift. Die Pipeline macht sich das zunutze, indem diese Patterns als Marker in einem String dienen, und zusätzlich zu den Custom-Tags im XML File auf eine Autorenschaft hinweisen können. Diese Marker sind in fünf Listen auf RegEx-Basis notiert.

- `GREETING_PATTERNS` als Schlussformel in einem Brief zeigt die Autorenschaft an.
Beispiel: mit freundlichen Grüßen oder Heil Hitler.
- `RECIPIENT_PATTERNS` Erkennt direkte Anreden von Empfänger:innen im Text.
Beispiel: An Herrn/Frau oder Lieber Fritz
- `direct_patterns` zeigen die unmittelbaren Empfänger eines Briefes an.
- `INDIRECT_RECIPIENT_PATTERNS` zeigen an, dass ein Brief oder Postkarte an einen Zwischenempfänger geht.
Beispiel: zu Händen von
- `SIGNATURE_PATTERNS` ist eine auf das nachfolgende `ROLE_PATTERNS` stützende RegEx, die reine Rollenzeilen am Ende eines Briefes erkennt. Beispiel:
der Vereinsführer
Alfons Zimmermann.
- `ROLE_PATTERNS` ist eine dynamisch generierte RegEx, die auf der Rolle basierende Signaturen überprüft. Grundlage ist eine aus `export-roles.csv` geladene Liste bekannter Rollenbegriffe.

Extraktion von Autorinnen und Autoren

Die konkrete Anwendung dieser Patterns erfolgt in den jeweiligen Funktionen, deren Grundlage eine Verarbeitung der raw strings ist. Die Funktion `extract_authors_raw` analysiert hierfür zu Beginn alle Textzeilen innerhalb der Transkription, ohne dabei auf strukturierte XML-Tags angewiesen zu sein. Stattdessen orientiert sich die Funktion an den oben erläuterten typischen linguistischen Mustern. Somit kann die Funktion auch dann Ergebnisse liefern, wenn strukturierte XML-Tags wie `<author>` fehlen oder unvollständig sind, was als Backup-Lösung die LLM-Annotation komplettiert. Ziel ist es, möglichst frühzeitig Hinweise auf Autor:innen zu erkennen und diese mit einem einfachen Scoring-System zu bewerten. Das erkannte Scoring wird in einem separaten Feld `author_score` festgehalten und dient als Bewertungsgrundlage für die

spätere Auswahl der wahrscheinlichsten Autor:innen.

Da Autoren in der Regel am Briefende unterschreiben, werden die Zeilen des Transkripts von unten nach oben analysiert. Dabei wird zunächst geprüft, ob eine typische Rollensignatur wie **Der Vereinsführer** alleine am Dokumentende steht. Wird eine solche erkannt, wird unmittelbar eine entsprechende Rolle extrahiert, ohne dass ein Name erforderlich ist.

Findet sich keine Rollenzeile, durchsucht die Funktion den Text nach einer Schlussformel basierend auf den in `GREETING_PATTERNS` definierten Mustern. Der Textabschnitt, der auf diese Grussformel folgt, wird im nächsten Schritt nach Signaturen durchsucht. Die Auswertung erfolgt dabei in mehreren aufeinanderfolgenden Prüfungen.

Zunächst wird versucht, eine Kombination aus Rolle und Nachname zu identifizieren¹²⁷. Alternativ wird geprüft, ob Rolle und Name in zwei aufeinanderfolgenden Zeilen vorkommen. In Fällen, in denen sowohl Rolle und Name in benachbarten Zeilen auftreten – etwa **"Der Vereinsführer"** gefolgt von **"Alfons Zimmermann"** – wird diese kontextuelle Nähe berücksichtigt. Die Funktion vergibt in solchen Fällen einen erhöhten `match_score`, da die unmittelbare Abfolge als zuverlässiger Hinweis auf eine zusammengehörige Signatur gewertet wird. Technisch erfolgt die Prüfung auf ± 1 Zeile Abstand im Transkripttext.

Weitere Varianten umfassen typische Signaturformate wie **Max Ganter**, **Schriftführer** oder Namensinitialen wie **M. Ganter**. Auch vollständige Namen (Vorname + Nachname) oder einzelne Namen (z.B. nur **Fritz**) werden erkannt und mit Platzhaltern übernommen. In Ausnahmefällen werden auch unvollständige Namensformen¹²⁸ übernommen, sofern sie in typischen Signaturkontexten auftreten. Dabei wird der fehlende Vor- oder Nachname mit einem Platzhalter (`None`) belegt und der Eintrag entsprechend mit `needs_review = true` markiert. Die Übernahme solcher Platzhalter erfolgt jedoch selektiv und nur bei klarer Signaturstruktur.

Wird kein valides Format erkannt, liefert die Funktion ein leeres Dictionary mit einem Eintrag für `closing`, der die gefundene Grussformel dokumentiert.

Neben der heuristischen Pattern-Suche durch `extract_authors_raw()` stützt sich die Pipeline auch auf strukturierte Annotationen aus dem XML. Die Funktion `extract_author_from_custom_tag()` sucht gezielt nach `<author>`-Tags und übernimmt diese, sofern sie vollständig sind. Im Anschluss wird jeder Kandidateneintrag durch `resolve_llm_custom_authors_recipients()` validiert und bewertet. Insbesondere dann, wenn er von einem LLM vorgeschlagen oder aus unstrukturiertem Text extrahiert wurde, kommt die Bewertung zum tragen. Abschliessend übernimmt `letter_match_and_enrich()` die Anreicherung durch bekannte Personen aus der Ground Truth-Liste. Hierbei werden unvoll-

¹²⁷z.B. **Der Vereinsführer Zimmermann**

¹²⁸etwa Initialen oder Einzelnamen

ständige Einträge (z.B. nur ein Vorname) durch vollständigere bereits in `mentioned_persons` aufgenommene Personen ersetzt, sofern möglich. Auch in Fällen, in denen keine strukturierte LLM-Annotation vorliegt, erfolgt eine nachgelagerte Prüfung gegen die Ground Truth. Die Funktion `resolve_llm_custom_authors_recipients()` gleicht unvollständige oder generische Einträge – etwa lediglich "Otto" oder "Herrn" – mit den zuvor erkannten `mentioned_persons` ab. So können auch partielle oder schwach formatierte Namensangaben aufgelöst und angereichert werden.

Die Pipeline deckt jedoch nicht nur einzelne Autoren ab. `match_authors()` kombiniert die Roh-Extraktion von Autor:in (`extract_authors_raw()`) mit der Anreicherung durch `letter_match_and_enrich()`. Ist das Ergebnis leer, prüft sie, ob zumindest eine Rolle ohne Namensangabe vorliegt. In diesem Sonderfall wird ein Eintrag mit leerem Namen, aber übernommener Rolle erstellt. Die Rolle wird normalisiert (`normalize_and_match_role()`) und in ein `role_schema` überführt. Der Rückgabewert enthält `needs_review = True` sowie eine niedrige `match_score` (10), um auf die Unsicherheit der reinen Rollenangabe hinzuweisen.

Extraktion von Empfängerinnen und Empfängern

Die Extraktion potenzieller Empfänger:innen erfolgt analog zur Autorenerkennung über eine Kombination aus regulären Ausdrücken, strukturierten XML-Tags und nachgelagerter Anreicherung. Im Gegensatz zur Autorenschaft, die typischerweise am Ende eines Dokuments steht, werden Empfänger:innen meist im Briefkopf genannt. Die Funktion `extract_recipients_raw()` konzentriert sich daher auf die ersten fünf Zeilen des Transkripts. Dieser Abschnitt enthält häufig Adressierungen wie „An Herrn Otto Bollinger“ oder zeilenweise strukturierte Formate, die auf eine Empfängeradresse hindeuten. Die Funktion arbeitet unabhängig von expliziten XML-Tags wie `<recipient>`, wodurch sie auch bei unvollständig annotierten Dokumenten Ergebnisse liefern kann.

Die Erkennung erfolgt in drei heuristischen Stufen, abhängig von der Länge und Struktur der jeweiligen Adressierung. Einzeilige Formulierungen wie „An Herrn Otto Bollinger“ werden als besonders zuverlässig bewertet (`recipient_score = 90`, `confidence "header-inline"`). Dreizeilige Strukturen wie

```
An
Herrn
Otto Bollinger
```

werden als semistrukturiert interpretiert und mit einem Score von 80 markiert (`confidence = "header-3line"`). Bei zweizeiligen Konstruktionen – etwa Herrn in ei-

ner Zeile und ein Name in der nächsten – wird aus methodischer Vorsicht ein niedrigerer Score (70) vergeben, da diese auch als Fliesstextnennung interpretiert werden könnten (`confidence = "header-2line"`). Diese Liste von Empfänger Kandidaten wird anschliessend in mehreren Schritten angereichert und validiert.

In einem weiteren Schritt werden `<recipient>`-Tags aus dem XML durch `extract_recipient_from_custom_tag()` übernommen, sofern vorhanden. Wie bei den Autoren durchlaufen Rezipienten danach die Funktion `resolve_llm_custom_authors_recipients()`. So werden die Rohdaten zusammengeführt, und anschliessend eine Validierung vorgenommen. Insbesondere unvollständige oder generische Einträge wie „Herrn“ oder häufig vorkommende Namen wie „Otto“¹²⁹ werden entweder mit dem Vermerk `needs_review = true` versehen oder durch bereits `mentioned_persons` ersetzt. Diese Ersetzung erfolgt abschliessend in `letter_match_and_enrich()`. Unvollständige Einträge von Autor:innen werden durch vollständigere Einträge aus `mentioned_persons` ersetzt, sofern diese zuvor im Text erkannt wurden. Mit `extract_multiple_recipients_raw()` können darüber hinaus potenzielle Empfänger:innen aus dem Transkripttext anhand direkter und indirekter Anredeformen aus den jeweiligen Patternlisten extrahiert werden. Im Gegensatz zu `extract_recipients_raw()`, das sich auf die ersten Zeilen des Dokuments konzentriert, analysiert die Funktion `extract_multiple_recipients_raw()` den gesamten Transkripttext. Sie erkennt direkte und indirekte Anredeformen auch im Fliesstext¹³⁰ und erweitert damit die Empfängerliste über die Briefkopfregion hinaus. Direkte Formulierungen wie *Lieber Otto* werden mit `recipient_score = 100` als besonders zuverlässig bewertet. Indirekte Anreden wie *zu Händen des Herrn Alfons Zimmermann* werden mit Score 70 erfasst. Die Ergebnisse enthalten Vorname, Nachname (falls vorhanden), Rolle (leer), Score und Confidence-Label. Durch die unterschiedlichen Ratings im Recipient-Score können so Staffellungen dargestellt werden.

Extraktion des Absendeorts und Empfangsort

Die Extraktion des Absendeorts (`creation_place`) und des Empfangsorts (`recipient_place`) erfolgt in einem mehrstufigen Verfahren, das strukturierte XML-Informationen, heuristische Pattern-Erkennung und ein darauf aufbauendes Ground Truth-Matching kombiniert. Ziel ist es, auch in unvollständig annotierten oder uneinheitlich formulierten Briefen möglichst präzise Ortsinformationen zu identifizieren und anzureichern. Die Verarbeitung ist dabei modular aufgebaut und erfolgt durch vier aufeinander abgestimmte Funktionen.

¹²⁹die Ground Truth kennt 11 verschiedene Personen namens Otto

¹³⁰etwa im Kontext von Zwischenempfängern oder weitergeleiteten Briefen

Die Funktion `extract_places_and_date()` bildet den Einstiegspunkt für die Ortserkennung. Sie durchsucht alle Zeilen im XML-Dokument (`<TextLine>`) nach Attributen vom Typ `custom`, in denen strukturierte Angaben zu `creation_place`, `recipient_place` und `creation_date` hinterlegt sein können. Wird ein solches Attribut gefunden, wird der zugehörige Ort mittels RegEx aus dem Text extrahiert. Grundlage hierfür sind die LLM-ergänzten, im Transkribus-Export enthaltenen Offset- und Length-Parameter, die den Ortstext innerhalb der Zeile markieren. Die Funktion liefert drei `raw-strings` zurück: den vermuteten Absendeort (`raw_creation_place`), den Empfangsort (`raw_recipient_place`) sowie das Datum der Entstehung (`creation_date`)¹³¹. Sind keine Custom-Tags vorhanden, aktiviert die Funktion ein Fallback-Verfahren, das den Briefkopf anhand typischer Formate mit regulären Ausdrücken analysiert. Erkannt werden insbesondere Muster wie `Ort, den dd.mm.yyyy` oder `An ... in Ort`. Die so ermittelten Raw-Strings werden in `assign_sender_and_recipient_place(...)` weiterverarbeitet. Diese Hauptfunktion ruft zunächst `extract_places_and_date()` auf, um die Rohdaten zu extrahieren, und analysiert dann alle Zeilen des Transkripts, um auch Kontexte für zusammengesetzte Ortsbezeichnungen¹³² zu erfassen. Anschliessend wird das Orts-Matching über die Hilfsfunktion `enrich_place_candidate()` durchgeführt. Diese Funktion gleicht den erkannten Komposit-Ortsnamen mit einer Ground Truth-Liste ab, in der alle bekannten Orte und ihre Varianten hinterlegt sind. Neben exakten Treffern erlaubt das Matching auch Fuzzy Matching über die Levenshtein-Distanz. Ergibt sich dabei ein Treffer, wird ein Score berechnet und mit weiteren Metadaten¹³³ angereichert. Im Kontext von Empfängeradressen wird zusätzlich geprüft, ob im unmittelbaren Umfeld ein Vereins- oder Organisationsname vorkommt. In diesem Fall wird ein Score-Malus vergeben, da das ein Hinweis auf eine mögliche Verwechslung zwischen Ort und Organisation sein kann.

Eine besondere Stärke der Pipeline liegt in der Erkennung zusammengesetzter Ortsnamen. Die Funktion `find_combined_place()` durchsucht benachbarte Zeilen nach Kombinationen wie `Laufenburg` und `Rhina`, die zusammen in einem Eintrag der Ground Truth-Liste vorkommen (`Laufenburg-Rhina`). Wird eine solche Kombination erkannt, wird sie bevorzugt, erhält einen Bonus im Matching-Score und ersetzt die Einzeltreffer. Weil so auch Teilorte oder Stadtteile abgebildet werden können, wird die Ortserkennung deutlich präziser.

Abschliessend wird durch `finalize_recipient_places()` der am besten bewertete Empfängerort ausgewählt. Die Funktion sortiert alle erkannten Ortskandidaten nach Score und übernimmt den besten Eintrag als `recipient_place`. Weitere valide, aber weniger eindeutige Orte werden ebenfalls gespeichert, jedoch mit dem Attribut `needs_review = true` gekennzeichnet,

¹³¹dazu mehr im nächsten Abschnitt

¹³²z.B. `Laufenburg-Rhina`

¹³³z.B. `Nodegoat_ID`

um eine manuelle Prüfung zu ermöglichen.

Die finale Funktion `match_place()` kann mehrere potenzielle Ortstreffer zurückgeben, insbesondere wenn es konkurrierende Varianten mit ähnlichem Namen gibt. In diesem Fall wählt die Funktion `finalize_recipient_places()` den Ort mit dem höchsten Score als `recipient_place` aus, während weitere plausible, aber unsicherere Treffer mit dem Vermerk `needs_review = true` gespeichert werden. Durch das modulare Zusammenspiel der vier Funktionen können auch in schwierig strukturierten oder unvollständig annotierten Dokumenten präzise Ortsangaben extrahiert werden.

Extraktion des Erstellungsdatum

Die Erkennung von Entstehungsdaten (`creation_date`) erfolgt im Modul `letter_metadata_matcher.py` primär über die von oben bereits bekannte Funktion `extract_places_and_date()`. Im Zentrum steht die Auswertung strukturierter Custom-Tags im PAGE-XML sowie ein regulärer Fallback-Mechanismus zur heuristischen Analyse des Briefkopfs.

Im ersten Schritt durchsucht die Funktion alle `<TextLine>`-Elemente nach dem Attribut `custom`, das von Transkribus für semantische Annotationen verwendet wird. Wird ein Eintrag vom Typ `creation_date` oder `date` erkannt, wird mittels RegEx nach einem `when="YYYY-MM-DD"`-Feld innerhalb des custom-Attributs gesucht. Dieser Wert wird direkt übernommen und als `creation_date` gespeichert. Da bei dem manuellen Tagging besonders auf die korrekten Daten geachtet, und diese händisch mit `when` immer ergänzt wurden, sind diese Angaben in jedem Fall vorhanden.

Wird jedoch kein Custom-Tag erkannt, greift ein Fallback-Verfahren: Die Funktion analysiert hierfür die ersten fünf Zeilen des Dokuments – typischerweise der Briefkopf – auf typische Orts-Datumskombinationen. Dazu zählt insbesondere das Muster

Ort, den dd.mm.yyyy

Die RegEx erfasst dabei sowohl den Ort (als möglichen Absendeort) als auch das Datum. Sie sind daher auch gemeinsam in dieser Funktion aufgenommen. Wird ein solches Datum gefunden, wird es als `creation_date` übernommen, selbst wenn kein zugehöriger Ort erkannt werden konnte. Die Erkennung erfolgt dabei tolerant gegenüber dem optionalen Wort „den“ und erlaubt sowohl zwei- als auch vierstellige Jahresangaben¹³⁴. Ist kein solcher Eintrag vorhanden, bleibt `creation_date = None`.

In Kombination mit `assign_sender_and_recipient_place()` wird das erkannte Datum an-

¹³⁴erkannt werden also sowohl 1939, als auch nur 39

schliessend gemeinsam mit den Ortseinträgen an die Hauptverarbeitung zurückgegeben und als `attributes["creation_date"]` in den Metadaten des Dokuments gespeichert. Im Gegensatz zu `mentioned_dates`¹³⁵, die beliebige Datumsangaben im Fliesstext zählt, bezieht sich `creation_date` ausschliesslich auf das Erstellungsdatum des Dokuments

5.3.7 `type_matcher.py`

Dieses Modul besteht aus nur einer einzigen Funktion: `def get_document_type ( )`. Diese zerlegt den Namen der zu verarbeitenden Datei und extrahiert daraus die siebenstellige Transkribus-ID und die Seitennummer. Es folgt ein Vergleich in der `Akten_Gesamtübersich.csv`, die als Ground Truth alle Dateien im Korpus listet.

Da während der Transkription in dieser Tabelle auch die Dokumententypen festgehalten wurden, kann über einen einfachen Lookup für jede Seite der zugehörige Typ extrahiert werden. Mögliche Kategorien sind unter anderem „Brief“, „Postkarte“ und „Protokoll“.¹³⁶

Falls kein passender Eintrag in der Ground Truth gefunden wird, bietet die Funktion ein optionales Fallback: Wird zusätzlich ein `xml_path` übergeben, so wird versucht, den Dokumenttyp direkt aus einem `<Dokumententyp>`-Tag innerhalb der zugehörigen XML-Datei auszulesen. Auf diese Weise wird sichergestellt, dass jede Seite im JSON-Output einen Typ zugewiesen erhält – entweder verlässlich aus der CSV oder über das XML-Fallback.

Die extrahierte Typinformation wird im Attributblock des finalen JSON unter `attributes["document_type"]` gespeichert. So kann später seitengenau nach Dokumententypen gefiltert und eine differenzierte Verteilung der Typen innerhalb einer Akte nachvollzogen werden.

`event_matcher.py`

Das Modul `event_matcher.py` dient der Erkennung, Strukturierung und Anreicherung von Ereignistags aus den Transkribus-Dokumenten. Die zugrundeliegende Funktion `extract_events_from_xml()` verarbeitet die gesamte XML-Datei zeilenweise und extrahiert jene Textstellen, die als Ereignis markiert wurden oder im unmittelbaren Zusammenhang mit einem Ereignisblock stehen. Grundlage dafür ist die Auswertung des `custom`-Attributs einzelner `<TextLine>`-Elemente, insbesondere wenn dieses explizit mit „event“ gekennzeichnet ist.

Die Funktion arbeitet blockbasiert: Ereignisblöcke bestehen in der Regel aus mehreren aufeinanderfolgenden Zeilen, die durch inhaltliche Merkmale wie Bindestrichs, kleingeschriebene Fortsetzungszeilen oder fehlende Datumsmarkierungen zusammenhängend interpretiert werden.

¹³⁵mehr in [date_matcher.py](#)

¹³⁶siehe Kapitel [Quellenbeschreibung](#)

Die Funktion `is_continuation()` prüft dabei, ob eine Zeile an die vorhergehende angeschlossen werden kann, etwa durch typische Anschlusswörter oder formale Merkmale. Sobald ein abgeschlossener Block erkannt ist, wird dieser mit der Hilfsfunktion `build_event()` zu einem vollständigen Ereignisobjekt zusammengeführt.

In `build_event()` wird der aus mehreren Zeilen bestehende Textblock zunächst als Fliesstext zusammengesetzt und inhaltlich analysiert. Es werden mehrere Entitäten erkannt und dem Ereignis zugeordnet:

- **Orte:** Über die Funktion `match_place()` aus dem Modul `place_matcher.py` werden potenzielle Ortsnamen im Text identifiziert und mit der Ground Truth abgeglichen. Dabei wird jeder erkannte Ort zusätzlich durch eine Plausibilitätsprüfung überprüft, bevor er als `Place`-Objekt übernommen wird.
- **Daten:** Die Funktion `extract_custom_date()` durchsucht die XML-Zeile nach Datumsangaben in den XML-Tags. Wenn kein strukturiertes Datum vorhanden ist, aber einfache numerische Formate wie „15.03“ im Fliesstext erkannt werden, werden diese als Datum übernommen.
- **Organisationen:** Über `match_organization_from_text()` wird der Textblock mit bekannten Organisationseinträgen abgeglichen. Bei Übereinstimmung werden entsprechende Organisationen als strukturierte Objekte ergänzt.
- **Personen:** Mögliche Namen werden durch reguläre Ausdrücke identifiziert und mit Hilfe der Funktion `extract_name_with_spacy()` in Vor- und Nachnamen getrennt. Anschließend erfolgt ein Abgleich mit der Personen-Ground Truth über `match_person()`. Positive Treffer werden inklusive Match-Score und Herkunftskennzeichnung als `Person`-Objekte dem Ereignis zugeordnet.

Die fertigen Ereignisse bestehen jeweils aus einer Kurzbeschreibung (in der Regel der ersten Zeile), einer ausführlichen Beschreibung (bestehend aus allen zugehörigen Textzeilen), einem Datumsfeld, einer Ortsangabe und einer strukturierten Liste aller beteiligten Orte, Organisationen und Personen. Der vollständige Satz aller so erkannten Ereignisse wird am Ende als Liste von `Event`-Objekten zurückgegeben.

Das Modul arbeitet vollständig dateibasiert und benötigt als einzige Eingabe den Pfad zur Transkribus-XML-Datei sowie eine initialisierte Instanz des `PlaceMatcher`. Es greift auf zentrale Komponenten der Projektarchitektur zurück, darunter die Ground Truth-Listen für Orte, Personen und Organisationen. Die extrahierten Ereignisse werden im finalen JSON unter dem Attribut `events` gespeichert. Da ein Event ein eher abstraktes Konstrukt ist, liegt der Fokus der Pipeline weniger auf diesem Modul, das zu einem späteren Zeitpunkt beispielsweise durch präziseres Prompten für das Eventtagging optimiert werden soll.

date_matcher.py

Das Modul `date_matcher.py` dient der systematischen Extraktion, Normalisierung und Zählung von Datumsangaben in den Transkribus-Dokumenten. Es basiert auf der Auswertung strukturierter Angaben, die im Rahmen der Transkription über XML-Custom-Tags im `custom`-Attribut einzelner `<TextLine>`-Elemente eingebettet werden. Diese Daten gelten innerhalb des Korpus als zuverlässig, da sie während der manuellen Korrekturprozesse in Transkribus einheitlich normiert und im Format `dd.mm.yyyy` ergänzt werden.

Treten im historischen Text verkürzte Datumsangaben wie „1. d. Mts“ auf, so handelt es sich um Abkürzungen, die bei der Transkription mit einem entsprechenden `abbreviation`-Tag markiert werden.¹³⁷ Lässt sich aus dem weiteren Kontext¹³⁸ ein vollständiges Datum erschliessen, kann dieses anschliessend in strukturierter Form übernommen und im JSON als normiertes Datum gespeichert werden.

Innerhalb der Verarbeitungspipeline wird das Modul über die Funktionen `extract_custom_date()` und `combine_dates()` aufgerufen. Zunächst durchläuft `extract_custom_date()` das XML-Dokument und extrahiert alle `custom`-Attribute, die ein `date{...}`-Muster enthalten. Die Inhalte dieser Attribute werden bereinigt und zur weiteren Analyse an die Funktion `extract_date_from_custom()` übergeben.

Diese Funktion überprüft mithilfe RegEx, ob der String tatsächlich eine gültige Datumsangabe enthält. Dabei wird insbesondere nach einem `when`-Feld gesucht, das im Inneren des `date`-Blocks enthalten ist. Die in diesem Feld hinterlegten Daten werden anschliessend mit der Funktion `parse_custom_attributes()` als Key-Value-Paare interpretiert. Liegt ein gültiges Datum vor, wird dessen Format mit `normalize_to_ddmmyyyy()` überprüft und gegebenenfalls vereinheitlicht.

Unterstützt werden mehrere Eingabeformate, darunter standardisierte Formen wie `dd.mm.yyyy`, ISO-Formate wie `yyyy-mm-dd` oder zweistellige Jahresangaben, die automatisch in vierstellige Jahre des 20. Jahrhunderts umgewandelt werden. Zusätzlich erkennt die Funktion auch Intervallangaben wie `01/03.04.1944`, bei denen ein Datumsbereich über einen Schrägstrich kodiert ist. Solche Intervalle werden in strukturierter Form mit einem `from`- und `to`-Wert als `date_range` gespeichert.

Die Funktion `combine_dates()` führt schliesslich alle erkannten Einzel- und Intervallangaben zusammen, zählt deren Häufigkeit im Dokument und erstellt eine deduplizierte, sortierte Liste für den späteren Export. Dabei wird jede identifizierte Angabe – ob Einzeldatum oder Zeitspanne

¹³⁷Die Funktionsweise dieser Markierungen sowie die heuristische Auflösung solcher verkürzter Angaben wird im Anhang [Tagging in Transkribus](#) erläutert.

¹³⁸Zum Beispiel durch Hinweise im Seiteninhalt oder durch übergeordnete Informationen im Umfeld der Akte

– um eine Zählung der Nennungen ergänzt. Bei Intervallen wird zusätzlich der Originalstring dokumentiert, aus dem die Angabe hervorging.

Das Ergebnis der Verarbeitung wird im Feld `mentioned_dates` gespeichert. Jeder Eintrag enthält entweder ein einzelnes Datum oder einen Datumsbereich, ergänzt um die Häufigkeit und gegebenenfalls den ursprünglichen Wortlaut aus dem `custom`-Attribut.

Das Modul arbeitet unabhängig von externen Ressourcen und benötigt lediglich das XML-Baumobjekt des jeweiligen Dokuments. Die so gewonnenen Zeitangaben bilden die Grundlage für die chronologische Einordnung, Kontextualisierung und Auswertung der digitalen Quellenbasis.

5.3.8 unmatched_logger.py

Das Modul `unmatched_logger.py` dient der systematischen Protokollierung von Entitäten, die in der aktuellen Version der Ground Truth noch nicht enthalten sind. Diese Protokolle bilden die Grundlage für weiterführende Recherchen, durch die die Ground Truth schrittweise ergänzt und verbessert werden kann.

Innerhalb der Verarbeitungs-Pipeline wird das Modul `unmatched_logger.py` über die Funktion `process_single_xml()` im Hauptprogramm aufgerufen.

Bereits in der Testphase kam das Modul mehrfach zum Einsatz, um die Erkennung und Zuordnung bislang nicht erfasster Entitäten zu überprüfen.

Im Kern stellt das Modul die Funktion `log_unmatched_entities` bereit. Diese übernimmt die von den zuvor beschriebenen Matcher-Funktionen ermittelten Entitäten und prüft, ob sie in den entsprechenden Ground Truth-CSV-Dateien vorhanden sind.

Die Suche erfolgt iterativ innerhalb der Listenstrukturen für Personen, Orte, Rollen, Organisationen und Ereignisse. Wird eine Entität über ein XML-Custom-Tag einer dieser Kategorien zugewiesen, ohne dass sie in der Ground Truth verzeichnet ist, wird sie in einer spezifischen JSON-Datei protokolliert.

Die folgenden Dateien werden dabei erzeugt:

- `unmatched_persons.json`
- `unmatched_places.json`
- `unmatched_roles.json`

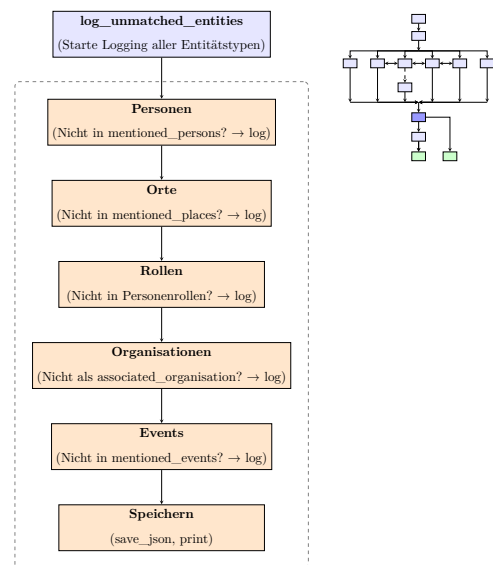


Abbildung 14:
Oben links: Prozessdiagramm für `unmatched_logger.py`,
Oben rechts: Pipelineübersicht

- `unmatched_events.json`
- `unmatched_organisations.json`

Zusätzlich werden alle Einträge in einer zusammengeführten Datei `unmatched.json` gespeichert, um einen vollständigen Überblick nicht zugeordneter Entitäten zu gewährleisten.

Alle Ergebnisse werden zudem in einer Datei `unmatched.json` gespeichert, um einen Gesamtüberblick zu erhalten.

5.3.9 validation_module.py

Das Modul `validation_module.py` übernimmt die strukturierte Qualitätskontrolle der erzeugten JSON-Dokumente in der finalen Verarbeitungsphase. Es wird in der zentralen Verarbeitungsfunktion `process_single_xml()` unmittelbar vor dem Speichern der Ausgabedateien aufgerufen:

```

1 validation_result = validate_extended(doc)
2     if validation_result:
3         print(f"[WARN] Validierungsfehler im Dokument {full_doc_id}:")
4         for err_type, errors in validation_result.items():
5             print(f"    - {err_type}: {'', '.join(errors)}")
6

```

Die Validierung erfolgt anhand des erzeugten `BaseDocument`-Objekts und basiert auf festen Regeln zur Überprüfung von Pflichtfeldern, Formatvorgaben sowie typischen Erkennungsfehlern bei Personen- oder Ortsangaben.

Die Hauptfunktion `validate_extended(doc)` analysiert das Dokument auf Inkonsistenzen und gibt ein Dictionary mit Fehlermeldungen pro Feld zurück. Neben formalen Anforderungen (z.B. korrektes Datumsformat) werden auch häufige Named-Entity-Fehlklassifikationen erkannt, etwa wenn Titelwörter als Vorname interpretiert werden.

Beispielhafte Prüfungen:

- **Datumsformat:** Ein Eintrag wie 1941.5.28 wird zurückgewiesen, da das erwartete Format YYYY.MM.DD lautet.
[creation_date] Fehlend oder ungültiges Format (YYYY.MM.DD)
- **Leere Empfängerliste:** Bei Dokumenten vom Typ Brief oder Postkarte wird überprüft, ob mindestens ein valider Empfänger mit Name oder Rolle vorhanden ist. Ist dies nicht der Fall, erfolgt eine Fehlermeldung.
[recipients] Empfänger fehlt oder ist ungültig
- **Fehlerhafte Personennamen:** Wird ein Vorname wie `des` oder `Herrn` erkannt, erfolgt

eine heuristische Warnung auf möglichen OCR- oder Parsingfehler.

`[mentioned_persons[0]] Möglicher Fehlname: des`

Die Hinweise des Validierungsmoduls werden insbesondere während der Erprobungs- und Optimierungsphasen intensiv genutzt, etwa zur Feinjustierung der Methoden der Named-Entity-Recognition bei Personen. Sie dienen nicht nur der strukturellen Sicherung eines gültigen JSON-Outputs, sondern ermöglichen auch gezielte Nachkorrekturen im Rahmen der qualitativen Auswertung und visuellen Darstellung der Daten.

5.3.10 `llm_enricher.py`

Das Modul `llm_enricher.py` dient der nachträglichen semantischen Anreicherung von JSON-Dokumenten mithilfe von GPT-4. Es wird gegen Ende des Verarbeitungsprozesses innerhalb von `transkribus_to_base.py` aufgerufen, nachdem die initiale Konvertierung der Transkribus-XML-Dateien abgeschlossen und das Entitätenmatching bereits erfolgt ist. Das Ziel besteht darin, fehlende Felder wie `author`, `recipient`, `creation_date` oder `content_tags_in_german` abzufangen und ggf. zu ergänzen.

Die Funktion `enrich_document_with_llm` übergibt das vollständige JSON-Dokument an das Sprachmodell. Dieses erhält einen Prompt mit präzisen Anweisungen zur Vervollständigung strukturierter Felder, inklusive Regeln z.B. Schutz von IDs wie der `Nodegoat_ID`, Kombination von Ortsnamen, Erkennung von Vereinsnamen als Organisationen. Der Rückgabewert wird mit dem Originaldokument zusammengeführt und unter `_enriched.json` gespeichert. Zusätzlich wird die LLM-Nutzung dokumentiert (`input_tokens`, `output_tokens`, `cost_usd`).

Abgrenzung zu `llm_xml_preprocessing.py`

Trotz des gemeinsamen Einsatzes eines Sprachmodells unterscheiden sich die beiden Module grundlegend in Funktion, Datenbasis und Zielsetzung. Während `llm_xml_preprocessing.py` im Frühstadium der Verarbeitung eingesetzt wird, um Textabschnitte vorzubereiten, Prompts zu formulieren oder Entitäten lokal zu erkennen, operiert `llm_enricher.py` am Ende des Workflows auf dokumentenweiter Ebene des neu generierten JSON. Die folgende Tabelle fasst die Unterschiede zusammen:

	<code>llm_xml_preprocessing.py</code>	<code>llm_enricher.py</code>
Ziel	Vorbereitung des LLM-Einsatzes: Segmentierung, Prompt-Generierung und lokale Entitätenerkennung	Nachträgliche Anreicherung strukturierter JSONs mit fehlenden Metadaten auf Dokumentebene
Fokus	Lokale NER in isolierten Transkriptzeilen oder Abschnitten	Globale Dokumentinterpretation, Rollen- und Feldzuordnung, Dublettenfilterung
Input	Textsegmente oder XML-Zeilen aus Transkribus	Strukturiertes JSON-Dokument nach algorithmischer Vorverarbeitung
Output	Markierte Entitäten in Form von XML-Tags oder Listen (z.B. Personen, Orte)	Vervollständigtes JSON mit algorithmisch unausgefüllten Feldern

Notwendigkeit beider Module trotz LLM-Einsatz

Auch wenn beide Module aktuell das selbe Sprachmodell verwenden, besteht keine redundante Dopplung. Sie erfüllen komplementäre Aufgaben in einer zweistufigen Pipeline: Zuerst erfolgt mit `llm_xml_preprocessing.py` die Erkennung und Markierung von Entitäten in isolierten Textblöcken zur Named-Entity-Recognition. Anschliessend übernimmt `llm_enricher.py` die Integration, Interpretation und strukturelle Vervollständigung dieser Einheiten im Gesamtkontext des Dokuments. Es handelt sich somit um ein hybrides Verfahren nach dem Prinzip: *Erkennen → Zuordnen → Strukturieren*.

Das Modul `llm_enricher.py` wird in `transkribus_to_base.py` optional aufgerufen, nachdem die regulären Verarbeitungsschritte abgeschlossen sind. Nur wenn bestimmte Kernfelder im JSON-Dokument fehlen, wird das Enrichment aktiviert. Dies verhindert unnötige LLM-Aufrufe und erhöht die Effizienz. Die Rückgabe wird mit dem Originaldokument vereinigt, bestehende IDs und Daten werden geschützt, neue Felder ergänzt. Damit wird eine nachträgliche Qualitätssteigerung des Outputs erzielt – bei vollständiger Nachvollziehbarkeit.

6 Projekt-Webseite

Die im Rahmen dieser Arbeit entwickelte Verarbeitungspipeline generiert eine Vielzahl heterogener Datenprodukte in unterschiedlichen Formaten und an verschiedenen Speicherorten. Während die stark normalisierten Ground-Truth-Daten zu Personen, Orten und Organisationen in einer Nodegoat-Instanz verwaltet werden, liegen die Resultate der automatischen Pipelineverarbeitung

als JSON-Dateien auf der lokalen Verarbeitungsumgebung vor. Die digitalisierten historischen Dokumente selbst werden auf der Transkribus-Plattform als Bilddateien archiviert. Diese verteilte Infrastruktur wirft zentrale Fragen auf: Wie lassen sich diese unterschiedlichen Datenspeicher konsistent zusammenführen? Wie so aufbereiten, dass sie für Forschung und Öffentlichkeit zugänglich sind? Und dabei so hinterlegen, dass sie den FAIR-Data-Prinzipien (Findable, Accessible, Interoperable, Reusable) genügen?

Von Projektbeginn an wurde der Anspruch formuliert, sämtliche extrahierten Daten nach diesem Standard bereitzustellen. Das zeigt sich in der [Pipeline](#) etwa durch die Anreicherung aller erkannten Named Entities mit Linked Open Data. Zur Erreichung dieses Ziels bei der Präsentation der Ergebnisse wurde eine webbasierte Präsentationsplattform konzipiert. Sie ermöglicht es, Daten aus den verschiedenen Quellsystemen zu aggregieren, gemeinsam darzustellen und mit den Digitalisaten der historischen Quellen zu verknüpfen. Nutzerinnen und Nutzer erhalten so über eine einzige Anwendung Zugriff auf sämtliche relevanten Inhalte – unabhängig davon, in welchem Primärsystem diese gespeichert sind.

Die technische Umsetzung basiert auf der Plattform **NDR Core**, einer Webumgebung zur Präsentation digitaler Projekte. Sie ist bei RISE¹³⁹ angesiedelt und ging ebenfalls aus einer Masterarbeit hervor.¹⁴⁰ NDR Core ermöglicht die sonst komplexe Zusammenführung und Präsentation von Daten aus unterschiedlichen Quellen an einem zentralen Ort. Über die Anbindung verschiedener Schnittstellen können mehrere Datenquellen parallel abgefragt und in einem kuratierten Such- und Anzeigeerlebnis zusammengeführt werden. Darüber hinaus bietet die Plattform grundlegende CMS-Funktionalitäten, wie die Einbindung von Texten, Bildern, IIIF-Manifest-Viewern sowie iFrames externer Plattformen wie der Visualisierungen im Public Interface von Nodegoat. Einen Überblick über den Aufbau der Projektseite bietet diese Grafik.

Neben NDR Core sind weitere Komponenten notwendig.

Zur Speicherung der Korpusdaten wird eine **MongoDB**-Datenbank eingesetzt¹⁴¹. Dabei handelt es sich um eine dokumentenorientierte NoSQL-Datenbank. Im Gegensatz zu klassischen relationalen SQL-Systemen wie MySQL oder PostgreSQL erfordert dies kein starres, vordefiniertes Datenmodell. Stattdessen können Kollektionen gleichartiger Dokumente flexibel gespeichert und abgefragt werden, was eine hohe Anpassungsfähigkeit an die heterogene Struktur der Quell- und Metadaten ermöglicht. Diese Flexibilität geht jedoch mit gewissen Herausforderungen für Linked-Data-Ansätze einher. Die fehlende strikte Schemabindung erschwert die eindeutige semantische Verknüpfung von Datensätzen. Um sowohl die Vorteile der flexiblen Datenhaltung zu nutzen, als auch eine hochgradige Normalisierung und eindeutige Referenzierung sicherzustellen,

¹³⁹vgl. Decker 2025.

¹⁴⁰vgl. Marti 2024.

¹⁴¹vgl. *MongoDB* 2025, <https://www.mongodb.com/de-de>.

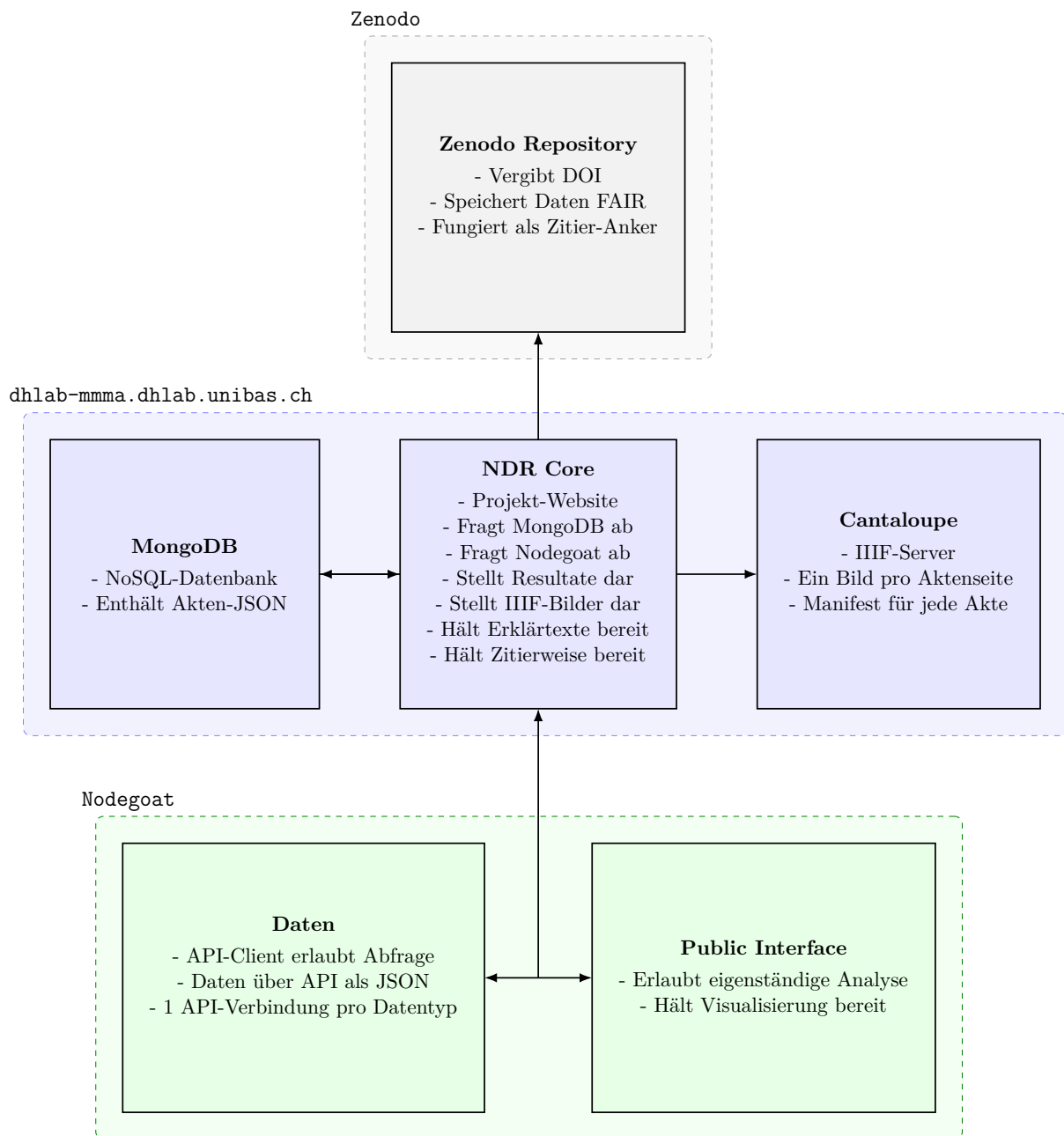


Abbildung 15: Übersicht aller Verarbeitungsschritte

len, wird in dieser Arbeit eine Mischform aus MongoDB und der strukturierten Datenhaltung in Nodegoat eingesetzt.

Für die Bildbereitstellung der historischen Quellen wird der IIIF-kompatible **Cantaloupe**-Server¹⁴² verwendet. Cantaloupe ist eine in Java implementierte Open-Source-Lösung, die aufgrund bestehender Projekterfahrungen ausgewählt wurde.

Nodegoat übernimmt in dieser Architektur die Modellierung und Verwaltung der entitätsbasierten NER-Daten einschliesslich der externen Verknüpfungen zu Normdatenquellen wie *Wikidata*, *Geonames* oder *GND*.

¹⁴²<https://cantaloupe-project.github.io/>

In der Zusammenschau ergibt sich folgende Rollenverteilung. Die MongoDB enthält die Rohtranskriptionen und Metadaten der Quelldokumente, Nodegoat die strukturierten Entitätsdaten mit Normdatenverknüpfungen, und Transkribus stellt die Bilddaten bereit.

Damit gelingt es gleichzeitig, alle vier Anforderungen des zugrundeliegenden **FAIR**-Standards mit nur einer Anwendung zu erfüllen. Alle drei Datenströme werden in der Webanwendung vereint und auf Zenodo als **FAIR**-konformer Datensatz veröffentlicht. Das gewährleistet insbesondere die Auffindbarkeit durch gängige Suchmaschinen (**F**indable). Da sowohl über Suchergebnisse auf der Website als auch der zugrundeliegenden Gesamtdatensatz via GitHub und Zenodo zugänglich ist, wird auch das Kriterium **A**ccessible erfüllt. Die Interoperabilität (**I**nteroperable) wird durch standardisierte Referenzen zu Normdaten sichergestellt und die Wiederverwendbarkeit (**R**eusable) durch den Download vollständiger, mit Autoritätsdaten angereicherter Datensätze unterstützt.

7 Fazit und Ausblick

Diese Pipeline entwickelt mit der Verwendung von Large Language Models eine neue Art der Verarbeitung historischer Dokumente. Neben der methodischen Innovation konnte die Auswertung des Männerchor-Murg-Korpus neue Einblicke in Vereinsstrukturen, regionale und überregionale Netzwerke sowie die Einbindung des Vereins in den kulturellen Kontext der NS-Zeit gewinnen. Die Kombination von klassischer Archivarbeit mit einer modular aufgebauten digitalen Pipeline hat sich als fruchtbar erwiesen, um sowohl mikrohistorische Zusammenhänge als auch grossräumigere Beziehungsgeflechte sichtbar zu machen. Diese Ergebnisse können dabei von den Usern eigenständig betrachtet und individuell generiert werden, ohne dass sie vorher für unterschiedliche Anwendungszwecke modelliert werden müssen. Hinzu kommt, dass alle Daten **FAIR** bereitstehen, und jederzeit weitergenutzt werden zu können.

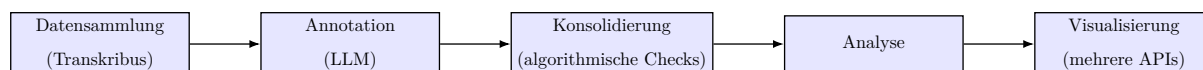


Abbildung 16: Workflow

Der entwickelte Workflow zeichnet sich durch eine modulare Struktur und hohe Wiederverwendbarkeit aus. Die Verarbeitungsschritte der Pipeline sind so gestaltet, dass sie sich mit geringen Anpassungen, auf andere geisteswissenschaftliche Korpora übertragen lassen.¹⁴³ Dies betrifft sowohl eine Fortsetzung der Analyse innerhalb des Männerchor-Bestands als auch die Anwendung auf andere historische Quellenbestände. Darüber hinaus bildet die vorliegende Arbeit eine fundierte digital-geisteswissenschaftliche Grundlage für weiterführende historische Untersuchungen.

¹⁴³etwa im Bereich der regulären Ausdrücke und der projektspezifischen Ground-Truth-Daten.

Die mit Hilfe der Pipeline rekonstruierten Netzwerke aus der Zeit des Nationalsozialismus eröffnen neue Perspektiven für diachrone Analysen. Die Selektion der Quellen basiert auf einem Korpus, der bis ins Jahr 1925 zurückreicht. Dadurch kann der nun verarbeitete Korpus so erweitert werden, dass sich Entwicklungen in den vereinsinternen Strukturen sowie im weiteren sozialen Umfeld auch für die Vorkriegszeit systematisch nachvollziehbar werden.

Ein weiterer methodischer Mehrwert besteht darin, dass die mit LLMs annotierten PageXML-Dateien sich an den Stil von Transkribus halten. Der Prompt baut auf den dort etablierten Custom-Tags auf. So können die durch das LLM annotierten PageXML-Daten ohne Hindernisse wieder in Transkribus importiert werden. Das ermöglicht es, dort sämtliche automatisch gesetzten Tags zu visualisieren, wie es in [Tags der Transkription von Abbildung 5](#) dargestellt wird. So können sie einerseits manuell geprüft und korrigiert werden, andererseits auch über das Webinterface eingesehen werden. Darüber hinaus erlauben die durch Transkribus erzeugten Koordinaten für Textlinien, in Kombination mit den Tags jede erkannte Entität präzise auf Zeilenebene zu verorten. Dies schafft eine Grundlage für hochauflösende, positionsgenaue Visualisierungen, die aus Zeitgründen für dieses Projekt bislang nicht implementiert wurden.

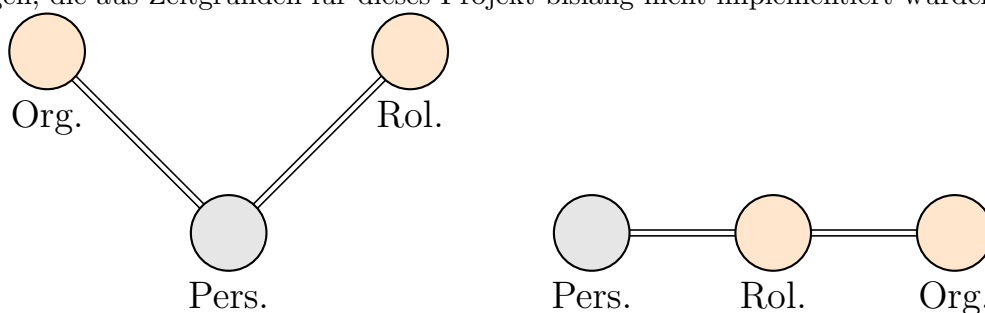


Abbildung 17: Stand des Rollenmodells (links); Ziel (rechts)

Für künftige Arbeiten bietet sich insbesondere eine erweiterte Einbindung der Ergebnisse in Nodegoat an, um komplexere Visualisierungsformen zu ermöglichen. Denkbar ist etwa die Darstellung des Rollenmodells als lineare Verknüpfung zwischen Akteuren. Die aktuelle Form erinnert eher an ein Wasser-Molekül und bildet damit vor allem die Zugehörigkeit zu einem Objekt ab. Eine lineare Verbindung wie in [Abbildung 17](#) könnte dagegen hierarchische oder funktionale Beziehungen deutlich präziser sichtbar machen.

Mit diesem methodischen Fundament zeigt die Arbeit, wie sich durch die Verbindung klassischer Quellenkritik mit innovativen digitalen Verfahren neue Erkenntnisse über historische Akteursnetzwerke gewinnen lassen. Sie legt damit den Grundstein für eine anschlussfähige und zukunftsorientierte Forschungspraxis.

8 Redlichkeit

Diese Arbeit, einschliesslich der beschriebenen Softwarepakete und ihrer Dokumentation, wurde vollständig von mir erstellt. Dennoch waren KI-Werkzeuge ein integraler Bestandteil der Softwareentwicklung und wurden unter anderem zur grammatikalischen und stilistischen Überprüfung dieses Textes eingesetzt. GitHub Copilot und ChatGPT kamen bei der Entwicklung mehrerer Pipeline-Module zum Einsatz, darunter insbesondere `transkribus_to_base.py` und die jeweiligen Module. Copilot unterstützt beim automatischen Vervollständigen von Code und kann etwa offensichtliche Ergänzungen oder methodenbezogene Dokumentationen generieren. ChatGPT hilft auf Fehler hinzuweisen, und liefert unterschiedliche Herangehensweisen. Für eine produktive Nutzung ist jedoch ein fundiertes Verständnis der zugrunde liegenden Entwicklungsprozesse erforderlich; solche Werkzeuge ersetzen nicht die konzeptionelle oder programmatische Arbeit.

Rechtschreibprüfungen und Übersetzungen wurden durch mehrere Systeme durchgeführt, sowohl von der integrierten Rechtschreibprüfung auf [Overleaf](#) als auch von ChatGPT und DeepL. Die genannten Werkzeuge dienten lediglich dazu, offensichtliche sprachliche Unstimmigkeiten zu erkennen – vergleichbar mit der Rechtschreibprüfung in Programmen wie MS Word.

Einzelne inhaltliche und konzeptionelle Beiträge stammen von Sorin Marti . Er unterstützte bei dem Aufsetzen der Projektwebsite und der dort zugrundeliegenden Infrastruktur, wie NDR Core. Dies ist eine von ihm entwickelte Software. Hierfür musste das `total_json.json` für die Website angepasst werden - auch dies wurde durch Sorin Marti ausgeführt.

Es sei darauf hingewiesen, dass streng genommen nicht einmal die Nutzung einer modernen Suchmaschine ohne KI-Technologien erfolgt. Ein Arbeiten gänzlich ohne KI-unterstützte Infrastruktur ist in der heutigen Forschungsumgebung praktisch nicht möglich. Dies vorausgeschickt – vorliegende Arbeit ist das Werk eines Menschen.



Universität
Basel

Philosophisch-Historische
Fakultät



Erklärung zur wissenschaftlichen Redlichkeit

Ich bestätige hiermit, dass ich vertraut bin mit den Regelungen zum Plagiat der «Ordnung der Philosophisch-Historischen Fakultät der Universität Basel für das Bachelorstudium vom 25. Oktober 2018» (§21) bzw. der «Ordnung für das Masterstudium vom 25. Oktober 2018» (§25) und die Regeln der wissenschaftlichen Integrität gewissenhaft befolgt habe. Die vorliegende Arbeit ist ausserdem weder ganz noch teilweise an einer anderen Fakultät oder Universität zur Begutachtung eingereicht und/oder als Studienleistung, z.B. in Form von Kreditpunkten verbucht worden.

Des Weiteren versichere ich, sämtliche Textpassagen, die unter Zuhilfenahme KI-gestützter Programme verfasst wurden, entsprechend gekennzeichnet sowie mit einem Hinweis auf das verwendete KI-gestützte Programm versehen zu haben.

Eine Überprüfung der Arbeit auf Plagiate und KI-gestützte Programme – unter Einsatz entsprechender Software – darf vorgenommen werden.

Bei begründetem Verdacht auf eine unerlaubte oder nicht gekennzeichnete Anwendung von KI verpflichte ich mich, an der Klärung des Sachverhaltes mitzuwirken, z.B. durch Teilnahme an einem Gespräch.

Ich habe zur Kenntnis genommen, dass unlauteres Verhalten zu einer Bewertung der betroffenen Arbeit mit einer Note 1 oder mit «nicht bestanden» bzw. «fail» oder zum Ausschluss vom Studium führen kann.

Name, Vorname: Burkhardt, Sven

Titel der schriftlichen Arbeit:

Digitale Harmonie aus historischer Dissonanz

Extraktion, Ordnung und Analyse unstrukturierter Archivdaten des Männerchors Murg

Datum: 14.08.2025

Unterschrift:

Bibliographie

Artikel

Beck, Maximilian u. a. (2020). „A review on the long short-term memory model. Artificial intelligence review“. In: *Artificial intelligence review* 53.8, S. ff. 5929–5955 (zit. auf S. 24).

Capurro, Carlotta, Vera Provatorova und Evangelos Kanoulas (29. Nov. 2023). „Experimenting with Training a Neural Network in Transkribus to Recognise Text in a Multilingual and Multi-Authored Manuscript Collection“. In: *Heritage* 6.12, S. ff. 7482–7494. ISSN: 2571-9408. DOI: [10.3390/heritage6120392](https://doi.org/10.3390/heritage6120392). URL: <https://www.mdpi.com/2571-9408/6/12/392> (zit. auf S. 27).

Martinez, Roxana und Gonzalo Pereyra Metnik (o.D.). „Comparative Study of Tools for the Integration of Linked Open Data: Case study with Wikidata Proposal“. In: () (zit. auf S. 20).

Wilkinson, Mark D. u. a. (15. März 2016). „The FAIR Guiding Principles for scientific data management and stewardship“. In: *Scientific Data* 3.1. Publisher: Nature Publishing Group, S. 160018. ISSN: 2052-4463. DOI: [10.1038/sdata.2016.18](https://doi.org/10.1038/sdata.2016.18). URL: <https://www.nature.com/articles/sdata201618> (zit. auf S. 20).

Monografien

Buchner, Alex (1989). *Das Handbuch der Deutschen Infanterie 1939 – 1945*. 2. Aufl. Friedberg: Podzun-Pallas, 1989. ISBN: 3-7909-0301-9 (zit. auf S. 6).

Gamper, Markus und Linda Reschke (27. Apr. 2015). *Knoten und Kanten III: Soziale Netzwerkanalyse in Geschichts- und Politikforschung*. Hrsg. von Martin Düring. transcript Verlag, 27. Apr. 2015. ISBN: 978-3-8394-2742-2. DOI: [10.1515/9783839427422](https://doi.org/10.1515/9783839427422). URL: <https://www.degruyter.com/document/doi/10.1515/9783839427422/html> (zit. auf S. 7).

Hartmann, Christian (2010). *Wehrmacht im Ostkrieg - Front und militärisches Hinterland 1941/42*. 2. Auflage. Bd. 75. Quellen und Darstellungen zur Zeitgeschichte Herausgegeben vom Institut für Zeitgeschichte. München: R. Oldenbourg Verlag, 2010 (zit. auf S. 6).

Haupt, Werner (1982). *Das Buch der Infanterie*. 1. Aufl. Friedberg, Hanau: Podzun-Pallas, 1982. ISBN: 3-7909-0176-8 (zit. auf S. 6 f.).

Rass, Christoph und René Rohrkamp (2009). *Deutsche Soldaten 1939-1945 Handbuch einer biographischen Datenbank zu Mannschaften und Unteroffizieren von Heer, Luftwaffe und Waffen-SS*. Aachen, 2009 (zit. auf S. 7).

Richard, Smiraglia und Scharnhorst Andrea (3. Mai 2022). *Linking Knowledge. Linked Open Data for Knowledge Organization and Visualization*. Version Number: editorsversion, prior to publication. Zenodo, 3. Mai 2022. DOI: [10.5771/9783956506611](https://doi.org/10.5771/9783956506611). URL: <https://zenodo.org/records/6513663> (zit. auf S. 7).

Tessin, Georg (1977). *Verbände und Truppen der deutschen Wehrmacht und Waffen-SS im Zweiten Weltkrieg 1939-1945*. Bd. Band 1 - Die Waffengattungen — Gesamtübersicht. Osnabrück: HIBLIO Verlag, 1977 (zit. auf S. 7).

Thompson, Edward P. und Edward Palmer Thompson (1968). *The making of the English working class*. Pelican Books A 1000. Harmondsworth: Penguin Books, 1968 (zit. auf S. 2).

Zentner, Christian (1983). *Illustrierte GEschichte des Zweiten Weltkriegs*. München: Südwest Verlag GmbH, 1983 (zit. auf S. 7).

Onlinequellen

Altenburger, Andreas (2023). *Lexikon der Wehrmacht*. (Zugriff am 15.01.2023). URL: <https://www.lexikon-der-wehrmacht.de/Gliederungen/Infanteriedivisionen/205ID.htm> (zit. auf S. 7, 20).

Burkhardt, Sven (12. Dez. 2022). *Feldpost an den Männerchor Murg - Storymaps*. ArcGIS StoryMaps. (Zugriff am 12.03.2025). URL: <https://storymaps.arcgis.com> (zit. auf S. 5).

Claude Code (2025). Anthropic. (Zugriff am 22.07.2025). URL: <https://docs.anthropic.com/en/release-notes/claude-code> (zit. auf S. 29).

Decker, Eric (2025). *Home / RISE / Research & Infrastructure Support / Universität Basel*. Research & Infrastructure Support. (Zugriff am 06.07.2025). URL: <https://rise.unibas.ch/de/> (zit. auf S. 76).

DRK Suchdienst / Suche per Feldpostnummer (2025). DRK Suchdienst. Unter Mitarb. von Christian Reuter. (Zugriff am 12.03.2025). URL: <https://vbl.drk-suchdienst.online/Feldpostnummer/FPN.aspx> (zit. auf S. 7, 20).

Feldpost Number Database / GermanStamps.net (2025). (Zugriff am 09.07.2025). URL: <https://www.germanstamps.net/feldpost-number-database/> (zit. auf S. 7).

Forum Geschichte der Wehrmacht (2025). Forum Geschichte der Wehrmacht. Unter Mitarb. von Dieter Hermans. (Zugriff am 12.03.2025). URL: <https://www.forum-der-wehrmacht.de/> (zit. auf S. 7).

- Gemini – unser größtes und leistungsfähigstes KI-Modell* (6. Dez. 2023). Google. (Zugriff am 22. 07. 2025). URL: <https://blog.google/intl/de-de/unternehmen/technologie/gemini/> (zit. auf S. 29).
- GeoNames* (2025). (Zugriff am 05. 07. 2025). URL: <https://www.geonames.org/> (zit. auf S. 21).
- Geschichte Gemeinde Murg* (2025). Unter Mitarb. von Gemeinde Murg. (Zugriff am 29. 06. 2025). URL: <https://www.murg.de/seite/33378/geschichte.html> (zit. auf S. 5).
- Hollmann, Prof. Dr. Michael (2025). *Freiburg*. Bundesarchiv Freiburg im Breisgau (Abteilung Militärarchiv). (Zugriff am 12. 03. 2025). URL: <https://www.bundesarchiv.de/das-bundesarchiv/standorte/freiburg/> (zit. auf S. 7).
- Model Release Notes* (2025). OpenAI Help Center. (Zugriff am 22. 07. 2025). URL: <https://help.openai.com/en/articles/9624314-model-release-notes> (zit. auf S. 31).
- MongoDB* (2025). *MongoDB: Die weltweit führende moderne Datenbanklösung*. MongoDB. (Zugriff am 11. 08. 2025). URL: <https://www.mongodb.com/> (zit. auf S. 76).
- Msty - Using AI Models made Simple and Easy* (2025). (Zugriff am 06. 07. 2025). URL: <https://msty.app/> (zit. auf S. 28 f.).
- OpenAI (7. Aug. 2025). *Introducing GPT-5*. (Zugriff am 10. 08. 2025). URL: <https://openai.com/index/introducing-gpt-5/> (zit. auf S. 32).
- OWL Guide* (10. Feb. 2004). *OWL Web Ontology Language Guide*. Unter Mitarb. von Michael K. Smith, Chris Welty und Deborah L. McGuinness. (Zugriff am 05. 07. 2025). URL: <https://www.w3.org/TR/owl-guide/> (zit. auf S. 18).
- Recognition and Enrichment of Archival Documents | READ | Projekt | Fact Sheet | H2020* (2025). CORDIS | European Commission. (Zugriff am 06. 07. 2025). URL: <https://cordis.europa.eu/project/id/674943> (zit. auf S. 26).
- Thomson, Martin, Erik Wilde und Adam Roach (7. Juni 2017). *Geographic JSON (geojson)*. (Zugriff am 11. 07. 2025). URL: <https://datatracker.ietf.org/wg/geojson/charter/> (zit. auf S. 23).
- Transkribus* (Nov. 2024). *Transkribus_Model_mmma*. Unter Mitarb. von Sven Burkhardt. (Zugriff am 25. 06. 2025). URL: <https://app.transkribus.eu> (zit. auf S. 14).
- WGS84 | Landesamt für Geoinformation und Landesvermessung Niedersachsen* (2025). Landesamt für Geoinformation und Landesvermessung Niedersachsen. (Zugriff am

05.07.2025). URL: https://www.lgln.niedersachsen.de/startseite/wir_uber_uns/hilfe_support/lgln_lexikon/w/wgs84-190576.html (zit. auf S. 21).

Wikidata (2025). (Zugriff am 05.07.2025). URL: https://www.wikidata.org/wiki/Wikidata:Main_Page (zit. auf S. 20).

Software

Brown, Tom B. u. a. (2020). *Language Models are Few-Shot Learners*. eprint: 2005.14165. 2020. URL: <https://arxiv.org/abs/2005.14165> (zit. auf S. 34).

Burkhardt, Sven (23. Apr. 2025b). *github/PDF_to_JPEG.py*. Version 1.0. Basel, 23. Apr. 2025. URL: https://github.com/Sveburk/masterarbeit/blob/main/3_MA_Project/Hilfs_Scripte/JPEG_to_PDF.py (zit. auf S. 12, 25).

Wang, Julian Junyan und Victor Xiaoqi Wang (17. Juni 2025). *Assessing Consistency and Reproducibility in the Outputs of Large Language Models: Evidence Across Diverse Finance and Accounting Tasks*. 17. Juni 2025. DOI: [10.48550/arXiv.2503.16974](https://doi.org/10.48550/arXiv.2503.16974). arXiv: 2503.16974[q-fin]. URL: <http://arxiv.org/abs/2503.16974> (zit. auf S. 33).

Vorträge und Manuskripte

Burkhardt, Sven (17. Juni 2025a). „Bias is not a Bug: Towards Methodological Awareness in AI-Assisted Humanities“. DH-CH Conference 2025. Rom, 17. Juni 2025. URL: <https://dh-ch.ch/dhch-isr/25/presentations.html> (zit. auf S. 50, 53).

Durst, Emil (14. Feb. 1948). „Sklaven des zwanzigsten Jahrhunderts - Ein Tatsachenbericht.“ historische Quelle, Manuskript, Einzelexemplar, historische Quelle, Manuskript, Einzelexemplar, Murg, 14. Feb. 1948 (zit. auf S. 31).

Hodel, Tobias (2019). „Machine Learning in Digital History: Texterkennung und Dokumentenanalyse mit Transkribus“. Universitäre Übung. Universität Basel, 2019. URL: <https://vorlesungsverzeichnis.unibas.ch/de/vorlesungsverzeichnis?id=243354> (zit. auf S. 5).

Mühlberger, Günter (25. Apr. 2019). „Transkribus Eine Forschungsplattform für die automatisierte Digitalisierung, Erkennung und Suche in historischen Dokumenten“. Kolloquium der ETH-Bibliothek. Zürich, 25. Apr. 2019. URL: https://ethz.ch/content/dam/ethz/associates/ethlibrary-dam/documents/Aktuell/Veranstaltungen/17-15-Kolloquium/2019-04-29_17-15-Kolloquium_transkribus.pdf (zit. auf S. 26).

Serif, Ina (2019). „Digital History: computergestützte Anwendungs- und Forschungsmöglichkeiten in den Geschichtswissenschaften“. Universitäre Übung. Universität Basel, 2019. URL: <https://vorlesungsverzeichnis.unibas.ch/de/vorlesungsverzeichnis?id=243345> (zit. auf S. 5).

Serif, Ina (2022). „Von Bücherschätzen zu Datensätzen. Digitalisierung, Aufbereitung und Auswertung historischer Quellen“. Universitäre Übung. Universitäre Übung. Universität Basel, 2022. URL: <https://vorlesungsverzeichnis.unibas.ch/de/vorlesungsverzeichnis?id=269454> (zit. auf S. 5).

Abbildungsverzeichnis

1	Dokumententypen im untersuchten Bestand (150 Akten – 381 Seiten).	9
2	Geschätzte Verteilung der Dokumententypen im restlichen Bestand.	10
3	Beispiel für handschriftlichen Text in Akte_076	13
4	Transkription durch <i>The German Giant I</i> -Modell der Abbildung 3	14
5	Durch ChatGPT unterstützte Transkription von Abbildung 4.	14
6	Tags der Transkription von Abbildung 5.	16
7	Ausschnitt der TTL-Ontologie.	19
8	Übersicht aller Verarbeitungsschritte	34
9	Übersicht der gesamten XML-to-JSON-Pipeline	35
10	Oben links: Prozessdiagramm für <code>Personen_matcher.py</code> , Oben rechts: Pipelineübersicht	45
11	Links: Prozessdiagramm für <code>Assigned_Roles_Module.py</code> , Rechts: Pipelineübersicht	56
12	Links: Prozessdiagramm für <code>place_matcher.py</code> , Rechts: Pipelineübersicht	58
13	Links: Prozessdiagramm für <code>organization_matcher.py</code> , Rechts: Pipelineübersicht	61
14	Oben links: Prozessdiagramm für <code>unmatched_logger.py</code> , Oben rechts: Pipelineübersicht	72
15	Übersicht aller Verarbeitungsschritte	77
16	Workflow	78
17	Stand des Rollenmodels (links); Ziel (rechts)	79

A Anhang

A.1 PDF_to_JPEG.py

```
1 import os
2 import fitz # PyMuPDF
3
4 def convert_pdf_to_jpg(src_folder, dest_folder):
5     # Überprüfen, ob der Zielordner existiert, und ihn ggf. erstellen
6     if not os.path.exists(dest_folder):
7         os.makedirs(dest_folder)
8
9     # Durchgehen durch alle Dateien im Quellordner
10    for root, dirs, files in os.walk(src_folder):
11        for file in files:
12            # Überprüfen, ob die Datei eine PDF-Datei ist
13            if file.lower().endswith(".pdf"):
14                # Vollständigen Pfad zur PDF-Datei erstellen
15                pdf_path = os.path.join(root, file)
16                # PDF-Datei öffnen
17                doc = fitz.open(pdf_path)
18                # Durch alle Seiten der PDF-Datei gehen
19                for page_num in range(len(doc)):
20                    page = doc[page_num]
21                    # Seite in ein QPixmap-Objekt umwandeln (für die Konvertierung in
22                    ↪ JPG)
23                    pix = page.get_pixmap()
24                    # Dateinamen ohne Dateiendung extrahieren
25                    filename_without_extension = os.path.splitext(file)[0]
26                    # Ausgabedateinamen erstellen mit führenden Nullen für die
27                    # Seitennummer
28                    output_filename = f"{filename_without_extension}_S{page_num +
29                    ↪ 1:03d}.jpg"
30
31                    # Vollständigen Pfad zur Ausgabedatei erstellen
32                    output_path = os.path.join(dest_folder, output_filename)
33                    # Bild speichern
34                    pix.save(output_path)
35                    # PDF-Datei schliessen
36                    doc.close()
37
38                # Erfolgsmeldung ausgeben
39                print(f"{file} wurde erfolgreich umgewandelt und gespeichert
40                in {dest_folder}")
41
42 # Pfade zu den Ordnern mit den PDF-Dateien (Quelle) und den JPG-Dateien (Ziel)
43 src_folder = r"/Users/svenburkhardt/Documents/D_Murger_Männer_Chor_Forschung/Scan_Mä_
44 ↪ nnerchor/Männerchor_Akten_1925-1945/Scan_Männerchor_PDF"
```

```

43 dest_folder = r"/Users/svenburkhardt/Documents/D_Murger_Männer_Chor_Forschung/Master_J
   ↳ arbeit/JPEG_Akten_Scans"
44
45
46 # Funktion aufrufen, um die Konvertierung durchzuführen
47 convert_pdf_to_jpg(src_folder, dest_folder)
48

```

A.2 Tagging in Transkribus

Transkribus und seine Modelle unterstützen nicht nur beim Transkribieren der Texte, sondern erlauben auch das Taggen von *Named Entities*. Für die vorliegende Arbeit sind dabei besonders Personen, Orte, Organisationen und Daten relevant. Um hierfür ein stringentes Verfahren zu entwickeln, wurden die Tags wie folgt definiert:

A.2.1 Strukturelle Tags

abbrev

Mit dem Tag **abbrev** werden alle Abkürzungen getaggt, die für eine eindeutige Entität stehen.

☞ **Beispiel 1:** Dr., Prof., St., Hr., Frl., Dipl.-Ing., etc.

☞ **Beispiel 2:** Organisationskürzel, wenn sie eindeutig sind:

```
<abbrev> V.D.A. </abbrev> .
```

☞ **Beispiel 3:** Falls eine dazugehörige Entität vorhanden ist, wird die Abkürzung getaggt und wird gleichzeitig als zugehörige Entität getaggt:

```
<person> <abbrev> Dr. </abbrev> Weiss </person>
```

unclear

Mit dem Tag **unclear** werden unleserliche oder schwer entzifferbare Textstellen markiert.

☞ **Beispiel 1:** Unklare Zeichen oder fehlende Buchstaben:

```
Er wohnte in <unclear> [...] <unclear> .
```

☞ **Beispiel 2:** Teilweise lesbare Wörter:

```
<place> Frei <unclear> [...] <unclear> <place> .
```

sic

Mit dem Tag `sic` werden Wörter markiert, die im Originaltext in einer falschen oder ungewöhnlichen Schreibweise geschrieben wurden.

☞ Beispiel 1: Veraltete oder falsche Schreibweisen:

```
<sic> daß </sic> für dass mit tz.
```

☞ Beispiel 2: Offensichtliche Tippfehler, wenn sie im Originaltext so vorkommen:

```
Wir haben <sic> einen </sic> grosse Freude.
```

☞ Beispiel 3: Falls eine Korrektur notwendig ist, kann sie als Kommentar ergänzt werden.

A.2.2 Inhaltliche Tags

person

Mit dem Tag `person` sind alle Strings getaggt, die eine direkte Zuordnung einer Person ermöglichen.

☞ **Beispiel 1:** Vereinsführer, Alfons, Zimmermann, Alfons Zimmermann, Z. A. Zimmermann, Herr Zimmermann, Herr Alfons Zimmermann, etc.

☞ **Beispiel 2:** Funktionen wie Oberlehrer, Chorleiter, etc. Wenn Ort, Name oder Organisation bekannt sind. Eine Person kann sowohl mit ihrem Namen als auch ihrer Funktion (wie Dirigent) getaggt werden. Aus der Korrespondenz ist in der Regel eine zugehörige Organisation ersichtlich, mit deren Verknüpfung eine namentlich nicht genannte Person identifiziert werden könnte.

signature

Mit dem Tag `signature` werden alle Strings getaggt, die eine handschriftliche Unterschrift darstellen. Der Tag `signature` ist nahezu deckungsgleich mit dem Tag `person`. Er dient zur **graduellen Unterscheidung**, ob ein Name im Fliesstext als gesichert leserlich oder handschriftlich als Signatur vorliegt.

☞ **Beispiel 1:** Eindeutig lesbare Signaturen werden direkt getaggt:

```
<signature> A. Zimmermann </signature>.
```

☞ **Beispiel 2:** Teilweise unleserliche Signaturen werden mit dem Tag `unclear` innerhalb von `signature` markiert:

```
<signature> R. We <unclear> [...] </unclear> </signature>.
```

☞ **Beispiel 3:** Wenn nur ein Teil des Namens lesbar ist, aber eine Identifikation unsicher bleibt, sollte die Unterschrift vollständig im Tag `unclear` innerhalb von `signature` stehen:

```
<signature> <unclear> etwas unleserliches </unclear> </signature> .
```

☞ **Beispiel 4:** Wenn eine Signatur einer bekannten Person zugeordnet werden kann, aber nicht vollständig lesbar ist, bleibt die Signatur erhalten und wird **ohne** den Tag `person` zu verwenden:

```
<signature> A. Zimm <unclear> [...] </unclear> </signature> .
```

☞ **Beispiel 5:** Wenn eine Unterschrift vollständig transkribiert wurde und die Person bekannt ist, wird sie nur mit `signature` getaggt, **ohne** den Tag `person` zu verwenden:

```
<signature> Alfons Zimmermann </signature> .
```

organization

Mit dem Tag `organization` werden alle Strings getaggt, die eine direkte Zuordnung einer Organisation ermöglichen.

☞ **Beispiel 1:** Männerchor Murg, Verein Deutscher Arbeiter (V.D.A.), Murgtalschule, etc.

☞ **Beispiel 2:** Abkürzungen, wenn sie eine Organisation eindeutig bezeichnen, z.B. V.D.A., NSDAP, STAGMA, etc.

place

Mit dem Tag `place` werden alle Strings getaggt, die sich auf einen geografischen Ort beziehen.

☞ **Beispiel 1:** Murg (Baden), Freiburg, Berlin, Murgtal, Schwarzwald, etc.

☞ **Beispiel 2:** Orte mit näherer Bestimmung, z.B. „bei Berlin“, „im Murgtal“ werden getaggt:

```
<place> im Murgtal</place> .
```

date

Mit dem Tag `date` werden alle expliziten und implizierten Datumsangaben markiert.

☞ **Beispiel 1:** 29.05.1936

☞ Beispiel 2: 29. Mai 1936

☞ Beispiel 3: den 29. d. Mts.:

```
<date when="29.05.1936"> den 2. <abbrev> d. Mts. </abbrev> </date>
```

event

Mit dem Tag **event** werden explizite und implizierte Ereignisse markiert. Diese Ereignisse haben einen zeitlichen oder räumlichen Bezug, und können benannt werden. Dazu zählen:

☞ Beispiel 1: „Jubiläumskonzert“

☞ Beispiel 2 „Gründung des Vereins“

☞ Beispiel 2 „Kriegsausbruch“ oder „Kriegsende“

Konzepte, die nicht klar in den Texten benannt werden, können nicht immer Ereignis getaggt werden. Exemplarisch soll die Suche nach einem neuen Dirigenten erwähnt sein. Sie sollen später aber in der Datenbank implementiert werden.

A.3 Prompt der LLM Vorverarbeitung

```
1  prompt = f"""
2  Systemrolle:
3  Du bist ein spezialisiertes XML-Annotationstool für historische Transkribus-Dokumente.
4
5  Aufgabe:
6  Analysiere das gesamte PAGE-XML-Dokument. Extrahiere Entitäten aus dem Unicode-Text aller <TextLine>-Elemente und füge strukturierte
  ↳ `custom="..."`-Attribute hinzu.
7
8  Strikte Regeln:
9  1. Dokumentanalyse:
10 - Verarbeite ausschließlich <TextLine>-Elemente.
11 - Verwende nur <Unicode>-Inhalte als Eingabetext.
12
13  2. Globale Personenerkennung:
14 - Erkenne Personen (inkl. Titel, Vorname, Nachname).
15 - Speichere `offset` und `length` für jede erkannte Person pro TextLine.
16 - Verwende **immer dieselben Offsets** bei wiederholten Nennungen im Dokument.
17
18  3. Empfängererkennung (`recipient`):
19 - Der Kopfbereich endet an der ersten komplett leeren TextLine.
20 - Erkenne dort Anreden (z.B. „Herr“, „Frau“, „Sehr geehrter Herr ...“).
21 - Verknüpfe Anrede mit passender Person und annotiere mit:
22 `recipient {{offset:X; length:Y;}}`
23
24  4. Autorenkennung (`author`):
25 - Der Fußbereich beginnt nach der letzten Grußformel (z.B. „Mit freundlichen Grüßen“).
26 - Namen → `author {{offset:X; length:Y;}}`.
27 - Funktion (z.B. „Chorleiter“) → `role {{offset:X; length:Y;}}`.
28
29  5. Ort- und Datumsannotation:
30 - **Absendeort** (creation_place) und **Erstellungsdatum** (creation_date):
31   zusätzlich zu den Tags place und date hinzu:
32   creation_place {{offset:X; length:Y;}} und creation_date {{offset:X; length:Y; when:TT.MM.JJJJ;}}.
33 - **Empfangsort** (recipient_place):
34   Füge im Empfänger-Block die passende Zeile mit:
35   place {{offset:X; length:Y;}}.
36
37  6. Entitäten pro Zeile (in dieser Reihenfolge):
38  Füge **ein** Attribut `custom="..."` ein mit nur den tatsächlich erkannten Entitäten:
39
40
41  person {{offset:X; length:Y;}}
```

```

42     recipient {{offset:X; length:Y;}}
43     author {{offset:X; length:Y;}}
44     organization {{offset:X; length:Y;}}
45     place {{offset:X; length:Y;}}
46     date {{offset:X; length:Y; when:TT.MM.JJJJ;}}
47     role {{offset:X; length:Y;}}
48     event {{offset:X; length:Y;}} → optional mit when:TT.MM.JJJJ;
49
50 Hinweise:
51 - Füge nur tatsächlich vorhandene Entitäten ein.
52 - Keine Platzhalter.
53 - Format für `date` und `event` (falls Datum erkennbar): `when:TT.MM.JJJJ`;
54 - Mehrzeilige Events (z.B. bei Bindestrich am Ende oder fortgeführtem Satz) erhalten dieselbe `event`-Annotation in allen betroffenen
    ↪ Zeilen.
55
56 6. XML-Regeln:
57 - Verändere nur `custom`-Attribute innerhalb von `` .
58 - Belasse alle anderen XML-Strukturen vollständig unverändert.
59 - Sind die XML Files leer, lasse sie unverändert leer.
60
61 7. Ausgabe:
62 - Gib ausschließlich ein vollständiges, wohlgeformtes XML zurück.
63 - Kein Freitext, kein Kommentar, kein Markdown.
64
65 Beispielausgabe, die unter keinen Umständen als XML gespeichert werden darf!:
66 <?xml version="1.0" encoding="UTF-8"?>
67 <PcGts xmlns="http://schema.primaresearch.org/PAGE/gts/pagecontent/2013-07-15">
68 <Page imageFilename="dummy.jpg" imageWidth="1000" imageHeight="1000">
69   <TextRegion id="r1">
70     <TextLine id="t11" custom="place {{offset:0; length:7;}} creation_place {{offset:0; length:7;}} date {{offset:8; length:9;}}
    ↪   when:28.05.1942;}} creation_date {{offset:8; length:9; when:28.05.1942;}}">
71       <Coords points="0,0 100,0 100,10 0,10"/>
72       <TextEquiv><Unicode>München 28.V.1942</Unicode></TextEquiv>
73     </TextLine>
74     <TextLine id="t12" custom="recipient {{offset:7; length:5;}} person {{offset:7; length:5;}} place {{offset:15; length:6;}}
    ↪   recipient_place {{offset:15; length:6;}}">
75       <Coords points="0,20 100,20 100,30 0,30"/>
76       <TextEquiv><Unicode>Lieber Otto, Berlin</Unicode></TextEquiv>
77     </TextLine>
78     <TextLine id="t13" custom="event {{offset:24; length:38; when:28.05.1942;}} place {{offset:42; length:7;}}">
79       <Coords points="0,40 100,40 100,50 0,50"/>
80       <TextEquiv><Unicode>Heute abend fand ein Konzert im Opernhaus in München statt, und ich</Unicode></TextEquiv>
81     </TextLine>
82     <TextLine id="t14" custom="organization {{offset:43; length:28;}} place {{offset:66; length:16;}}">
83       <Coords points="0,60 100,60 100,70 0,70"/>
84       <TextEquiv><Unicode>lauschte den himmlischen Stimmen des Männerchors Hintertuüpfingen eV.</Unicode></TextEquiv>
85     </TextLine>
86     <TextLine id="t15" custom="organization {{offset:34; length:3;}} organization {{offset:40; length:18;}}">
87       <Coords points="0,80 100,80 100,90 0,90"/>
88       <TextEquiv><Unicode>Das alles fand im Rahmen des WhW - des Winterhilfswerk statt.</Unicode></TextEquiv>
89     </TextLine>
90     <TextLine id="t16" custom="organization {{offset:50; length:17;}} place {{offset:72; length:4;}} place {{offset:83; length:6;}}">
91       <Coords points="0,100 100,100 100,110 0,110"/>
92       <TextEquiv><Unicode>Ich hoffe wir sehen uns bald bei einem Auftritt des Männerchors Murg wieder, oder in
    ↪   Hänner?</Unicode></TextEquiv>
93     </TextLine>
94     <TextLine id="t17" custom="role {{offset:14; length:14;}} person {{offset:29; length:4;}}">
95       <Coords points="0,120 100,120 100,130 0,130"/>
96       <TextEquiv><Unicode>Grüss mir den Vereinsführer Asal,</Unicode></TextEquiv>
97     </TextLine>
98     <TextLine id="t18">
99       <Coords points="0,140 100,140 100,150 0,150"/>
100      <TextEquiv><Unicode>Alles Liebe,</Unicode></TextEquiv>
101    </TextLine>
102    <TextLine id="t19" custom="author {{offset:6; length:17;}} person {{offset:6; length:17;}}">
103      <Coords points="0,160 100,160 100,170 0,170"/>
104      <TextEquiv><Unicode>Deine Lina Fingerdick</Unicode></TextEquiv>
105    </TextLine>
106    <!-- Neue Zeile für den Empfangsort -->
107    <TextLine id="t110" custom="salutation {{offset:0; length:2;}} recipient {{offset:3; length:13;}} address {{offset:18; length:21;}}
    ↪   place {{offset:41; length:4;}}">
108      <Coords points="0,180 100,180 100,190 0,190"/>
109      <TextEquiv>
110        <Unicode>An Otto Bolliger, Adolf-Hitler Platz 1, Murg</Unicode>
111      </TextEquiv>
112    </TextLine>
113  </TextRegion>
114 </Page>
115 </PcGts>
116
117 Hier ist das zu annotierende XML:
118 ...
119 {xml_content}
120 ...

```

A.4 Übersicht fehlerhaft verarbeiteter Listen in XML

Dokument ID	Dateiname	Fehlerbeschreibung
	0001_p001.xml	mismatched tag: line 279, column 18
	0002_p002.xml	mismatched tag: line 132, column 18
	0003_p003.xml	mismatched tag: line 186, column 18
	0004_p004.xml	mismatched tag: line 202, column 18
	0059_p061.xml	unclosed token: line 334, column 16
	0060_p062.xml	unclosed token: line 313, column 16
	0061_p063.xml	unclosed token: line 334, column 16
	0062_p064.xml	unclosed token: line 327, column 16
	0002_p002.xml	mismatched tag: line 240, column 18
	0003_p003.xml	mismatched tag: line 181, column 18
	0004_p004.xml	unclosed token: line 334, column 16
	0005_p005.xml	mismatched tag: line 239, column 18
	0001_p001.xml	unclosed token: line 558, column 20
	0002_p002.xml	mismatched tag: line 288, column 18
	0001_p001.xml	mismatched tag: line 300, column 18
	0003_p004.xml	mismatched tag: line 700, column 18
	0002_p003.xml	unclosed token: line 510, column 20
	0003_p005.xml	no element found: line 562, column 86
	0001_p001.xml	mismatched tag: line 220, column 18
	0002_p002.xml	duplicate attribute: line 206, column 72
	0004_p004.xml	duplicate attribute: line 381, column 72
	0008_p008.xml	duplicate attribute: line 59, column 70
	0012_p012.xml	duplicate attribute: line 38, column 70
	0014_p014.xml	unclosed token: line 585, column 16
	0019_p019.xml	duplicate attribute: line 115, column 72
	0021_p021.xml	duplicate attribute: line 220, column 72
	0023_p023.xml	duplicate attribute: line 199, column 72
	0025_p025.xml	not well-formed(invalid token): line 38, column 77
	0029_p029.xml	duplicate attribute: line 87, column 72
	0033_p033.xml	duplicate attribute: line 150, column 72
	0001_p001.xml	unclosed token: line 370, column 16
	0002_p002.xml	unclosed token: line 313, column 16
	0001_p001.xml	unclosed token: line 376, column 16
	0002_p002.xml	unclosed token: line 333, column 16
	0002_p002.xml	unclosed token: line 348, column 16
	0003_p003.xml	unclosed token: line 361, column 16
	0002_p002.xml	unclosed token: line 320, column 16

Tabelle 2: Übersicht der fehlerhafter Listen-XML.